SKALA

Cloud

Cloud기반 Container 이해

2026.02



본 문서는 SK(주)C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C



목차

1일차

- 환경 구성하기
- 컨테이너의 이해
- 나의 데이터베이스 만들기
- 컨테이너 기본 활용
- Dockerfile 작성
- 컨테이너 볼륨 연결하기
- CMD 명령 최적화
- [실습] 나의 서비스 컨테이너 만들기

2일차

- 컨테이너 이미지 구조
- 컨테이너 구조
- 컨테이너 플랫폼
- docker-compose 이해
- docker-compose 명령어
- 조별 토론하기
- Quiz
- [실습] 나의 서비스 docker-compose화하기



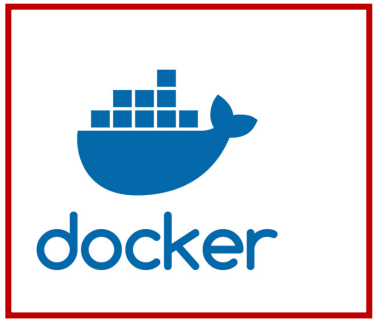
환경 구성

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

컨테이너 관리 도구

CICD 파이프라인 내에서 컨테이너를 빌드를 위한 도구





컨테이너 플랫폼

BuildKit



buildah

Kaniko

컨테이너 빌드 도구

containerd



cri-o

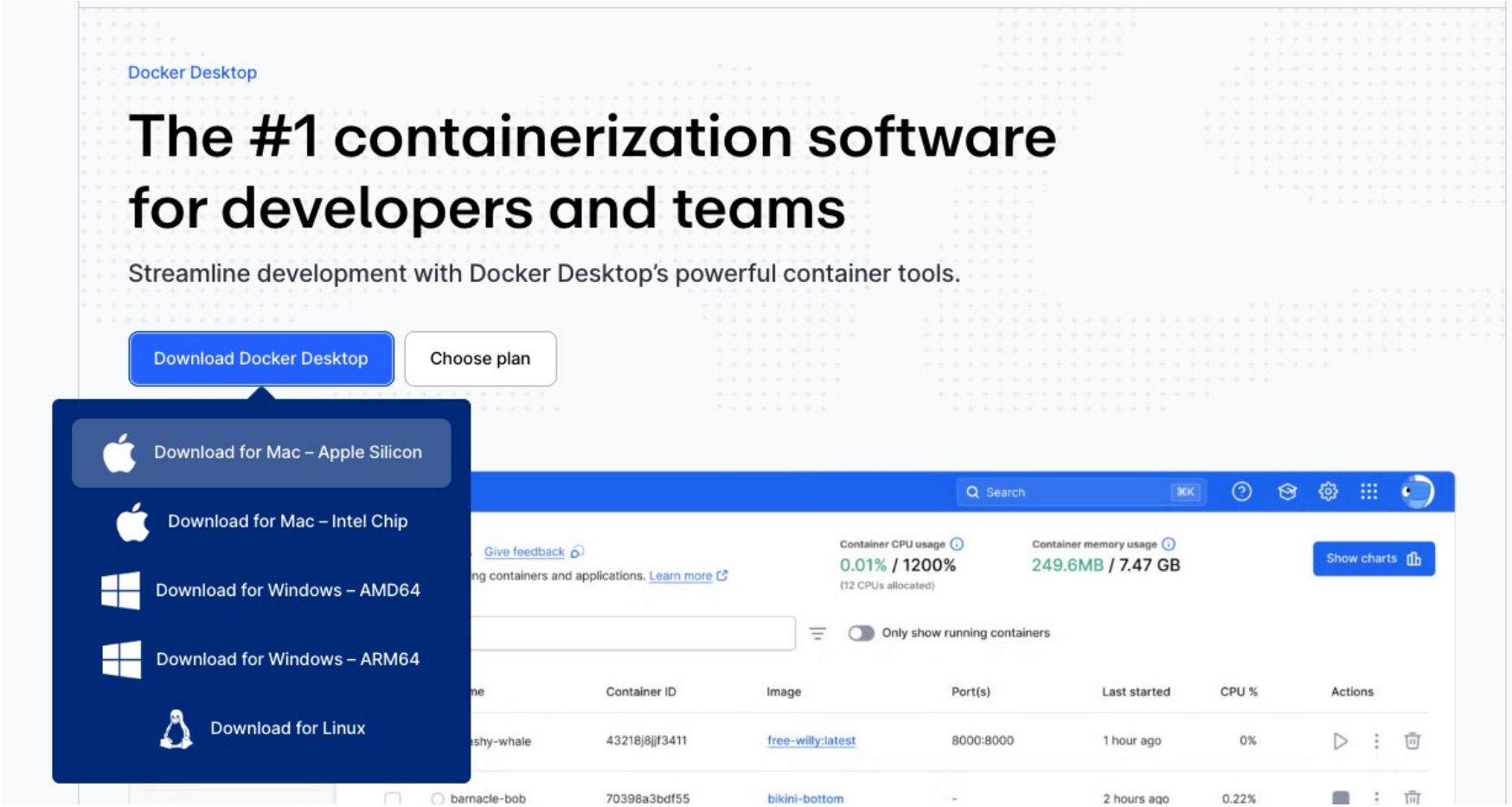
컨테이너 런타임 도구

Docker DeskTop 설치하기

Docker 프로그램은 아래 링크로 가서 Docker Desktop을 설치합니다

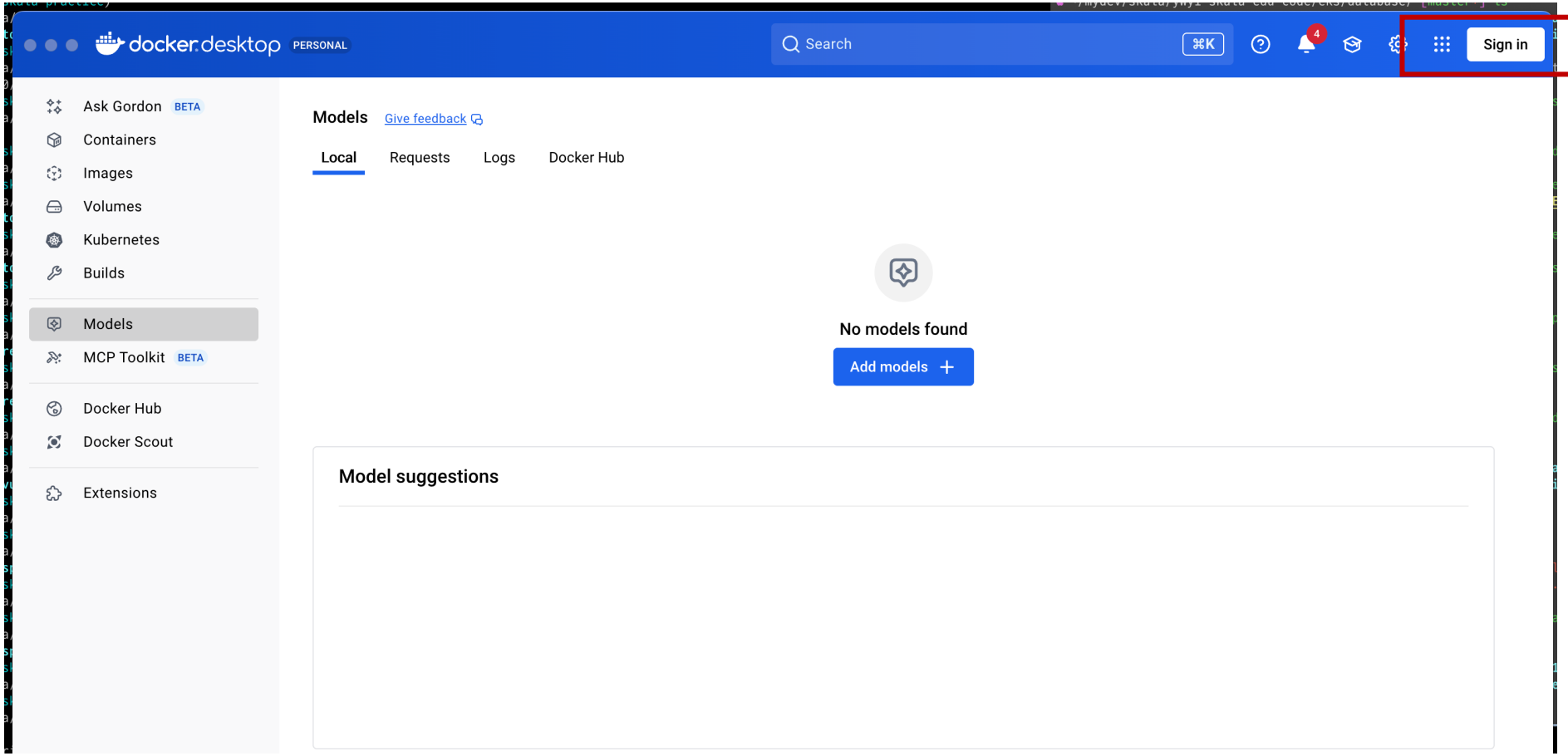
A. 접속

https://www.docker.com/products/docker-desktop/



Docker Login 하기

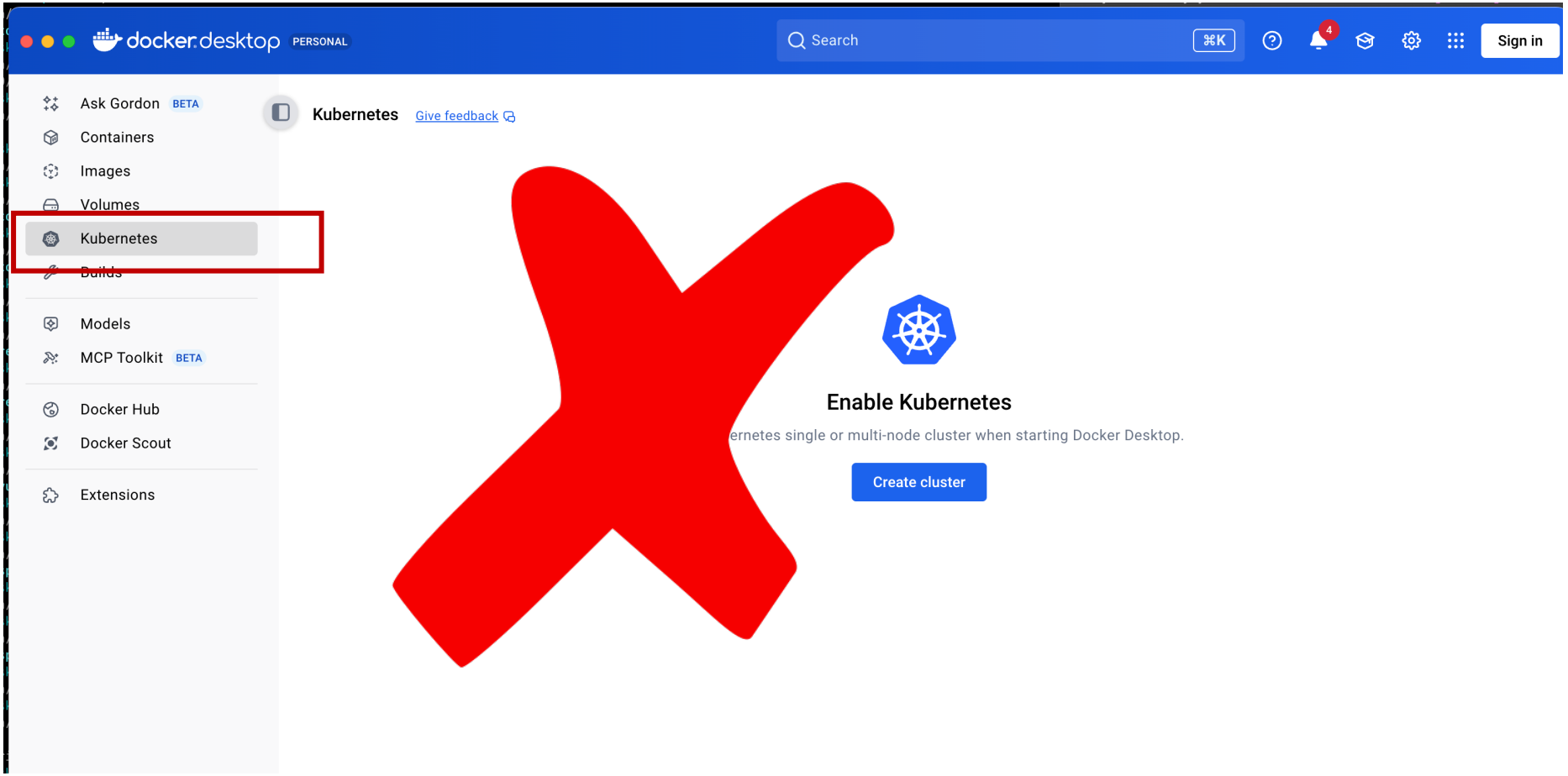
Docker 를 기반으로 Image Pull 하는 경우 로그인 하지 않으면 제약 사항이 있으므로 꼭 로그인 하기



[주의 사항] Docker의 Kubernetes는 설치하지 말자

Docker 의 Kubernets를 설치 시 kubernetes 컨트롤러 플래인 컴포넌트를 컨테이너로 기동 (minikube)

컨테이너의 성능적인 이슈가 발생할 수 있으므로 가능하면 설치하지 말자



실습 코드 다운로드

git hub로 부터 실습 코드를 다운로드 한다.

실습 디렉토리 구성하기

```
# home 디렉토리로 이동
cd

mkdir workspace
cd workspace

mkdir cloud
cd cloud
```

Container 실습 코드 다운로드

```
git clone https://github.com/himang10/skala-container.git

cd skala-container
```

[확인 사항] 아래와 같은 에러가 발생하는 경우

docker desktop을 종료후 재 실행 또는 mac book 재 부팅

```
Cannot connect to the Docker daemon at unix:///Users/himang10/.docker/run/docker.sock. Is the docker daemon running?
```

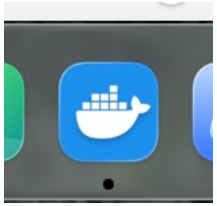
또는 컨테이너가 종료 또는 삭제가 안되는 경우

docker 프로세스를 모두 종료 하기

```
> sudo pkill -9 -f -i docker
```

패스워드는 맥북 로그인 패스워드

docker desktop 재실행

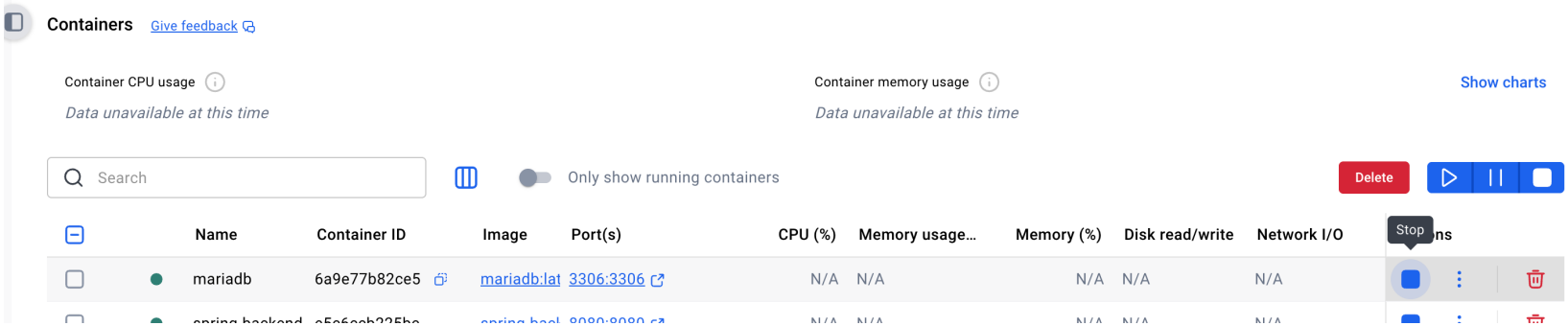


[확인 사항] 아래와 같은 에러가 발생하는 경우

8080 포트를 선점하고 있는 프로세스가 실행중

```
Bind for 0.0.0.0:8080 failed: port is already allocated
```

1. docker desktop에서 Container 중지



2. 아래의 명령 실행

```
> lsof -ti :8080 | xargs kill -9
```



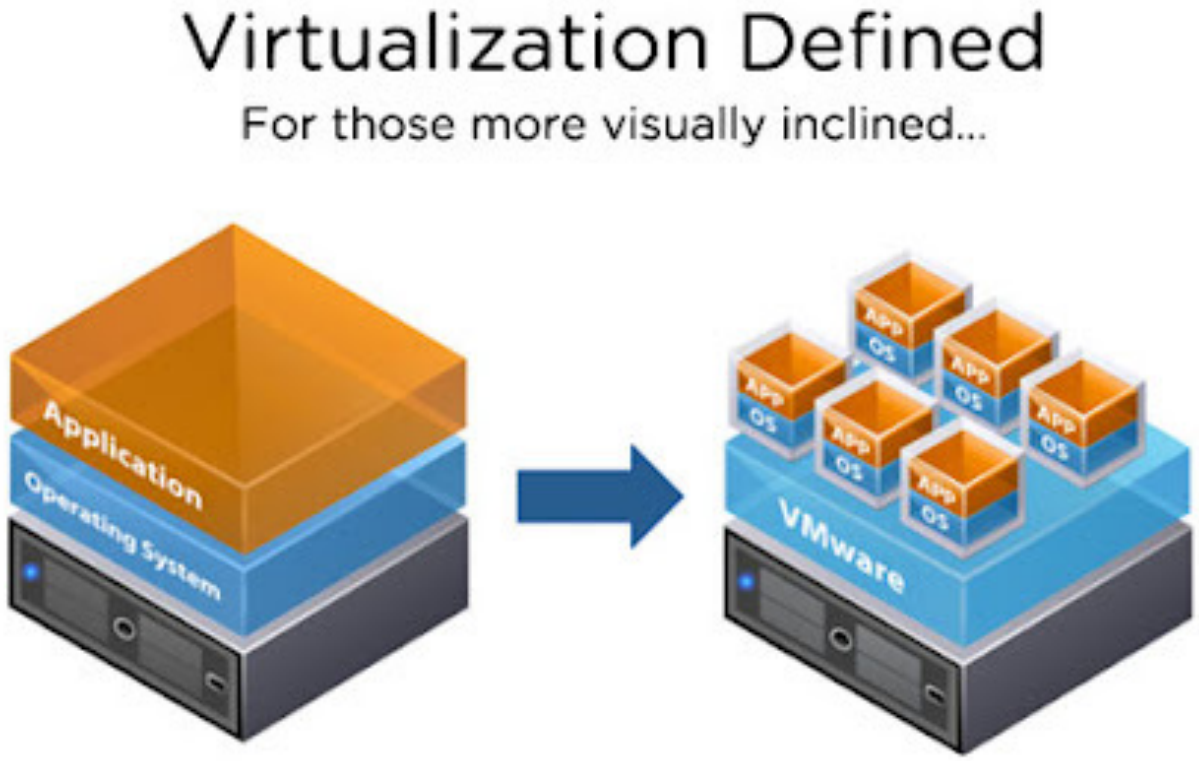
컨테이너의 이해

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

가상화 (Virualization)

물리적인 하드웨어를 추상화하여 소프트웨어로 구현하는 기술
가상화 기술을 활용하면 한 컴퓨터를 독립된 여러 대의 컴퓨터 처럼 사용할 수 있도록 지원



일반적 물리 서버 구조

High End : 수 억 ~ 수십억
Mid-Range: 수억

평균 6% 사용률

가상 서버로 분할

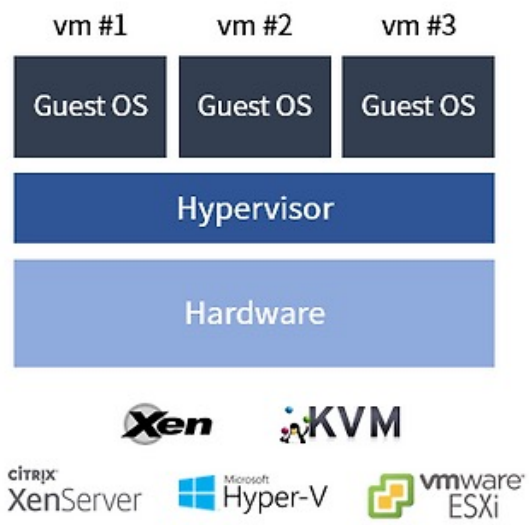
가상화를 위한 Hypervisor

물리 서버 위에서 여러 개의 가상 머신(VM, Virtual Machine)을 생성·실행·관리하는 소프트웨어 계층
즉, 하나의 컴퓨터 자원을 나눠서 여러 개의 '독립된 컴퓨터'처럼 운영할 수 있게 해주는 가상화 엔진

역할

- 물리 하드웨어(CPU, Memory, Disk, NIC 등)를 관리
- VM을 생성 / 삭제 / Snapshot / Migration 관리
- 각 VM에게 적절한 자원 할당
- VM 간 자원 충돌을 방지
- VM이 요청하는 하드웨어 명령을 중재 및 스케줄

Type-1
Native / Bare-metal Hypervisor



Type-2
Hosted Hypervisor



Hypervisor 유형

Type-1 (Bare-metal Hypervisor)

- 경량 전용 OS, 전용 하드웨어, 원격관리 도구 지원
- VM 전용 서버 구성 시 활용
- OS 오버헤드 없고, 높은 성능과 안정성 제공
- 클라우드 IDC, 데이터 센터용

솔루션

- VMware ESXi
- Microsoft Hyper-V
- KVM (리눅스 커널 포함 형태)
- Xen

```
AWS EC2 서버
└── ESXi Hypervisor
    ├── 웹서버 VM (24시간 가동)
    ├── DB 서버 VM (24시간 가동)
    └── 캐시 서버 VM (24시간 가동)
```

Type-2 (Hosted Hypervisor)

- 기존 장비에 가상화 환경 구성시 활용
- 기존 운영체제에서 동작
- OS 오버헤드 존재
- 데스크탑 테스트/개발용

솔루션

- VMware Workstation
- Oracle VirtualBox
- Parallels Desktop

```
개발자 맥북
└── macOS (주 작업 환경)
    ├── Chrome, Slack, VSCode 사용
    └── VirtualBox 실행
        └── Ubuntu VM (테스트용, 필요할 때만)
```

[참고] Windows Hypervisor: Hyper-V

Type 1 방식으로 적용하되 관리 OS 기능과 관리 도구를 별도의 VM을 구성해서 전담하도록 구성

- Micorsoft는 Windows OS를 관리권한을 가지는 Root Parition으로 분리하여
- Windows는 OS 로 , WSL은 VM 관리 도구로써 제공하는 방식 적용

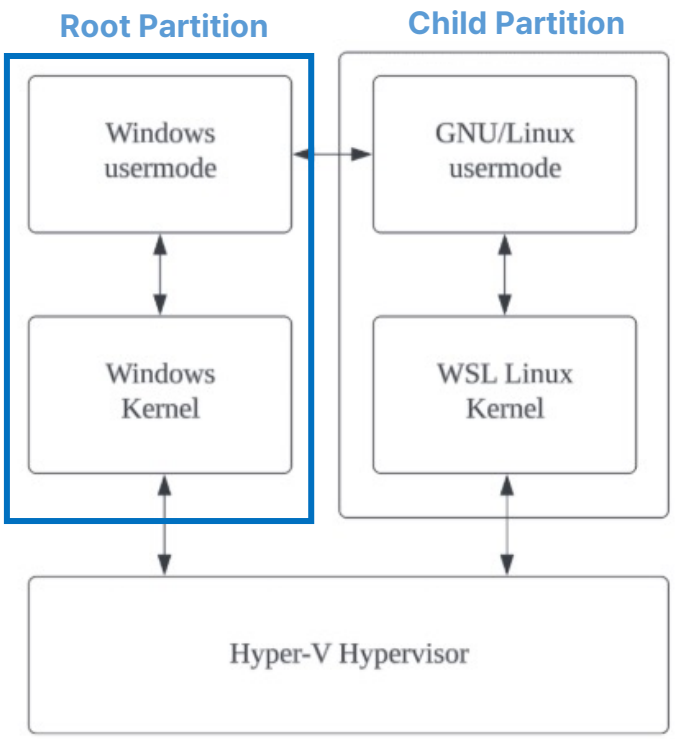
ESXi = 완전체

```
├── 하이퍼바이저
├── OS 커널 기능 (내장)
└── 관리 도구 (내장)
```

Hyper-V = 분리형

```
├── 하이퍼바이저 (순수 가상화만)
└── Root Partition (Windows)
    └── OS 커널 기능 + 관리 도구 (WSL)
```

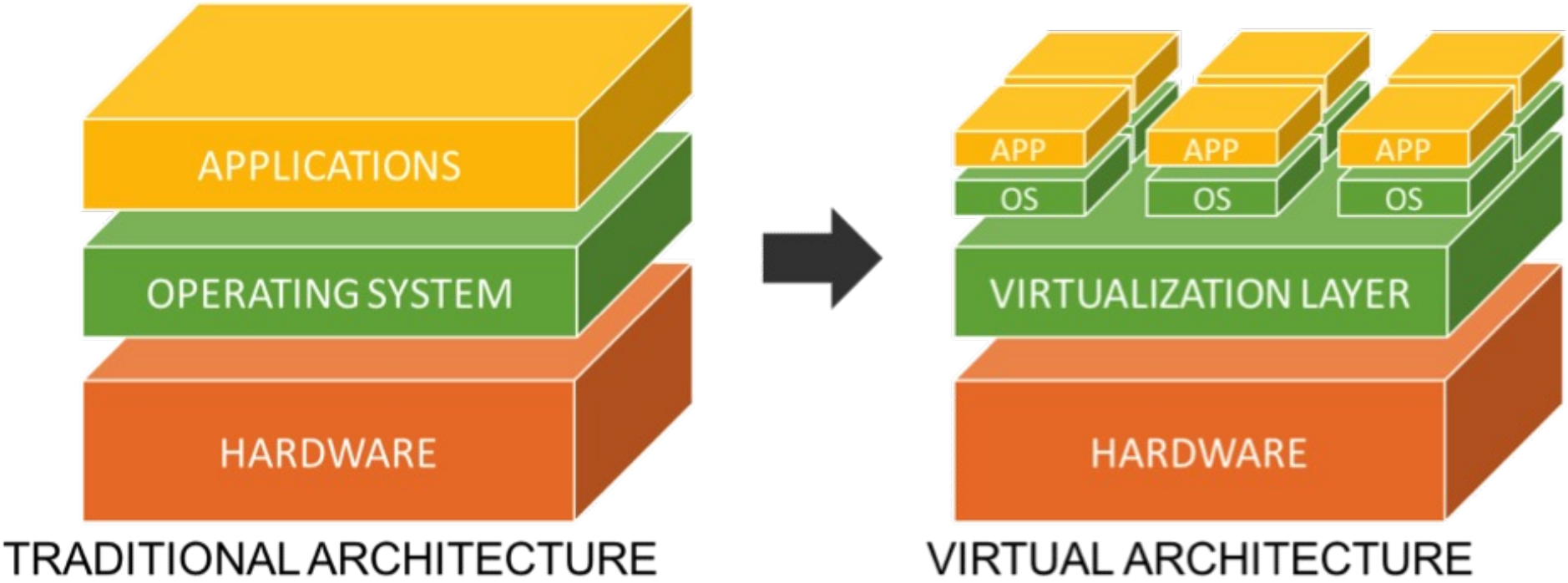
하이퍼바이저는 실제 하드웨어를 직접 노출하지 않고,
이를 추상화한 "가상 하드웨어(virtual hardware)"를
각 Guest OS에 제공



VM 구조의 문제점

Virtualization Machine은 물리 서버를 **서비스화하여 동적 할당을 지원**

But, 애플리케이션의 설치 및 운영은 기존 **전통적인 물리 서버 환경과 동일**한 구조



VM 구조의 고질적 문제

거대한 이미지 크기

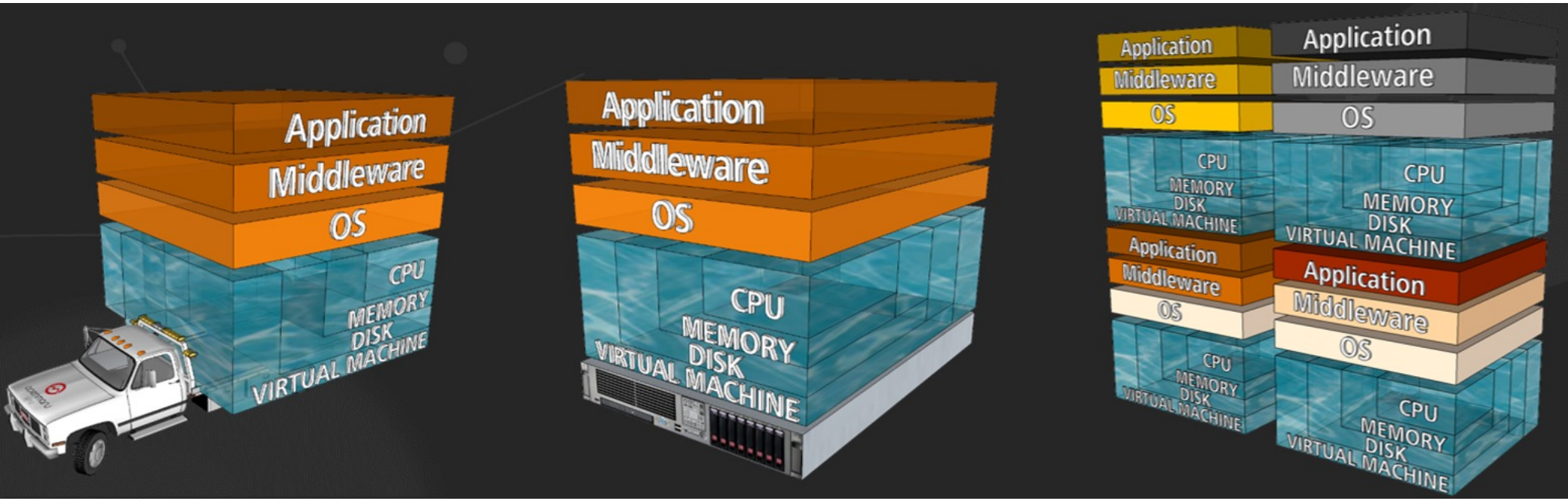
VM 이미지는 OS, 가상 하드웨어 등 포함
사이즈 : 수 GB ~ 수십 GB

느린 시작 시간

부팅 시 Hypervisor-OS-M/W-Appl 까지 단계적 실행
시간: 수십 초 ~ 수분

VM 간 환경 불일치

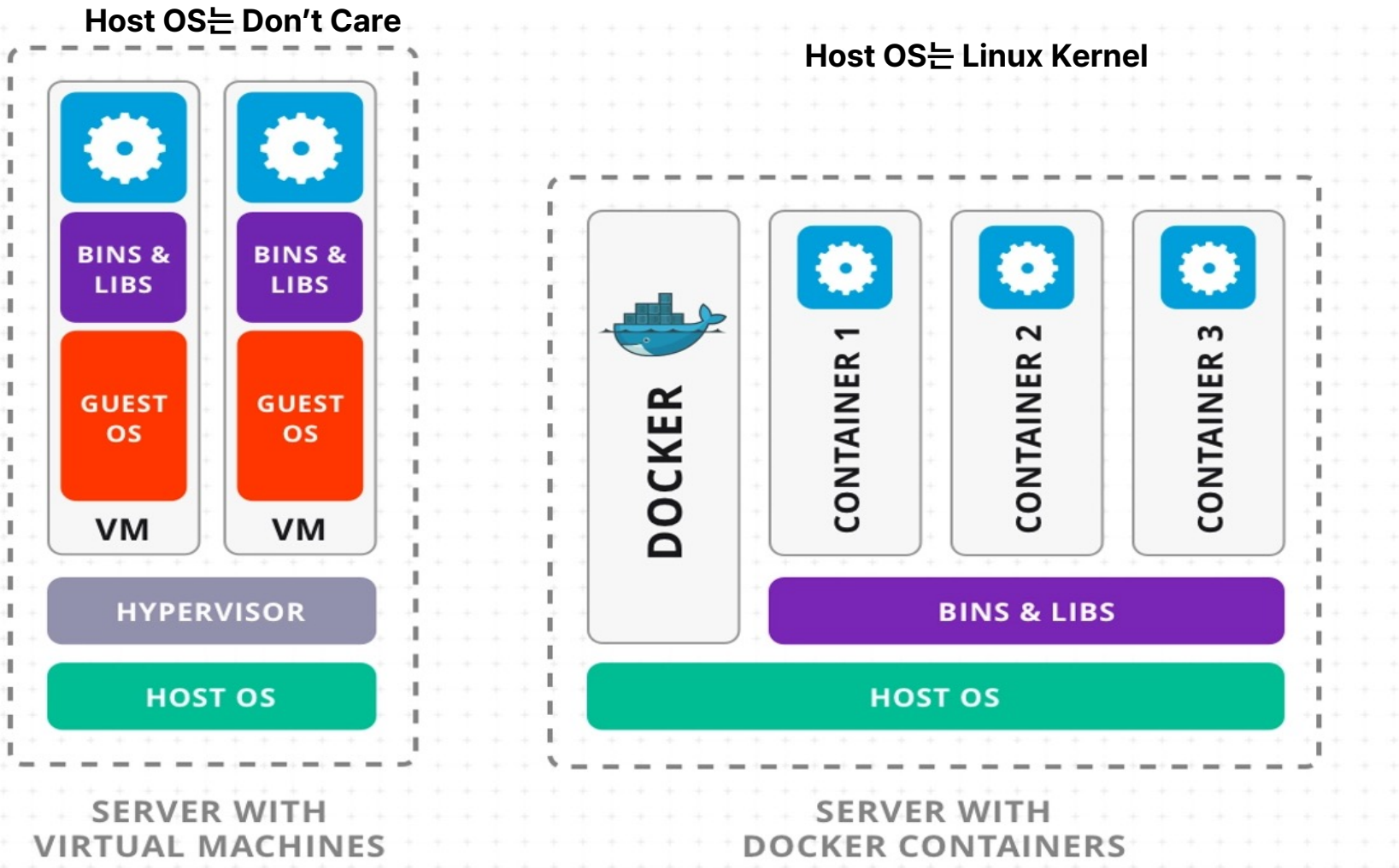
VM은 OS 까지만,
Appl 실행을 위한 환경 구성은 별로 필요
이로인한 환경 불일치 문제 발생



Virtualization Machine은 물리 서버를 **서비스화하여 동적 할당을 지원**
But, 애플리케이션의 설치 및 운영은 기존 **전통적인 물리 서버 환경과 동일**한 구조

작은 실행 단위 컨테이너화 도입

- Linux Kernel 환경으로 OS 환경 통합
- 애플리케이션과 실행 환경을 통합한 이미지로 생성 관리 (배포 단위에서 OS 이미지 제거)



컨테이너를 통한 VM 문제 해결

작은 이미지 크기

Appl. 실행을 위한 파일/라이브러리만 이미지화
사이즈: 수십 MB ~ 수백 MB

빠른 시작 시간

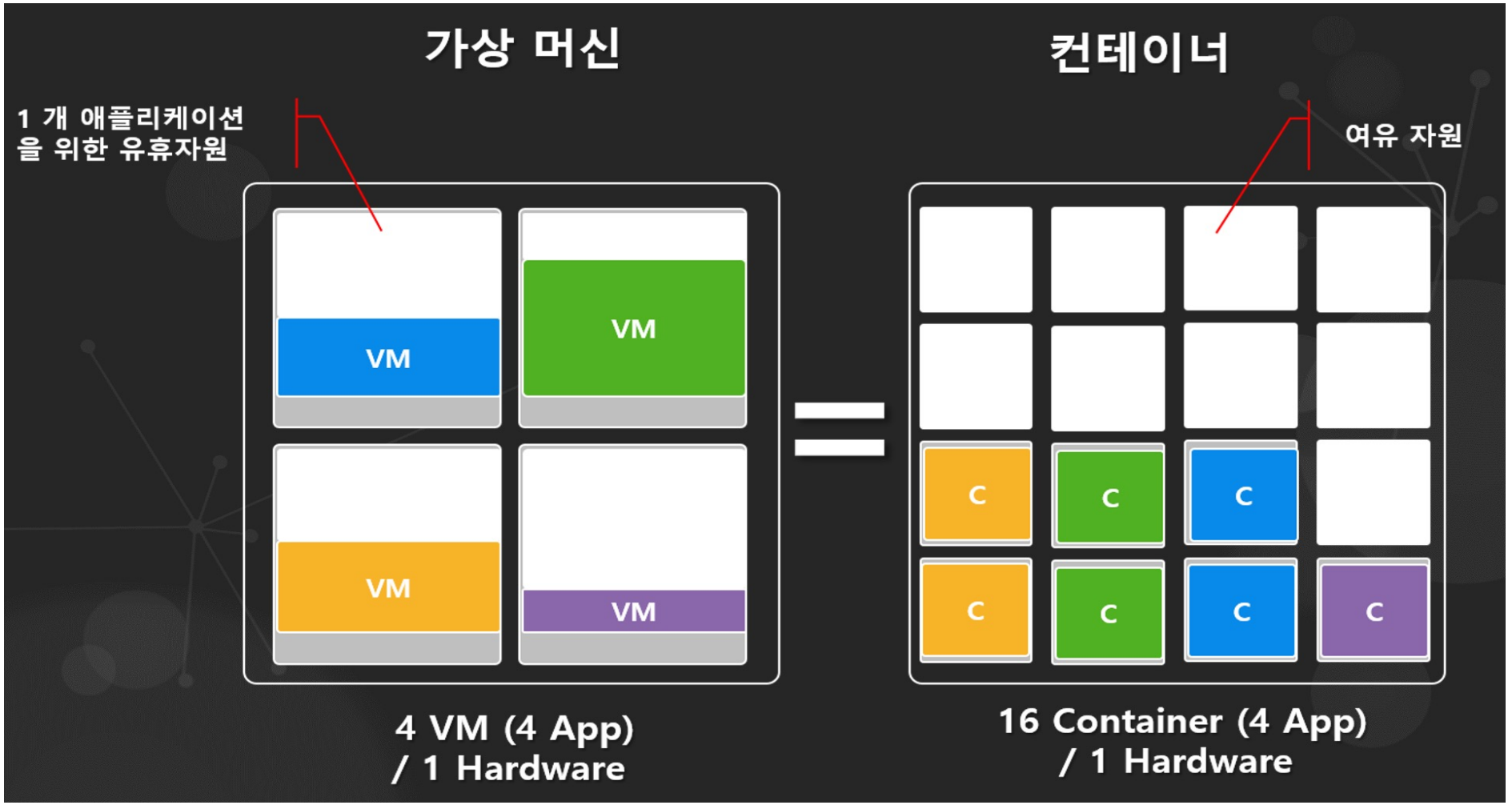
컨테이너는 Kernel 공유하므로
별도 부팅 시간 없음
실행 시간: 프로세스 실행 (밀리초, 초)

동일 실행 환경(높은 이동성)

Appl. 중심 의존관계 파일/라이브러리를 컨테이너 이미지로 포함
동일 Linux Kernel에서 동일 실행

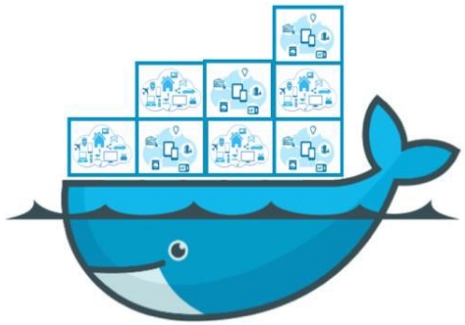


VM 대비 높은 개발 집적도



컨테이너 특징

비교 항목	기존 VM의 한계	컨테이너 도입 후 변화
이식성 (Portability)	환경마다 OS 패키지·라이브러리 버전이 달라 "내 PC에선 되는데 서버에선 에러" 발생	앱, 의존성 라이브러리를 단일 패키지로 묶어 로컬·클라우드 어디서든 100% 동일 실행
이미지 크기	Guest OS 전체를 포함해 이미지 크기가 **수 GB ~ 수십 GB**에 달함	OS 커널을 제외, 앱 구동에 필요한 파일만 담아 **수십 MB ~ 수백 MB** 수준
기동 속도	OS 부팅 및 가상 하드웨어 초기화로 기동에 **수십 초 ~ 수 분** 소요	프로세스 실행 수준으로 **수 밀리초 ~ 수 초**
자원 효율 & 집적도	각 VM마다 OS 구동을 위한 고정 CPU/메모리 오버헤드가 발생해 자원 낭비 큼	호스트 커널을 공유, 필요한 리소스만 소비



[참고] VM 과의 차이점

	가상 서버	컨테이너
가상화	서버 가상화, VM 간 격리	호스트 자원 공유 네임 스페이스를 통해 앱 간 격리
이미지화	OS와 가상 디바이스 등도 포함되어 **사이즈가 큼**	필요한 앱과 bin/lib만 포함되어 **사이즈가 매우 작음**
플랫폼간 이식성	동일 하이퍼바이저 간 only	**이기종 물리, 가상, 클라우드 간**
이기종 OS 사용	가능	불가능 (Host OS와 동일한 OS 계열이어야 됨)
부팅시간	수십초~수분	수 밀리초~수초
개발 환경 구축	수작업으로 앱 설치 및 삭제 필요 보통 하루 이상 소요	환경 구축의 수작업 요소 제거 **수 분~수 십분 소요**
직접도	물리 서버당 4~6 VM	물리 서버당 20~30 컨테이너
앱 확장성	Low (수작업 요소 많음)	High (완전 자동화 가능)
OS Cost	VM 당	Host 당
베어메탈 대비 성능	50~80%	98%
자원 할당	CPU, Memory, GPU, 네트워크, Storage 등	CPU, Memory, GPU, 네트워크, Storage 등

컨테이너 빌드 방법

- Docker는 image를 만들기 위해 Dockerfile에 DSL(Domain-Specific Language)로 이미지 생성 레이어 기술
- Dockerfile은 무엇을 이미지화해야하는지를 선언적으로 정의




```
FROM python:3.10-alpine

RUN apk add --no-cache bash curl gcc musl-dev linux-headers jq

# FastAPI 및 기타 라이브러리 설치
RUN pip install fastapi uvicorn psutil python-multipart prometheus-client

COPY fastserver.py fastserver.py

CMD ["python3", "fastserver.py"]
```



데이터베이스 만들기

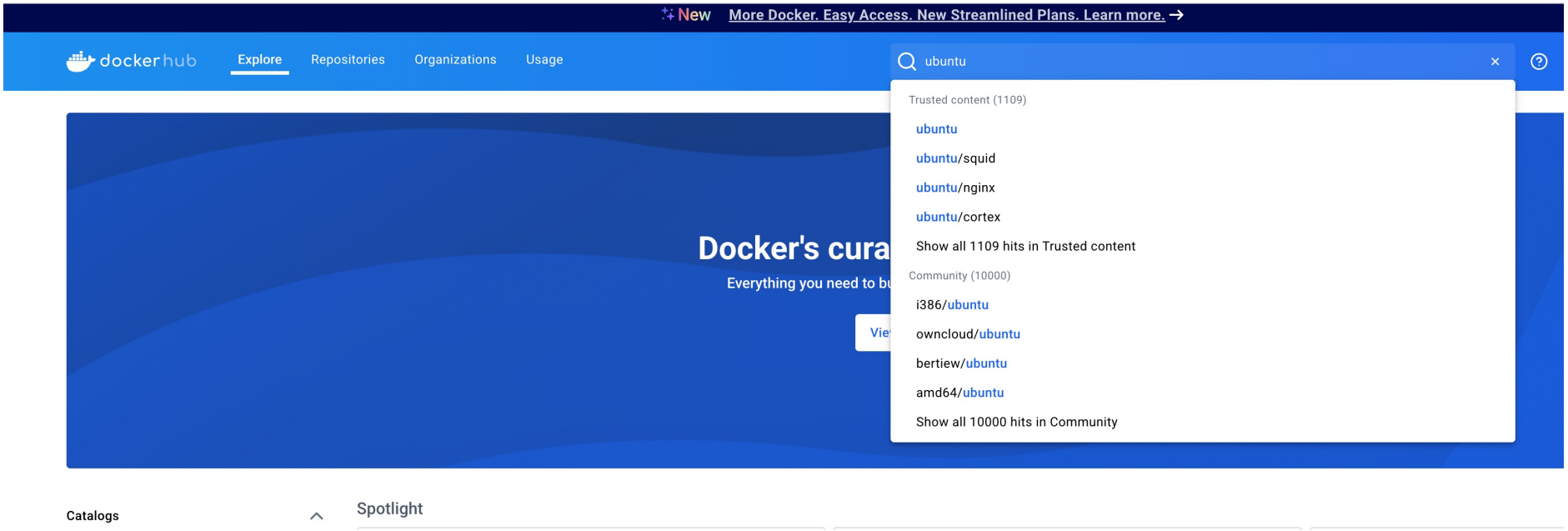
본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Docker hub 접속하기

컨테이너 이미지 저장하는 오픈 Hub

https://hub.docker.com



컨테이너로 maria db 실행하기

mariadb 를 Container로 로컬에서 실행해보자

참고: 01.answer-code/01.mariadb/run-mariadb.sh

```
> docker network inspect skala >/dev/null 2>&1 || \
  docker network create --driver bridge skala

> docker run -d \
  --name mariadb \
  -e MYSQL_ROOT_PASSWORD=password \
  -e MYSQL_DATABASE=skala \
  -e MYSQL_USER=user \
  -e MYSQL_PASSWORD=password \
  --network skala \
  -p 3306:3306 \
 mariadb:latest
>  docker ps
```

컨테이너로 Database 실행하기

컨테이너 내부로 접속하기

```
> docker exec –it mariadb /bin/bash
# mariadb   -u  root   -p
Enter password:
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 4
Server version: 11.6.2-MariaDB-ubu2404 mariadb.org binary distribution
Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
MariaDB [(none)]> SHOW DATABASES;
+--------------------+
| Database           |
+--------------------+
| information_schema |
| mysql              |
| performance_schema |
| skala              |
| sys                |
+--------------------+

MariaDB [(none)]> USE skala;
Database changed
```

```
MariaDB [skala]> CREATE TABLE key_values ( id INT AUTO_INCREMENT
PRIMARY KEY, key_column VARCHAR(255) NOT NULL UNIQUE, value_column
TEXT NOT NULL );

MariaDB [skala]> SHOW TABLES;
+-----------------+
| Tables_in_skala |
+-----------------+
| key_value       |
+-----------------+
MariaDB [skala]> INSERT INTO key_value (key_column, value_column) VALUES
('example_key', 'example_value' );

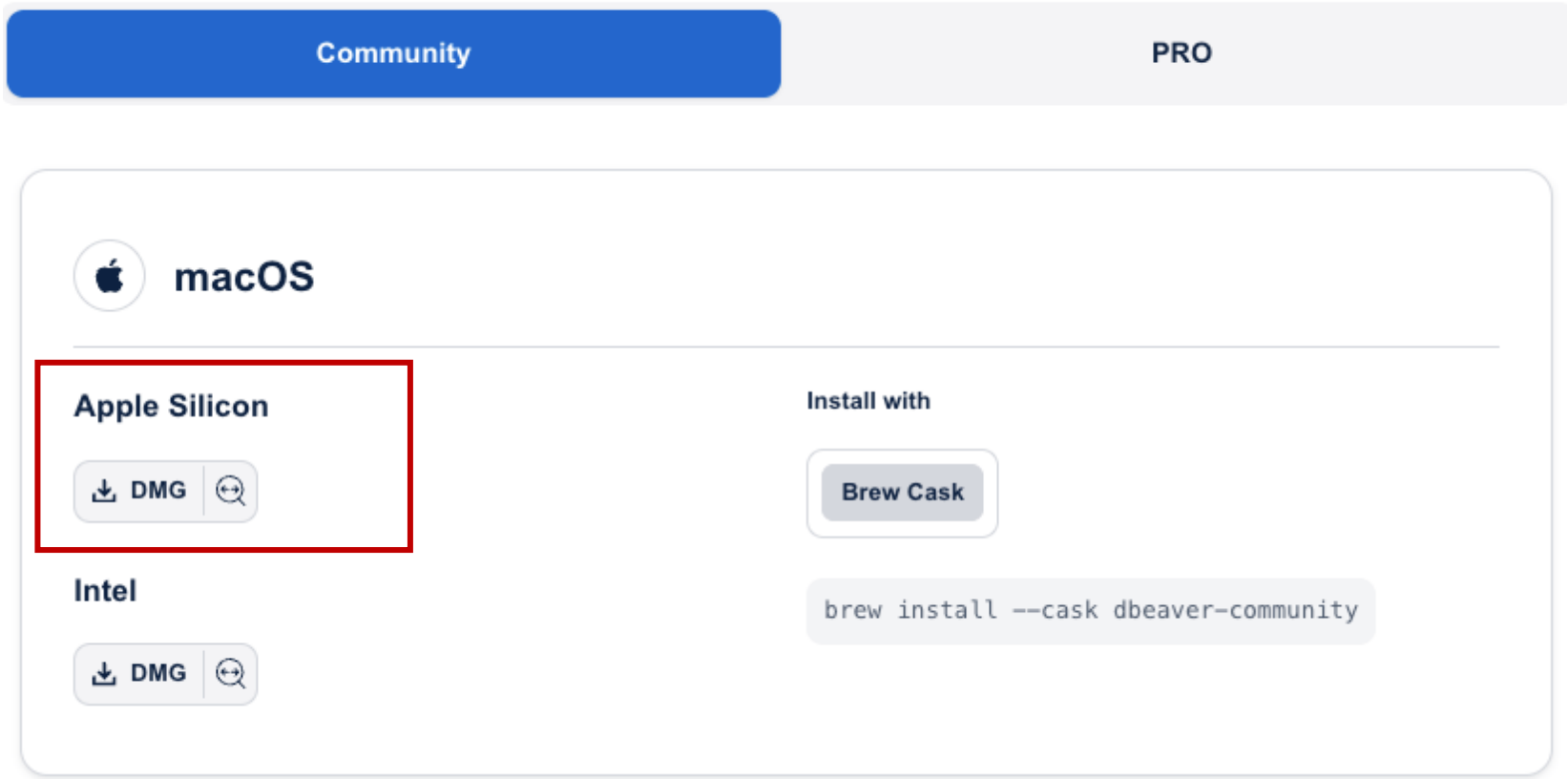
MariaDB [skala]> SELECT * FROM key_value;
+----+-------------+---------------+
| id | key_column  | value_column  |
+----+-------------+---------------+
|  1 | example_key | example_value |
+----+-------------+---------------+

MariaDB [skala]> exit
Bye
root@himang10-skala-pro-docker-rde-0:/# exit
```

DBeaver 설치하기

DB 접속 및 관리를 위한 Dbeaver를 설치해보자

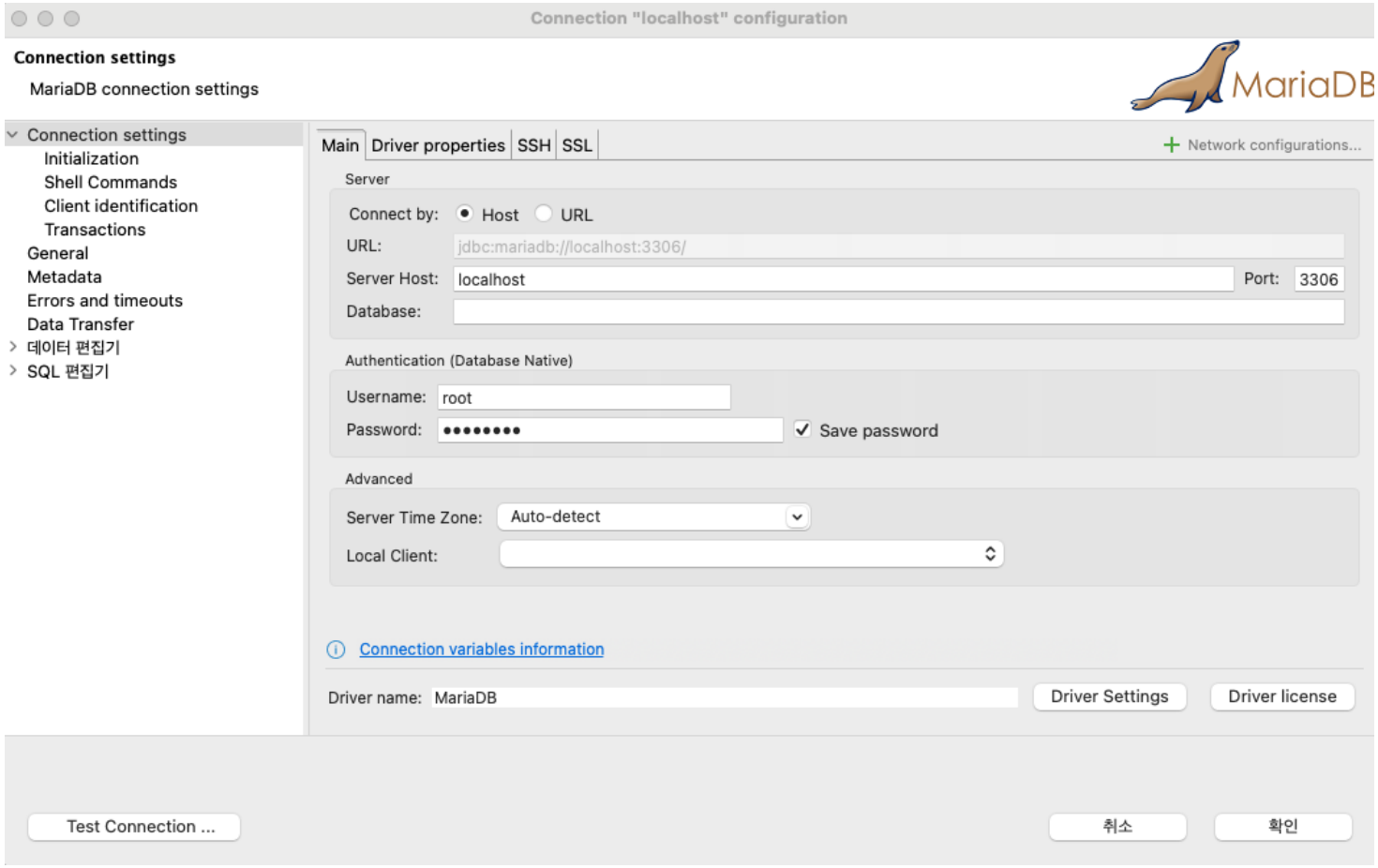
- https://dbeaver.io/download/



컨테이너로 Database 실행하기

Dbeaver 를 통해 접속해보기

- localhost:3306
- user : root
- pwd: password



컨테이너 삭제 하기

실행되고 있는 컨테이너를 종료하고 이미지를 삭제해보지

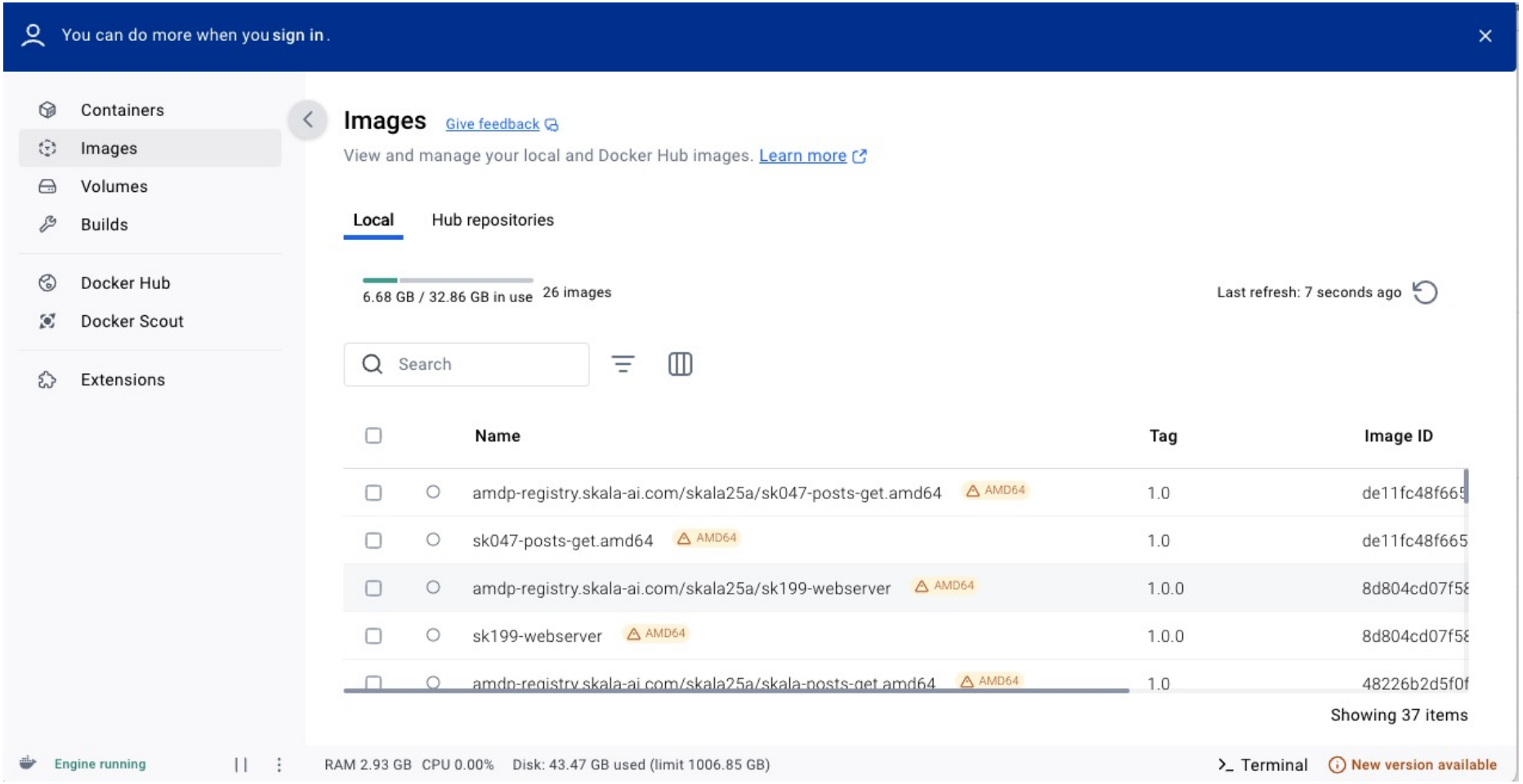
```
> docker ps
 CONTAINER ID  IMAGE                          COMMAND            CREATED        STATUS
be913f24c917  docker.io/library/mariadb:latest  mariadbd           53 minutes ago
> docker stop be913f24c917
> docker image prune -a
WARNING! This will remove all non running containers.
Are you sure you want to continue? [y/N] y
a14a76483ae1832de7700772028dd72025940b47f955f2e6ee4076a1f5e42f6f
be913f24c917e8b382faea3bf1bbdef14c8df860d87a9863782700d993f661b6
```

docker ps 로 컨테이너가 보이지 않으면

> docker ps -a

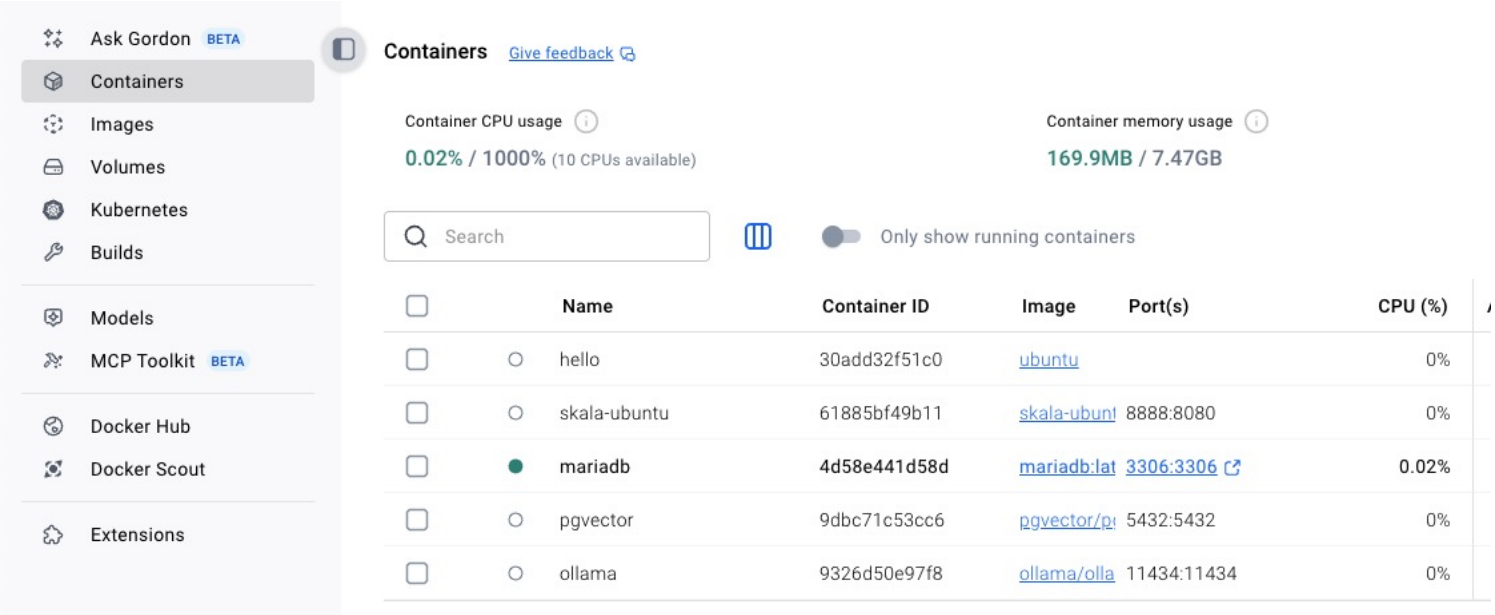
Docker Desktop

이미지 관리



Docker Desktop

컨테이너 관리



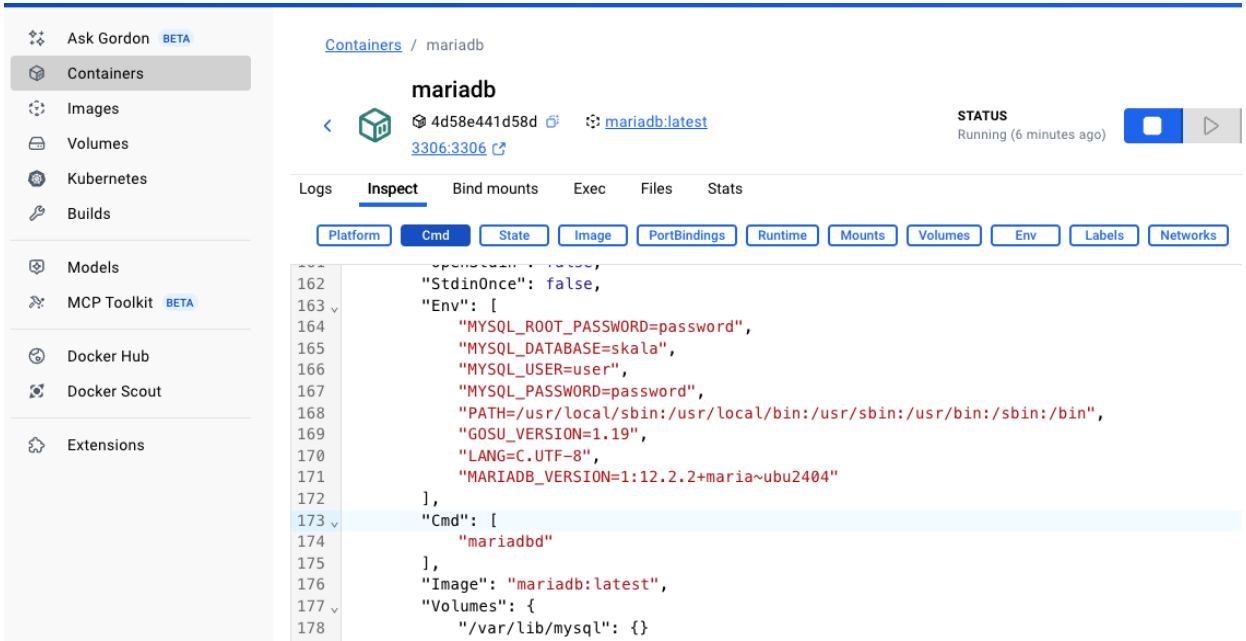
Docker Desktop

docker inspect: docker 가 관리하는 컨테이너 또는 이미지의 메타 데이터 전체를 Json 형태로 제공

Docker Engine 관점의 종합 정보

- 컨테이너 ID
- 이미지 정보
- 환경변수
- 네트워크 설정
- 마운트 정보
- 상태 정보
- 실행 명령
- 로그 경로
- restart 정책
- Docker가 내부적으로 관리하는 설정

Desktop



CMD 에서 Inspection 정보 검색

```
> docker ps
> docker inspect ${Container ID}
```

CLI 명령어로 환경 Clear 하기

기동중인 컨테이너 중지 및 삭제하기

```
> docker ps
CONTAINER ID   IMAGE            COMMAND              CREATED        STATUS         PORTS                NAMES
75432fbccc9b   mariadb:latest   "docker-entrs..."    2 sec ago      Up 2 sec       3306/tcp             mariadb


> docker stop  {{CONTAINER ID or NAMES}}  ex) docker stop 75432fbccc9b
> docker rm 75432fbccc9b
```

기존의 이미지, 네트워크, 디스크 정보를 먼저 클리어 해보자

```
# 사용하지 않는 컨테이너 리소스 삭제 (stop container, 미 사용 network, 빌드 캐시, 미 실행 이미지)
 > docker system prune -a

# 미 사용 볼륨 강제 삭제
> docker volume prune -f
```

컨테이너 기본활용



본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

도커 명령



✓ 도커의 명령은 docker <명령> 형식

예) docker run, docker push

✓ 항상 root 권한으로 실행

컨테이너 이미지 라이프사이클

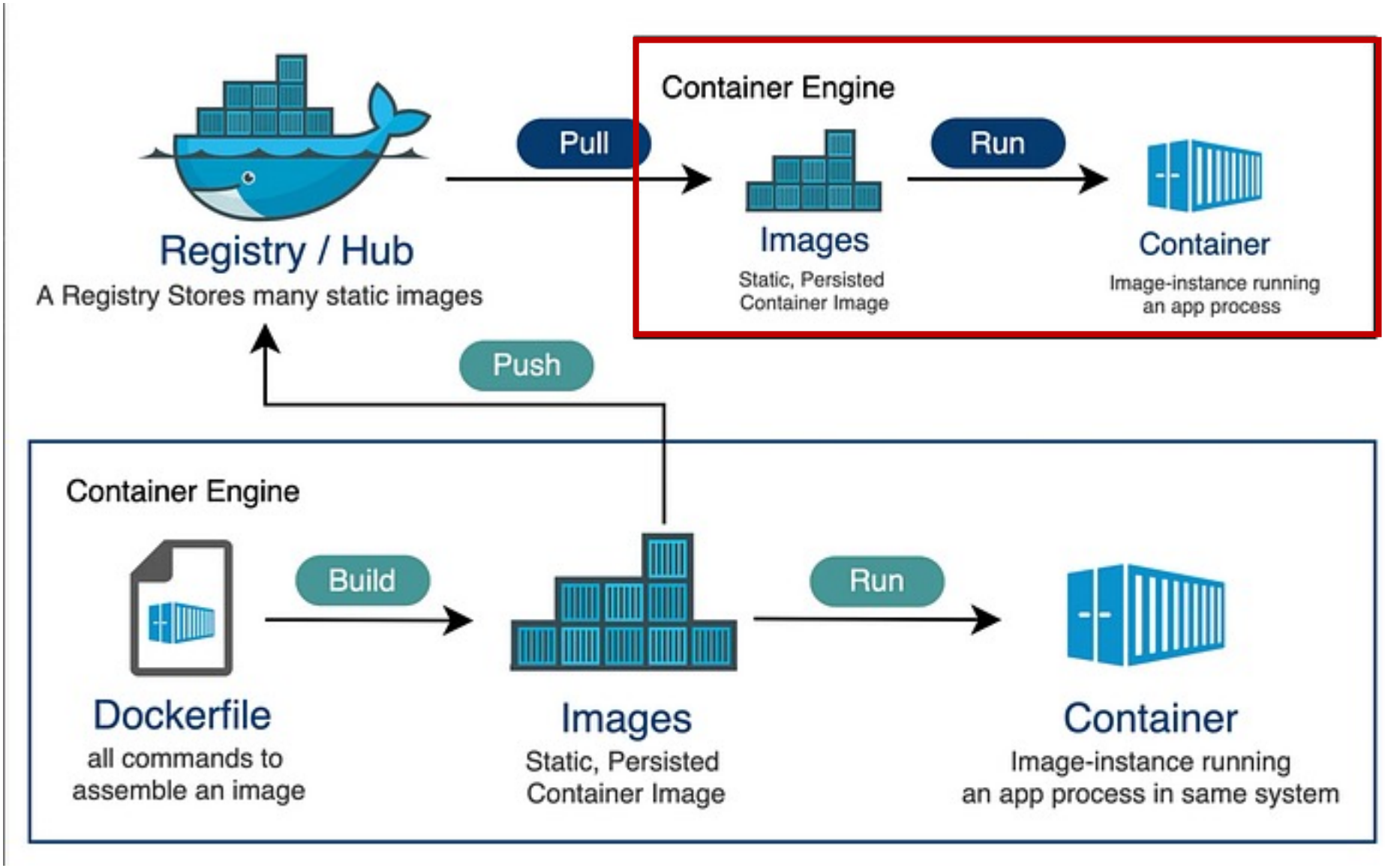
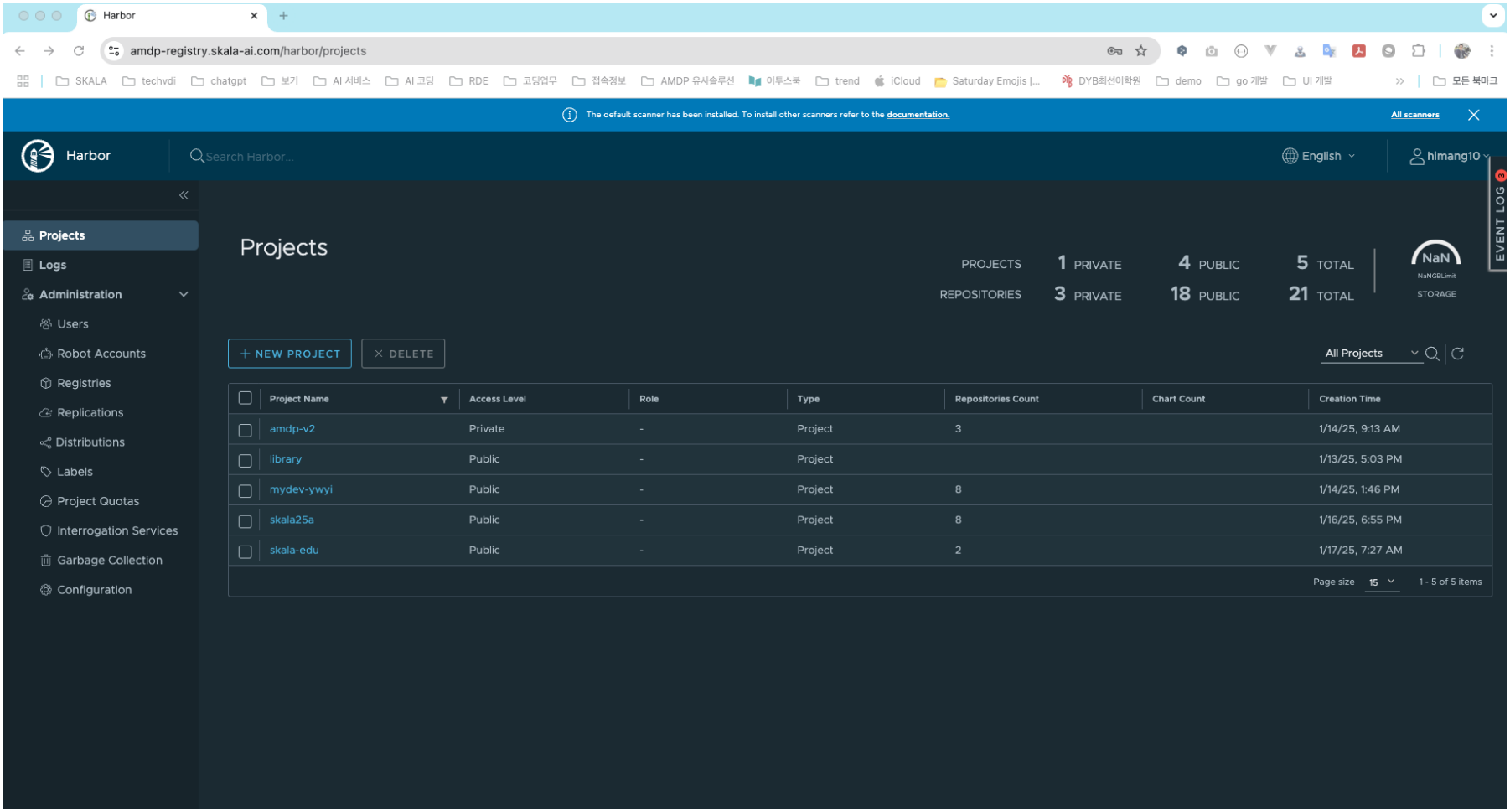


Image Registory

Private 도커 레지스트리!

본 과정에서는 Docker Hub를 사용하지 않고 Private Registry 를 사용

https://skala-registry.skala-ai.com/



Docker hub 접속정보

https://skala-registry.skala-ai.com

console 접속 정보

ID	skala26b-st
PWD	Skala26b-st!23$

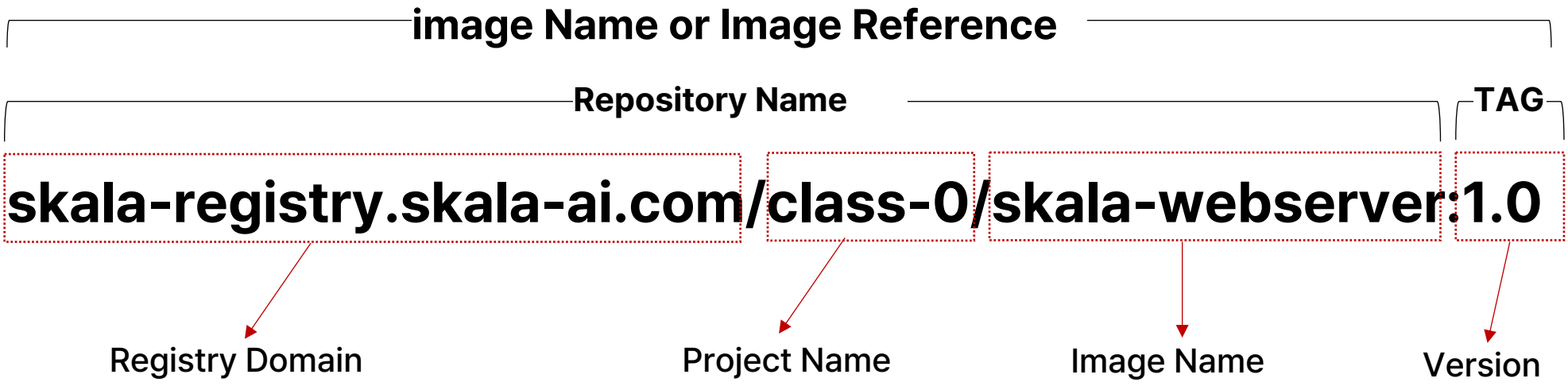
Image Push 정보

ID	robot$skala26b
PWD	oiPrruKtwhievCN1jFOLeZtI3Ge71EcD

컨테이너 기본 활용

❖ Container Images 저장을 위한 Repository 구조

```
> docker images
REPOSITORY                                                TAG       IMAGE ID       CREATED      SIZE
skala-registry.skala-ai.com/class-0/skala-webserver   2.0       b66bb516ce6d   9 days ago   569MB
skala-registry.skala-ai.com/class-0/skala-webserver   1.0       ed8a42c68454   9 days ago   831MB
```



Regisotry

여러 이미지를 저장·제공하는 **전체 서비스/서버**

Repository

Registry 안에서 특정 이미지 계열을 관리하는 **개별 저장소 단위**

[참고] docker hub의 image 이름은 ?

docker hub에 저장되는 image 이름

nginx → **docker.io/library**/nginx:latest

그러면 왜? nginx:latest만 하면 가져올까?

registry이름과 project 이름이 없는 경우: default에서 검색 (docker는 내장)

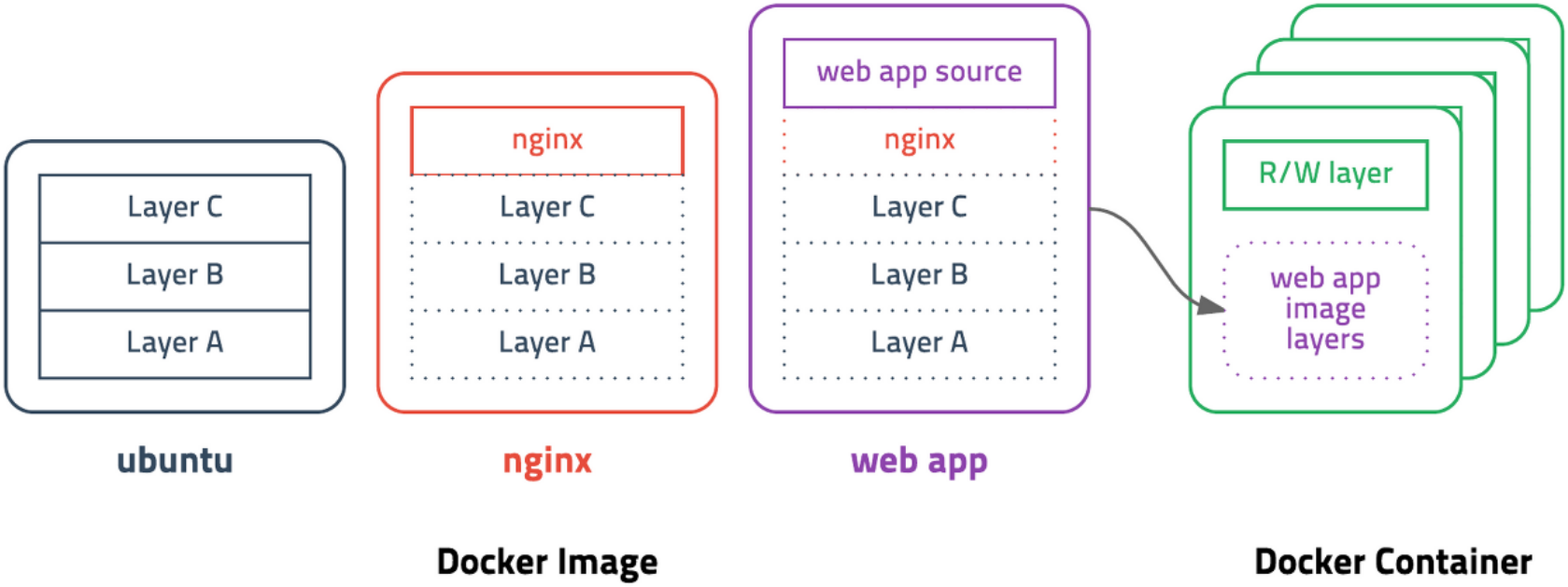
podman 과 buildah 등은 ?

:/etc/containers$ cat registries.conf

unqualified-search-registries = ["docker.io","quay.io","registry.fedoraproject.org"]

컨테이너 이미지 레이어 구조

컨테이너 이미지는 여러 개의 읽기 전용 레이어로 구성



Dockerfile

```
FROM ubuntu:22.04

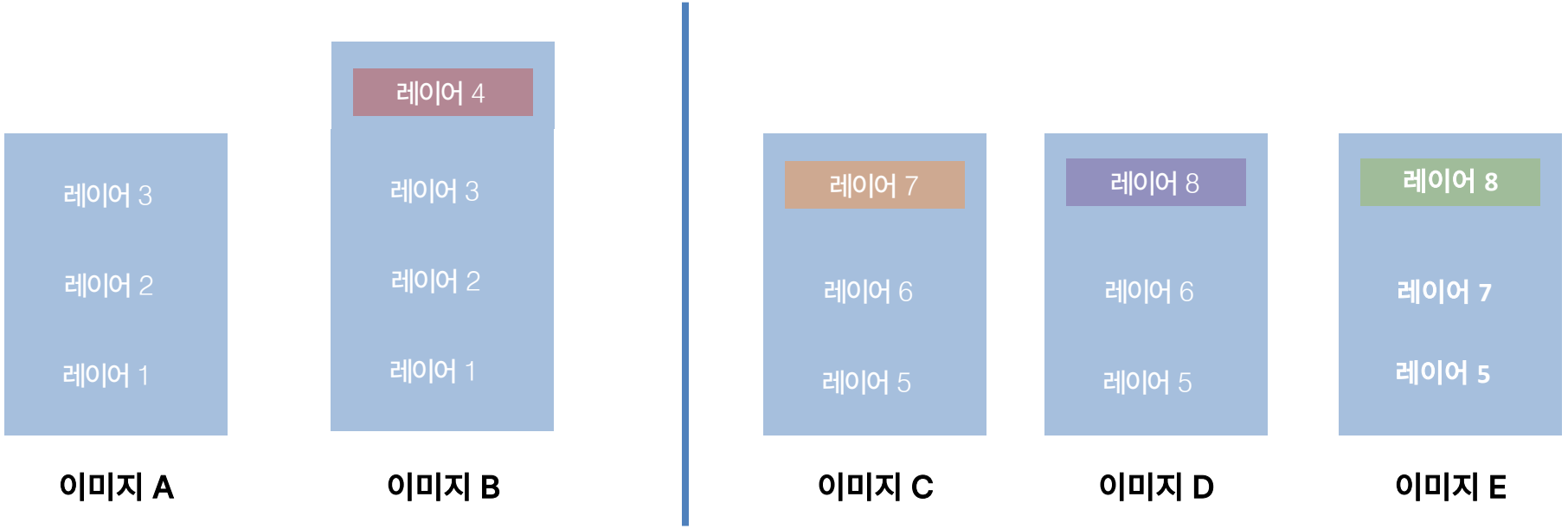
RUN apt-get update && \
    apt-get install -y --no-install-recommends nginx && \

COPY ./src /var/www/html

CMD ["nginx", "-g", "daemon off;"]
```

컨테이너 이미지 레이어 구조

컨테이너 이미지를 레이어 형태로 구성, 중복 다운로드 업로드 불필요

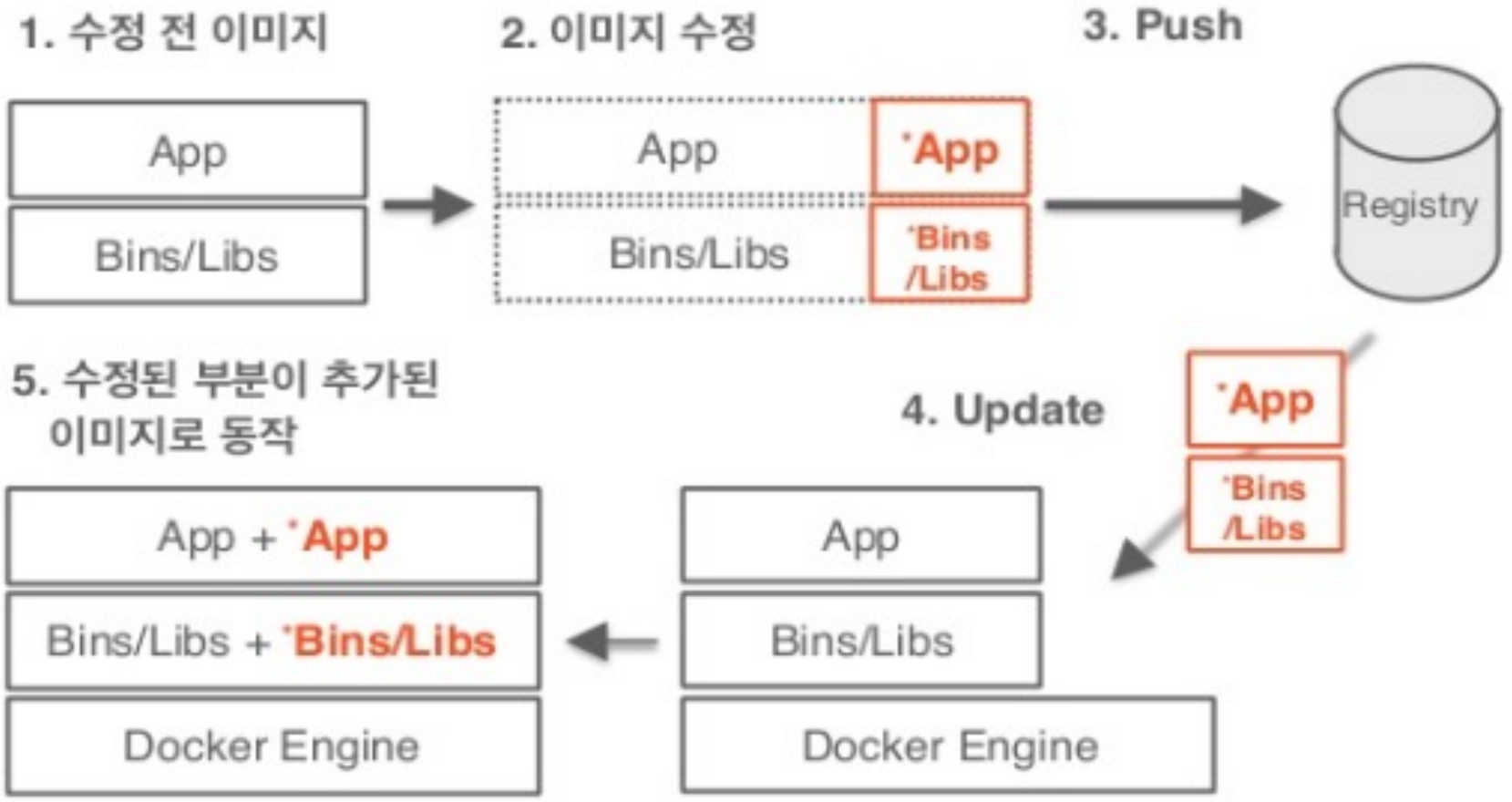


이미지 A 삭제 시
레이어 1, 2, 3는 삭제 되지 않음

이미지 C를 이미 다운로드 받은 경우
이미지 D는 레이어 8 다운로드 필요
이미지 E는 다운로드 불필요

컨테이너 이미지 Push/Pull

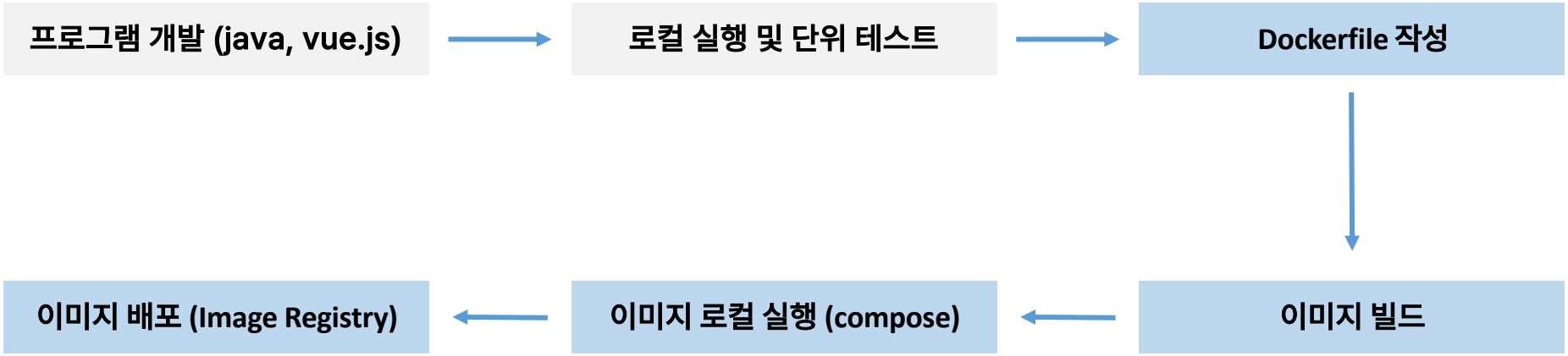
- 기본 이미지에서 수정된 부분만 업데이트



이미지 빌드/실행/배포 흐름

이미지 생성과 푸시

- 누군가 만들어 놓은 이미지를 활용하는 것도 매우 좋은 일!
- 그러나 경우에 따라 우리가 필요한 이미지가 없는 경우도 존재
- 우리가 필요한 도커를 만들고 레지스트리에 등록하는 방법에 대해 학습



[참고] Continous Integration & Continous Deploy (CICD)

애플리케이션을 컨테이너로 build/push 후 실행 환경에 배포하는 흐름

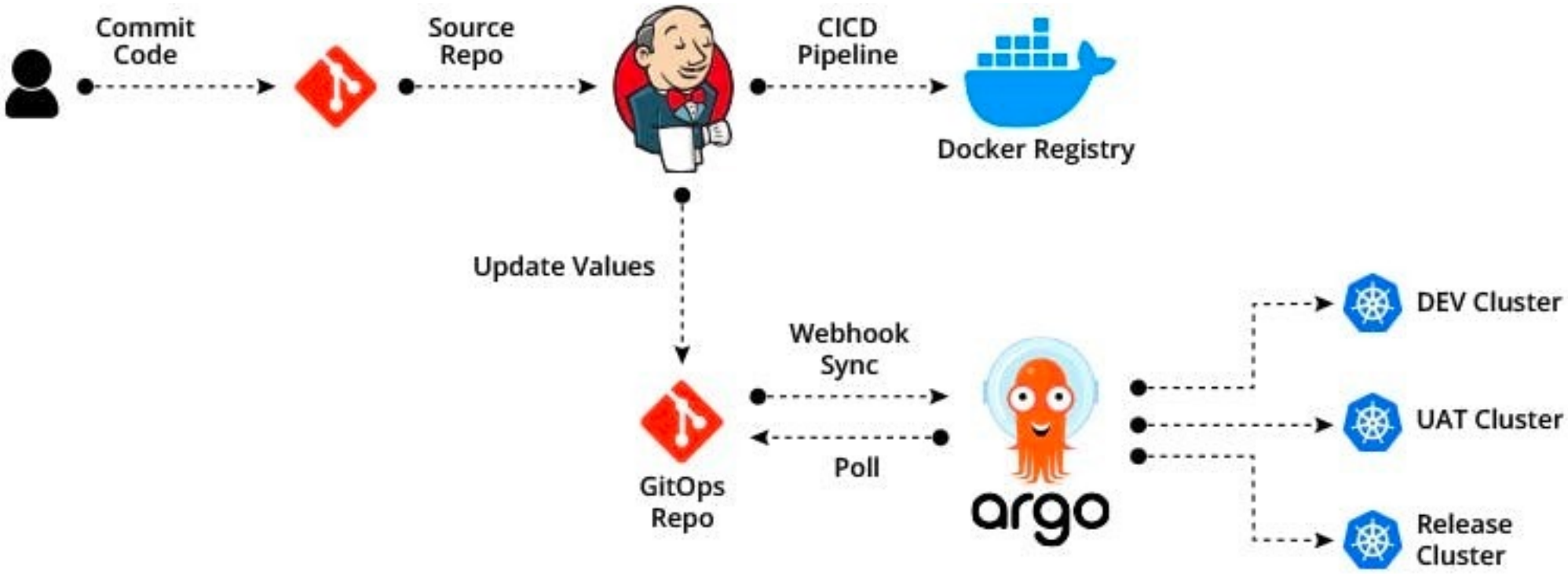


Image 1: CI/CD Flow with Argo CD

도커 사용해보기

앞으로 나오는 모든 명령을 차례대로
입력하면서 실습을 진행합니다.

이미지 검색하기 (1/3)

docker search <이미지 이름>

✓ Docker Hub에서 이미지 검색해보기

```
$ docker search ubuntu
NAME                                DESCRIPTION                                     STARS     OFFICIAL   AUTOMATED
ubuntu                              Official Ubuntu base image                      383
stackbrew/ubuntu                    Official Ubuntu base image                      40
crashsystems/gitlab-docker          A trusted, regularly updated build of GitL...   19                   [OK]
dockerfile/ubuntu                   Trusted Ubuntu (http://www.ubuntu.com/) Bu...   15                   [OK]
ubuntu-upstart                      Upstart is an event-based replacement for ...   7
cmfatih/phantomjs                   PhantomJS [ phantomjs 1.9.7, casperjs 1.1....   5                    [OK]
dockerfile/ubuntu-desktop           Trusted Ubuntu Desktop (LXDE) (http://lxde...   5                    [OK]
lukasz/docker-scala                 Dockerfile for installing Scala 2.10.3, Ja...   5                    [OK]
litaio/ruby                         Ubuntu 14.04 with Ruby 2.1.2 compiled from...   5                    [OK]
```

✓ 보통 ubuntu, centos, redis 등 OS나 프로그램 이름을 가진 이미지가 공식 이미지

✓ 나머지는 사용자들이 만들어서 공개한 이미지

이미지 검색하기 (2/3)

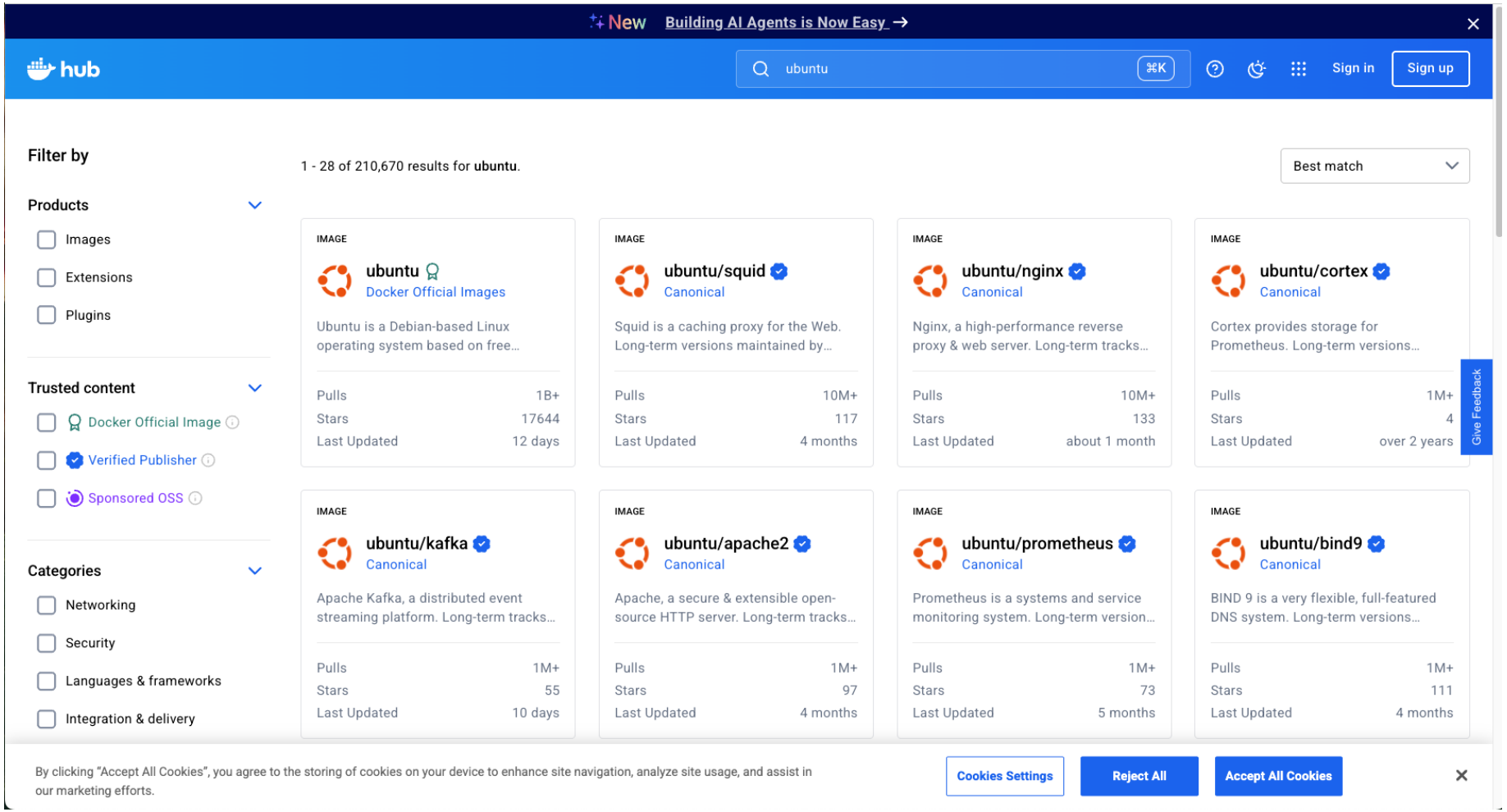
- ✓ 도커는 Docker Hub(https://registry.hub.docker.com)를 통해 이미지를 공유하는 생태계가 구축되어 있음
- ✓ 유명 리눅스 배포판과 오픈 소스 프로젝트(Redis, Nginx 등)의 이미지를 모두 Docker Hub에서 구할 수 있음
- ✓ 이미지와 관련된 명령은 기본적으로 Docker Hub를 이용하도록 설정되어 있음

이미지 검색하기 (3/3)

✓ Docker Hub에서 이미지를 검색한 뒤 해당 이미지의 Tags 탭에서 이미지의 버전을 볼 수 있음

오픈 이미지 https://hub.docker.com

harbor https://https://skala-registry.skala-ai.com/



이미지 내려 받기

docker pull <이미지 이름>:<태그>

✓ Docker Hub에서 우분투 이미지 받아보기

```
$ docker pull ubuntu
```

✓ 이미지 이름 뒤에 latest를 설정하면 최신 버전을 받음

✓ ubuntu:14.04, ubuntu:12.10처럼 태그를 지정할 수 있음

✓ 이미지 이름에서 pyrasis/ubuntu처럼 / 앞에 사용자명을 지정하면 해당 사용자가 올린 이미지를 받음(공식 이미지는 사용자명이 붙지 않음)

✓ 호스트에 설치된 리눅스 배포판과 도커 이미지의 배포판 종류는 달라도 됨.
즉 CentOS에서 우분투 컨테이너를 실행할 수 있음

이미지 목록 출력하기

✓ 모든 이미지를 출력해보기

```
$ docker images
REPOSITORY          TAG                 IMAGE ID            CREATED                  V IRTUAL   SIZE
ubuntu              latest              e54ca5efa2e9        Less than a second ago   276.1 MB
```

docker images ubuntu처럼 이미지 이름을 설정하면 이름은 같지만 태그가 다른 이미지가 출력됨

컨테이너 실행하기 (1/2)

docker run <옵션> <이미지 이름> <실행할 파일>

✓ 이미지를 컨테이너로 생성한 뒤 Bash 셸 실행해보기

```
$ docker run –it --name hello ubuntu:latest /bin/bash
```

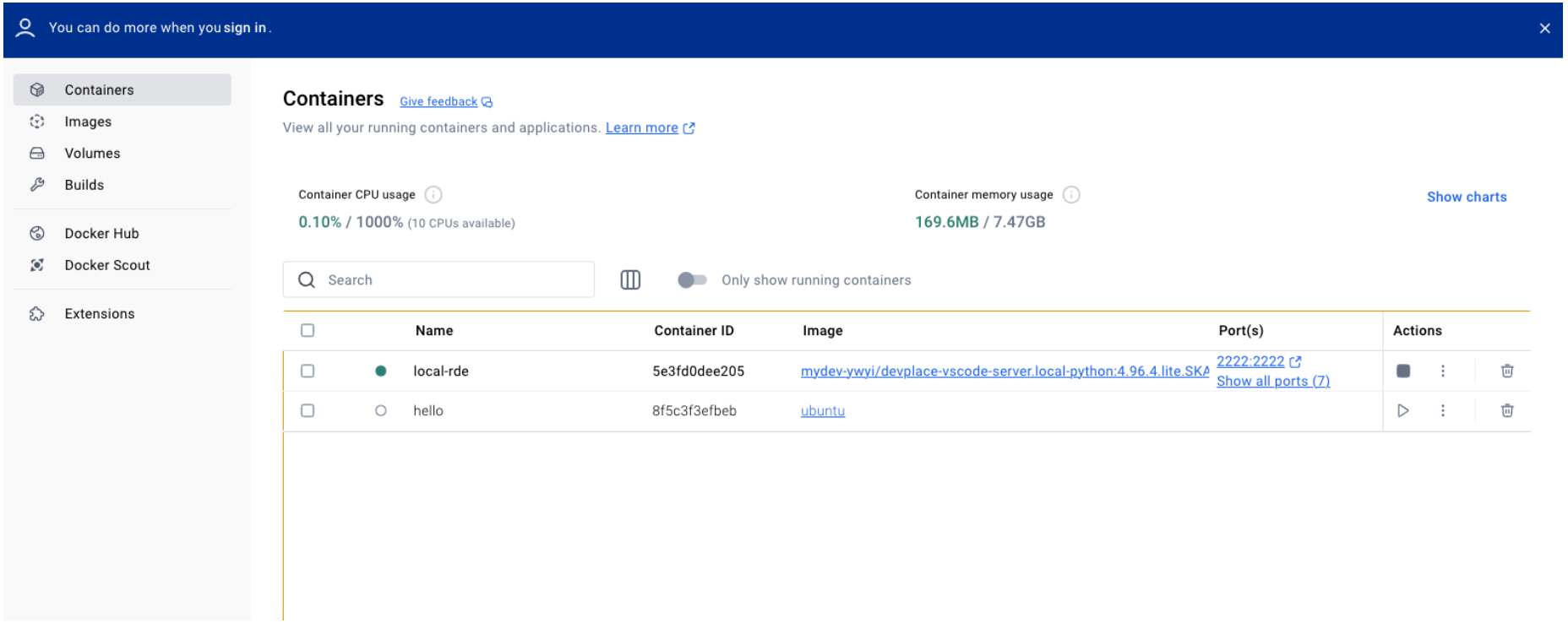
ubuntu 이미지를 컨테이너로 생성한 뒤 ubuntu 이미지 않의 /bin/bash를 실행

- ✓ -i(interactive), -t(Pseudo-tty) 옵션을 사용하면 실행된 Bash 셸에 입력 및 출력 가능
- ✓ --name 옵션으로 컨테이너에 이름을 지정할 수 있음. 이름을 지정하지 않으면 도커가 자동으로 이름을 생성하여 지정
- ✓ tag 정보가 없는 경우 latest 가 default (명시적으로 버전 정보를 지정하는 것을 권고)

컨테이너 실행하기 (2/2)

cd, ls 명령으로 컨테이너 내부를 둘러본 뒤 exit를 입력하여 Bash 셸에서 빠져 나오기

✓ 우분투 이미지에서 /bin/bash 실행 파일을 실행했기 때문에 여기서 빠져나오면 컨테이너가 정지(stop)됨



컨테이너 목록 확인하기

docker ps

✓ running 중인 컨테이너 목록만 확인

```
> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS         PORTS                                       NAMES
e5e081049b81   mariadb:latest   "docker-entrypoint.s..."   57 minutes ago   Up 57 minutes   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp   mariadb
```

docker ps --all (또는 -a)

✓ running 과 stop 모든 컨테이너 목록 확인

```
> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED         STATUS                     PORTS                                       NAMES
0885c28b8583   ubuntu:latest    "/bin/bash"              6 minutes ago    Exited (0) 4 minutes ago                                       hello
e5e081049b81   mariadb:latest   "docker-entrypoint.s..."   59 minutes ago   Up 59 minutes              0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp   maria
```

ubuntu:latest 컨테이너는 정지된 상태 (exited)

✓ -a 옵션을 사용하면 정지된 컨테이너까지 모두 출력됨

✓ 옵션을 사용하지 않으면 실행되고 있는 컨테이너만 출력됨

컨테이너 시작하기

docker start <컨테이너 이름> (또는 컨테이너 ID)

✓ 방금 정지한 컨터이너를 다시 시작하기

```
$ docker start hello
```

컨테이너 이름 대신 컨테이너 ID를 사용해도 됨

✓ 컨테이너를 시작했으면 컨테이너 목록 출력해보기

```
$ docker ps
CONTAINER ID   IMAGE          COMMAND     CREATED          STATUS          PORTS   NAMES
6338ce52d07c   ubuntu:latest  /bin/bash   15 minutes ago   Up 3 seconds            hello
```

hello 컨테이너가 시작되었음

컨테이너 재시작하기

docker restart <컨테이너 이름>

✓ OS 재부팅처럼 컨테이너를 다시 시작해보기

```
$ docker restart hello
```

컨테이너 이름 대신 컨테이너 ID를 사용해도 됨

외부에서 컨테이너 안의 명령 실행하기

docker exec <컨테이너 이름> <명령> <매개 변수>

- ✓ 현재 컨테이너가 /bin/bash로 실행된 상태
- ✓ /bin/bash를 통하지 않고 외부에서 컨테이너 안의 명령 실행해보기

```
$ docker exec hello echo "Hello World"
Hello World
```

- ✓ 컨테이너 이름 대신 컨테이너 ID를 사용해도 됨
- ✓ 컨테이너가 실행되고 있는 상태에서만 사용할 수 있으며 정지된 상태에서는 사용할 수 없음
- ✓ 컨테이너 안의 echo 명령을 실행하고 매개 변수로 "Hello World"를 지정했기 때문에 Hello World가 출력됨

외부에서 컨테이너 내 shell 프로세스 실행

docker exec –it <컨테이너 이름> /bin/bash

✓ 나의 shell을 기준으로 컨테이너 내부 shell 접속하기 위한 방법

```
$ docker exec –it hello /bin/bash     # 또는 /bin/sh
# ls
```

container 내로 shell로 접속하고자 할때 사용

✓ -i(interactive), -t(Pseudo-tty) 옵션을 사용하면 실행된 Bash 셸에 입력 및 출력 가능

컨테이너 정지하기

docker stop <컨테이너 이름>

✓ 먼저 실행된 컨테이너 목록 출력하기

```
$ docker ps
CONTAINER ID   IMAGE          COMMAND            CREATED          STATUS          PORTS                    NAMES
0885c28b8583   ubuntu:latest   "/bin/bash"         17 minutes ago     Up 7 minutes                             hello
```

✓ 컨테이너 정지해보기(컨테이너 이름 대신 컨테이너 ID도 사용해도 됨)

```
$  docker stop hello
```

✓ 실행된 컨테이너 목록을 출력해보기

```
$ docker ps -a
CONTAINER ID   IMAGE          COMMAND            CREATED          STATUS          PORTS                    NAMES
0885c28b8583   ubuntu:latest   "/bin/bash"         17 minutes ago     Exited (137) 14 seconds ago              hello
```

컨테이너 삭제하기

docker rm <컨테이너 이름>

✓ 생성된 컨테이너를 삭제해보기

```
$ docker rm hello
```

컨테이너 이름 대신 컨테이너 ID를 사용해도 됨

✓ 모든 컨테이너 목록을 출력해보기

```
$ docker ps -a
CONTAINER ID   IMAGE   COMMAND   CREATED   STATUS   PORTS   NAMES
```

hello 컨테이너를 삭제했기 때문에 아무것도 나오지 않음

컨테이너에 Volume 연결하기

docker run --name <컨테이너명> -v <호스트 경로>:<컨테이너 내부 경로> <이미지 명:버전> <명령>

✓ 먼저 컨테이너에 마운트할 ./data/info.txt 생성

```
$ mkdir -p ./data
$ echo “hello docker world” > ./data/info.txt
```

✓ 로컬 디렉토리를 컨테이너에 마운트 및 파일 추가 후 확인하기

```
$  docker run -it --name hello -v $(pwd)/data:/app/data ubuntu:latest /bin/bash
# cd /app/data
# ls
root@f363758dad9b:/app/data# cat info.txt
hello docker world
# touch add.txt
# exit
$ ls ./data/
add.txt info.txt
 $ docker stop hello  && docker rm hello
```

참고: $(pwd) vs ${PWD} 차이

이미지 삭제하기

docker rmi <이미지 이름>:<태그>

✓ 이미지 삭제해보기

```
$ docker rmi ubuntu:latest
```

✓ 이미지 이름 대신 이미지 ID를 사용해도 됨

✓ docker rmi ubuntu처럼 이미지 이름만 지정하면 태그는 다르지만 ubuntu 이름을 가진 모든 이미지가 삭제됨

✓ 이미지 목록을 출력해보기

```
$ docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             VIRTUAL SIZE
```

ubuntu 이미지를 삭제했기 때문에 아무것도 나오지 않음

[참고] 도커 명령어 참고

다양한 도커 명령어

- CLI: docker --help
- 웹: https://docs.docker.com/engine/reference/run/

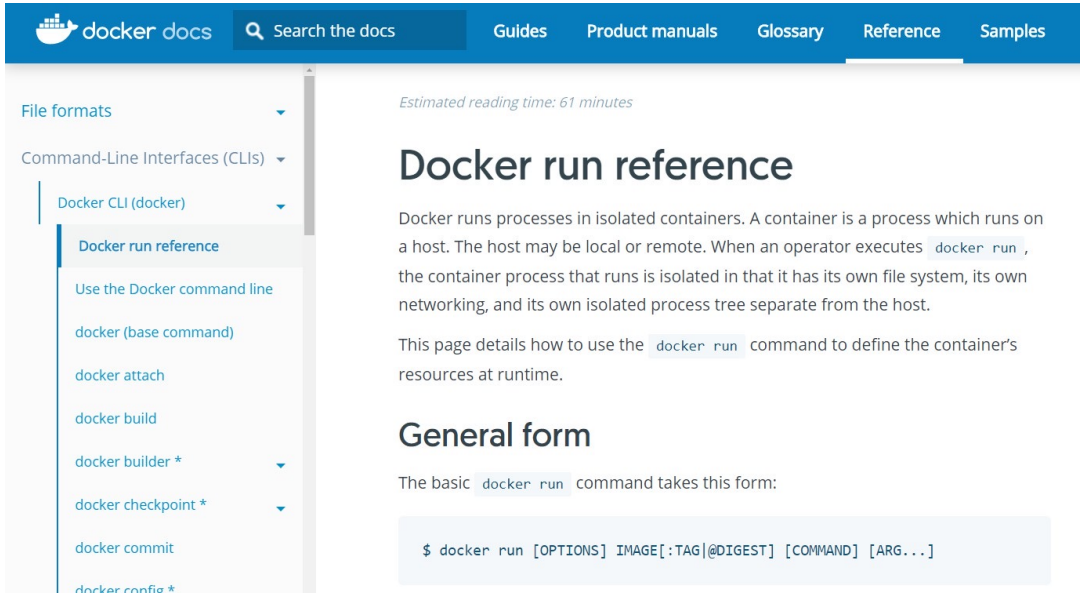
```
server1@server1-VirtualBox:~$ sudo docker docker --help

Usage:  docker [OPTIONS] COMMAND

A self-sufficient runtime for containers

Options:
      --config string      Location of client config files (default "/home/server1/.docker")
  -D, --debug              Enable debug mode
  -H, --host list          Daemon socket(s) to connect to
  -l, --log-level string   Set the logging level ("debug"|"info"|"warn"|"error"|"fatal") (default "info")
      --tls                Use TLS; implied by --tlsverify
      --tlscacert string   Trust certs signed only by this CA (default "/home/server1/.docker/ca.pem")
      --tlscert string     Path to TLS certificate file (default "/home/server1/.docker/cert.pem")
      --tlskey string      Path to TLS key file (default "/home/server1/.docker/key.pem")
      --tlsverify          Use TLS and verify the remote
  -v, --version            Print version information and quit

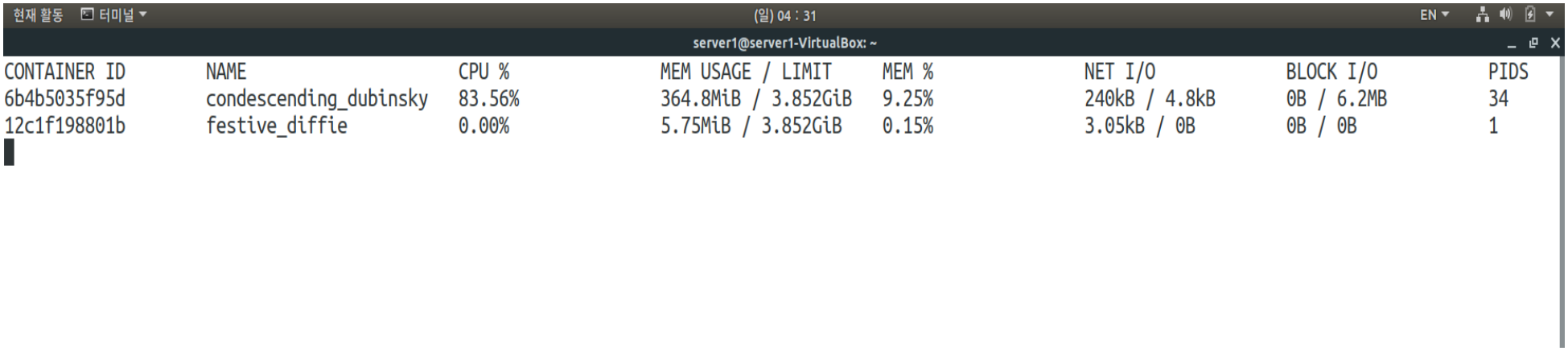
Management Commands:
  builder     Manage builds
  config      Manage Docker configs
  container   Manage containers
  engine      Manage the docker engine
  image       Manage images
  network     Manage networks
  node        Manage Swarm nodes
  plugin      Manage plugins
  secret      Manage Docker secrets
  service     Manage services
  stack       Manage Docker stacks
  swarm       Manage Swarm
```



docker 운영 환경 명령어

도커 명령어	설명
docker container prune	중지된 모든 컨테이너를 제거
docker image prune	태그가 없는 모든 이미지 파기. (-a option은 모든 이미지 파기)
docker system prune	사용하지 않는 모든 데이터 삭제(이미지, 컨테이너, 볼륨, 네트워크 등)
docker container stats (=docker stats)	컨테이너 사용 현황 출력
docker network list (=docker network ls)	생성되어 있는 network 정보 검색 (상세: inspect)

docker container states 실행



[참고] Docker & podman 명령어

Docker 명령어

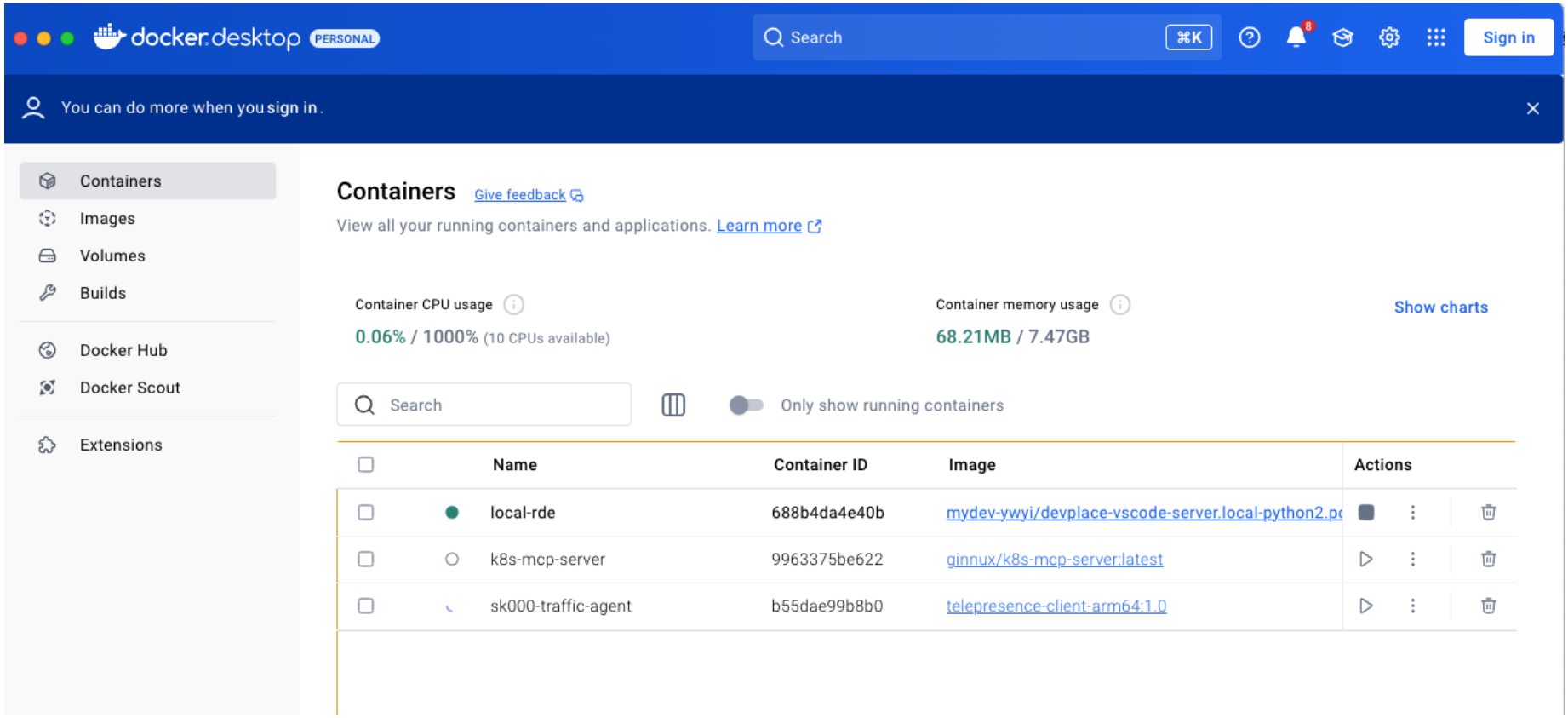
https://docs.docker.com/reference/cli/docker/

podman 명령어

https://docs.podman.io/en/stable/Commands.html

docker desktop 사용하기

Docker desktop을 실행해서 console을 통해 실행되고 있는 컨테이너를 중지/재실행/삭제하고, 이미지를 관리해보자





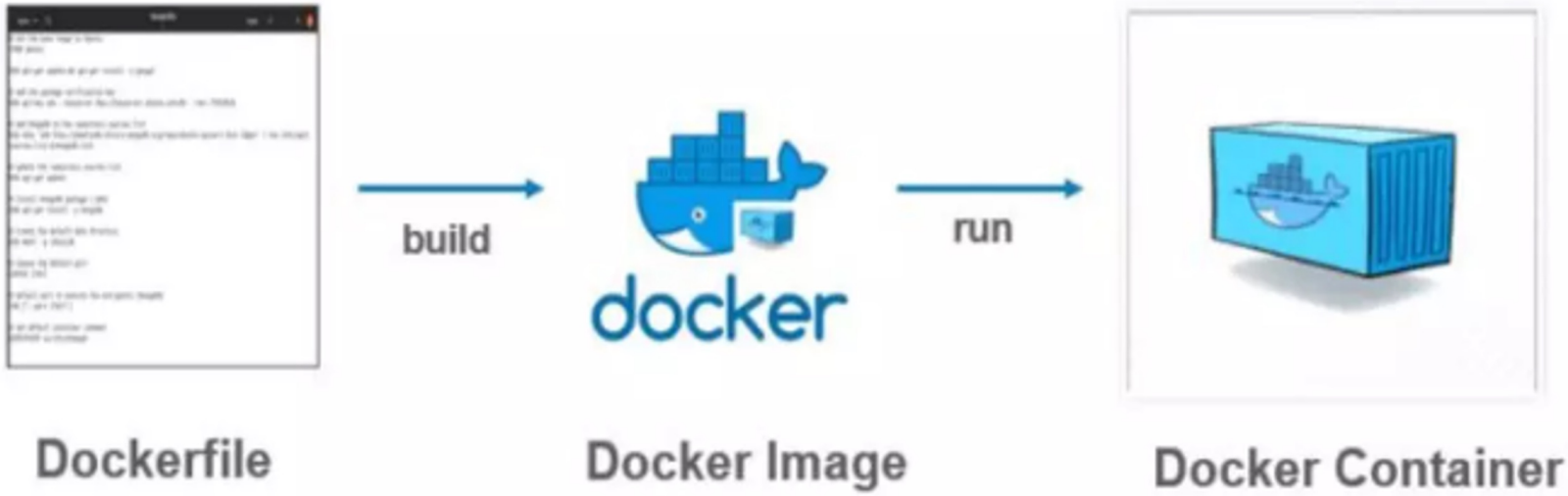
Dockerfile 작성

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

Dockerfile 이란?

- 컨테이너 이미지 빌드 절차 명세서
- 컨테이너 이미지를 생성하기 위한 명령어/지시문 집합
- Docker는 Dockerfile의 지시문을 읽어 컨테이너 이미지를 자동으로 생성



Dockerfile 작성하기

참고: 01.answer-code/03.dockerfile/01.nginx/Dockerfile

- ✓ Dockerfile을 만들고 아래의 내용을 작성해보자
- ✓ 우분투 14.04를 기반으로 nginx 서버를 설치한 도커 이미지를 생성하는 예제

Dockerfile

```
FROM ubuntu:latest

MAINTAINER Foo Bar himang10@gmail.com

RUN apt-get update
RUN apt-get install -y nginx
RUN echo "\ndaemon off;" >> /etc/nginx/nginx.conf
RUN chown -R www-data:www-data /var/lib/nginx

VOLUME ["/data", "/etc/nginx/site-enabled", "/var/log/nginx"]

WORKDIR /etc/nginx

CMD ["nginx"]

EXPOSE 80
EXPOSE 443
```

나의 이미지 생성하기

docker build <옵션> <Dockerfile 경로>

- ✓ 앞에서 작성한 Dockerfile로 이미지 생성해보기
- ✓ Dockerfile이 저장된 example 디렉터리에서 명령 실행

```
~/example$  docker build --tag myhello:0.1 .
```

- ✓ --tag 옵션으로 이미지 이름과 태그를 설정할 수 있음
- ✓ 이미지 이름만 설정하면 태그는 latest로 설정됨
- ✓ 잠시 기다리면 이미지가 생성됨. 이미지 목록 출력해보기

```
$ docker images
REPOSITORY          TAG                 IMAGE ID            CREATED             VIRTUAL SIZE
ubuntu              14.04               e54ca5efa2e9        Less than a second ago   276.1 MB
ubuntu              latest              e54ca5efa2e9        Less than a second ago   276.1 MB
myhello             0.1                 2031ee0736e8        9 minutes ago            298.4 MB
```

hello:0.1 이미지가 생성됨

나의 이미지 실행하기

✓ 앞에서 생성한 이미지를 실행해보기

```
$ docker run --name hello-nginx -d --net bridge  -p 8080:80 -v $(pwd)/data:/data myhello:0.1
```

✓ -d 옵션은 컨테이너를 백그라운드로 실행

✓ -p 8080:80 옵션으로 호스트의 80번 포트와 컨테이너의 80번 포트를 연결하고 외부에 노출 (RDE 내에서는 --net bridge 로 추가 설정 필요) podman는 default 가 network 이 host type

✓ -v $(pwd)/data:/data 옵션으로 호스트의 $(pwd)/data 디렉터리를 컨테이너의 /data 디렉터리에 연결

나의 컨테이너 실행 확인하기

✓ 실행된 컨테이너 목록을 출력해보기

```
$ docker ps
CONTAINER ID   IMAGE       COMMAND CREATED          STATU          PORT                                  NAME
3c06a0bebab6   hello:0.1   6nginx  10 minutes ago   Up 10 minutes  443/tcp, 0.0.0.0:80->80/tcp   6ello-nginx
```

hello-nginx 컨테이너가 실행되었음

웹 브라우저를 실행하고 http://<호스트 IP>:8080으로 접속하면

Welcome to nginx! 페이지가 표시됨

✓ 컨테이너 종료 및 삭제

```
$ docker ps
CONTAINER ID   IMAGE       COMMAND CREATED          STATU          PORT                                  NAME
3c06a0bebab6   hello:0.1   6nginx  10 minutes ago   Up 10 minutes  443/tcp, 0.0.0.0:80->80/tcp   6ello-nginx
```

Dockerfile 문법

Dockerfile은 반드시 **FROM** 명령어로 시작해야 합니다.

FROM 명령어 앞에는 하나 이상의 ARG 명령어만 사용할 수 있으며,

이는 파일 내 FROM 줄에서 사용되는 인자를 선언합니다.

Dockerfile의 형식은 다음과 같습니다

Comment (주석)

[명령어] [인자 1] ...

Dockerfile 명령어

명령어	설명	정의 방법
FROM	우분투와 같은 운영체제 이름이나 필요한 애플리케이션 이름을 지정	FROM ubuntu:14.04
RUN	컨테이너가 빌드될 때 실행할 명령어	RUN ["mv", "1234", "5678"]
EXPOSE	컨테이너 외부에 노출할 포트 지정	EXPOSE 80
ENV	환경 변수 이름 지정	ENV abc /tm p echo ${ab c}
CMD	컨테이너가 시작될 때 실행할 명령어	CMD ["echo", "1234"]
ENTRYPOINT	컨테이너가 실행될 때 기본 명령어 지정 (CMD와 달리 옵션을 넘길 수 있음)	ENTRYPOINT ["python"] ... $ docker run -t 이미지 -c "print('1234')"
WORKDIR	작업할 디렉토리를 세팅	WORKDIR /project
USER	도커 실행 시 사용할 유저 이름 또는 uid를 지정	USER user 또는 USER user:group
VOLUME	호스트의 디렉토리를 도커에 연결 커밋 없이 사용 가능하며 주로 로그 수집이나 데이터 저장용도로 사용	VOLUME ["/tmp"]
COPY, ADD	파일이나 디렉토리를 복사할 때 사용하는 명령어 명령어를 대신 사용하면 URL도 가능하며 압축된 파일은 압축을 해제하 면서 옮김	COPY file /copy/to/path ADD http://example.com/big.tar.xz /usr/src/things/ ADD rootfs.tar.x z /
ARG	이미지 빌드 시점 (docker build)에 사용할 변수 정의 docker build --build-arg NODE_VERSION=20	ARG NODE_VERSION=18
SHELL	원하는 타입의 쉘을 사용	SHELL ["/bin/sh", "-c"] SHELL ["cmd", "/S", "/C"]

FROM 명령어

```
FROM [--platform=<platform>] <image>[:<tag>] [AS <name>]
```

- **빌드 스테이지 초기화:** **FROM** 명령어는 새로운 빌드 스테이지(Build Stage)를 초기화하며, 이후 진행될 명령어들의 기본 바탕이 되는 **베이스 이미지(Base Image)**를 정의합니다.
- **ARG 지시어와의 예외 규칙:** Dockerfile은 반드시 `FROM`으로 시작해야 하지만, 유일하게 **ARG** 지시어만은 `FROM` 앞에 위치할 수 있습니다. 이를 통해 베이스 이미지의 버전 등을 변수화할 수 있습니다.
- **다중 스테이지 빌드 (Multi-stage Build):** 하나의 Dockerfile 내에서 `FROM`은 여러 번 사용할 수 있습니다. 이를 통해 최종 이미지의 용량을 최적화하거나 빌드 의존성을 효율적으로 분리할 수 있습니다.

FROM 명령어 활용 방법

샘플 A. 기본적인 베이스 이미지 지정

```
# 가장 단순하고 표준적인 형태의 이미지 지정
FROM ubuntu:22.04

RUN apt-get update && apt-get install -y curl
```

샘플 B. ARG 지시어 연동을 통한 버전 유연화

```
# FROM 이전에 변수를 선언하여 유연성 확보
ARG NODE_VERSION=18-alpine

FROM node:${NODE_VERSION}
WORKDIR /app
```

FROM 명령어 : Mutli-Stage 방법

```
# ========================================
# [Stage 1] Build Stage: Vue.js 소스코드 컴파일
# ========================================
FROM node:22-alpine AS builder
WORKDIR /app
# 의존성 설치 (캐싱 활용)
COPY package*.json ./
RUN npm install
# 소스 전체 복사 및 프로덕션 빌드 수행
COPY . .
RUN npm run build

# ========================================
# [Stage 2] Production Stage: Nginx 배포 및 실행
# ========================================
FROM nginx:stable-alpine
WORKDIR /usr/share/nginx/html
# 기존 Nginx 기본 정적 파일 제거
RUN rm -rf ./*
# 중요: 'builder' 스테이지에서 생성된 정적 빌드 파일(dist)만 추출하여 복사
COPY --from=builder /app/dist .
# 80 포트 개방 및 Nginx 실행
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

- Node.js 개발 도구, 소스코드 및 소스 캐시 파일 (node_modules)이 최종 배포 이미지에 포함되지 않음
- 보안성이 대폭 강화
- 생성되는 이미지 크기가 수백 MB에서 수십 MB로 축서

RUN 명령어

```
RUN <command>
```

주요 핵심 특징

- **이미지 빌드 시점의 실행:**

  패키지 설치, 파일 생성, 권한 변경 등 이미지에 영구적으로 반영되어야 할 기초 환경을 다질 때 사용

- **레이어(Layer) 생성**

  - RUN 명령어가 실행될 때마다 Docker 이미지의 스냅샷인 새로운 레이어가 하나씩 추가

  - RUN 명령어를 무분별하게 여러 줄 작성하면 이미지 용량이 비대해지고 생성 불가 발생

- **캐싱(Caching) 시스템:**

  동일한 RUN 명령은 도커가 기존에 빌드된 레이어를 재사용(캐시)하여 빠르게 처리

RUN 명령어 활용 예시

샘플 A. Shell 폼 (가장 대중적인 패키지 설치 방식)

```
FROM ubuntu:22.04

# 여러 명령을 &&로 묶어 단 하나의 레이어만 생성하게 함
RUN apt-get update && apt-get install -y \
    curl \
    git \
    && rm -rf /var/lib/apt/lists/*
```

샘플 B. Exec 폼 (셸을 거치지 않고 바이너리를 직접 실행)

```
FROM ubuntu:22.04

# /bin/sh를 거치지 않고 mkdir 바이너리를 직접 호출하여 디렉토리 생성
RUN ["mkdir", "-p", "/var/www/html"]
```

[실습] 이미지 빌드 및 확인하기

Dockerfile 만들기

참고: 03.dockerfile/02.command/Dockerfile

```
FROM ubuntu:22.04

RUN apt-get update && apt-get install -y curl lsb-release
```

빌드 및 라벨 확인하기

```
> docker buildx build --tag linux-container:1.0 .
> docker run -it --name linux-container linux-container:1.0 /bin/bash
# lsb_release -a
No LSB modules are available.
Distributor ID: Ubuntu
Description:    Ubuntu 22.04.5 LTS
Release:22.04
Codename:       noble
# exit
> docker rm linux-container
```

[참고] 에러 처리

```
docker: Error response from daemon: Conflict. The container name "/linux-container" is
already in use by container
"a999c503ae516838e93f3dbbcb286a5a4bf31064771016822bf5dac26634c800". You have to remove (or
rename) that container to be able to reuse that name.
```

1. docker ps 를 통해 동일 컨테이너가 동작하는지 확인
2. docker stop <container name or container-id>

```
docker: Error response from daemon: Conflict. The container name "/linux-container" is
already in use by container
"953b321275d98107006935ed8e7ec130758fa5e9da2b25683eef6881e90fcd99". You have to remove (or
rename) that container to be able to reuse that name.
```

1. docker ps -a 를 통해 동일 컨테이너가 남아 있는지 확인
2. docker rm <container name or container-id>

[실습] Multi-Stage 실습하기

Dockerfile 만들기

참고 : 06.execise-source/vue-frontend/Dockerfile

자신의 vue.js 코드에 대한 Dockerfile을 만들고 빌드하고 실행시켜보자

```
# ── 빌드 스테이지 ──────────────────────────────
FROM node:22-alpine AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# ── 배포 스테이지 ──────────────────────────────
FROM nginx:alpine

# 기본 nginx 설정 제거 후 커스텀 설정 복사
RUN rm /etc/nginx/conf.d/default.conf
COPY default.conf /etc/nginx/conf.d/

# 빌드 결과물 복사
COPY --from=builder /app/dist /usr/share/nginx/html

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

[실습] Multi-Stage 실습하기

default.conf

참고 : 06.execise-source/vue-frontend/default.conf

nginx가 자신의 Frontend source (Static resource)로 라우팅되도록 설정하기

```
server {
    listen 80;

    # 정적 파일 서빙
    location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html;

        expires -1;
        add_header Cache-Control "no-store, no-cache, must-revalidate";
        add_header Pragma "no-cache";
    }

    # Spring Boot API 프록시
    location /api {
        resolver 127.0.0.11 valid=10s;
        set $backend "spring-backend:8080";
        proxy_pass http://$backend;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
    }
}
```

[실습] Multi-Stage 실습하기

빌드 및 frontend 실행하기

```
> docker buildx build --tag vue-frontend:1.0 .
> docker run -d --name vue-frontend -p 8080:80 vue-frontend:1.0
de2117811cb2177045a83df441fef4e3ffe47f420e96d699b2848523830ea78f

> docker stop vue-frontend
>
```

접속 하기

http://localhost:8080

종료 하기

```
> docker stop vue-frontend
> docker rm vue-frontend
```

ARG 명령어

```
ARG <name>[=<default value>]
```

- **빌드 시점 변수 정의:** ARG는 사용자가 `docker build` 명령어를 실행할 때 `--build-arg <varname>=<value>` 플래그를 통해 넘겨줄 수 있는 변수를 정의합니다.
- **컨테이너 런타임 차단:** `ARG`로 설정된 변수는 **이미지가 빌드되는 동안에만 유효**하며, 완성된 이미지가 컨테이너로 실행되는 런타임 시점에는 유지되지 않습니다. (런타임용은 `ENV` 사용)
- **선언 위치 및 스코프(Scope):** Dockerfile 내에서 `FROM` 명령어보다 먼저 선언될 수 있는 유일한 지시어입니다. 단, `FROM` 이전에 선언된 `ARG`는 빌드 스테이지 내부에서 사용하려면 스테이지 안에서 한 번 더 선언해 주어야 합니다.

ARG 명령어 활용 방법

샘플 A. 기본적인 기본값 설정 및 활용

```
FROM alpine:latest
# 빌드 시 변수를 선언하고 기본값을 설정
ARG USER_NAME=developer
RUN echo "안녕하세요, ${USER_NAME}님!"
```

샘플 B. FROM 이전에 선언하여 이미지 버전 동적 제어

```
# 1. FROM 범위 밖에서 베이스 이미지 버전용 ARG 정의
ARG TAG=18-alpine
FROM node:${TAG}

# 2. 스테이지 내부에서 상위 ARG를 재사용하기 위해 빈 이름으로 다시 선언
ARG TAG
RUN echo "현재 빌드에 사용된 Node 버전 태그: ${TAG}"
```

Multi-Stage 구조를 회손하지 않기 위한 방법

[실습] 이미지 빌드 및 ARG 확인하기

Dockerfile 만들기

```
ARG UBUNTU_VERSION=22.04

FROM ubuntu:${UBUNTU_VERSION}

RUN apt-get update && apt-get install -y curl lsb-release
ARG UBUNTU_VERSION
RUN echo "현재 빌드에 사용된 ubuntu version: ${UBUNTU_VERSION}"
```

빌드 및 확인하기

```
> docker buildx build --tag linux-container:1.0 --build-arg UBUNTU_VERSION=24.04 .
> docker run -it --name linux-container linux-container:1.0 /bin/bash
# lsb_release -a
No LSB modules are available.
Distributor ID:   Ubuntu
Description:      Ubuntu 24.04.4 LTS
Release: 24.04
Codename:         noble
# exit
> docker rm linux-container
```

LABEL 명령어

```
LABEL <key>=<value> <key>=<value> <key>=<value> ...
```

주요 핵심 특징

- **이미지 메타 데이터 주입**: 버전, 작성자, 설명, 라이선스 등 설명 정보(메타데이터)를 이미지에 기록
- **빌드 및 실행에 영향 없음:** 순수 메모(주석)용 데이터
- **외부 연동 및 관리 용이:** 이미지를 배포한 후 docker inspect 명령어를 통해 라벨 정보 조회
- **이미지 분류 정보 이용**: 쿠버네티스나 사내 이미지 저장소(Registry) 등에서 이미지를 분류하고 관리하는 기준으로 사용

LABEL 명령어 활용 예시

샘플 A. 가장 표준적인 한줄 작성

```
FROM alpine:latest

# 이미지의 유지보수 담당자 정보 기입
LABEL maintainer="gildong@example.com"

# 이미지의 간략한 용도 설명 기입
LABEL description="사내 프론트엔드 배포용 기본 베이스 이미지"
```

샘플 B. 백슬래시(\)를 활용한 멀티라인 작성

```
FROM ubuntu:22.04

# 백슬래시를 이용해 여러 메타데이터를 깔끔하게 정돈하여 기입
LABEL vendor="Acme Corporation" \
      version="1.0.3" \
      release-date="2026-07-10" \
      is-production="true"
```

[실습] 이미지 빌드 및 라벨 확인하기

Dockerfile 만들기

```
ARG UBUNTU_VERSION=22.04

FROM ubuntu:${UBUNTU_VERSION}

RUN apt-get update && apt-get install -y curl lsb-release
ARG UBUNTU_VERSION
RUN echo "현재 빌드에 사용된 ubuntu version: ${UBUNTU_VERSION}"

LABEL maintainer="himang10@gmail.com"
LABEL description="SKALA Linux Version"
```

빌드 및 라벨 확인하기

```
> docker build --tag linux-container:1.0 --build-arg UBUNTU_VERSION=24.04 .
> docker inspect linux-container:1.0
```

CMD 명령어

컨테이너가 시작될 때 실행할 기본 명령과 인자를 지정하는 데 사용하는 명령어

```
CMD ["executable", "param1", "param2"]
CMD executable param1 param2
```

주요 핵심 특징

- **컨테이너 실행 시점 명령어**: 컨테이너가 실행될때(docker run) 기본 명령어와 인자값을 정의
- **가장 마지막 CMD 유효**: CMD가 여러 개일때 가장 마지막에 작성된 CMD 딱 하나만 유효
- **오버라이딩(덮어쓰기) 가능**: docker run <이미지> <새로운_명령어>와 같이 뒤에 새로운 인자를 주면, Dockerfile에 적어둔 CMD 내용은 완전히 무시되고 사용자가 입력한 명령어가 대신 실행

CMD 명령어 활용 예시

일반적 활용 예시

```
FROM nginx:alpine

# 컨테이너가 켜질 때 Nginx 웹 서버가 백그라운드가 아닌 포어그라운드로 실행되도록 설정
CMD ["nginx", "-g", "daemon off;"]
```

[실습] 이미지 빌드 및 확인하기

Dockerfile 만들기

```
ARG UBUNTU_VERSION=22.04

FROM ubuntu:${UBUNTU_VERSION}

RUN apt-get update && apt-get install -y curl lsb-release nginx
ARG UBUNTU_VERSION
RUN echo "현재 빌드에 사용된 ubuntu version: ${UBUNTU_VERSION}"

LABEL maintainer="himang10@gmail.com"
LABEL description="SKALA Linux Version”


CMD ["nginx", "-g", "daemon off;"]
```

[실습] 이미지 빌드 및 확인하기

빌드 및 확인하기

```
> docker build --tag linux-container:1.0 .
```

실행 및 확인하기

```
> docker run -d --name linux-container -p 8888:80 linux-container:1.0
CONTAINER ID   IMAGE                 COMMAND                  CREATED          STATUS          PORTS
NAMES
28ecbc410e64   linux-container:1.0   "nginx -g 'daemon of…"   17 seconds ago   Up 16 seconds   0.0.0.0:8888->80/tcp,
[::]:8888->80/tcp       linux-container


> docker ps
```

컨테이너 내 프로세스에 접속하기

Browser 를 통해 http://localhost:8888 접속하기

종료 및 삭제

```
> docker stop linux-container
> docker rm linux-container
```

EXPOSE 명령어

```
EXPOSE <port> [<port>/<protocol>...]
```

프로토콜을 지정하지 않으면 기본적으로 TCP가 적용 (예: EXPOSE 80은 EXPOSE 80/tcp , udp, sctp 와 동일)

주요 핵심 특징

- 문서화 역할 (Informational): 이 컨테이너가 런타임에 어떤 포트를 열고 Listen하는지 단순 안내

- 호스트 포트 개방 없음 (No Auto-Publish): 단순 정보성 정보 표시 (inspect 에 표시)

  실제로 포트를 외부와 연결하려면 컨테이너를 실행할 때 docker run -p 옵션을 지정

EXPOSE 명령어 활용 예시

작성 방법

```
FROM node:18
WORKDIR /app
COPY . .
# 컨테이너 내부 애플리케이션이 3000번 포트를 쓴다고 명시
EXPOSE 3000
CMD ["node", "server.js"]
```

이미지 확인

```
docker image inspect --format='{{.Config.ExposedPorts}}' <이미지_이름_또는_ID>
```

실행 방법

```
# 호스트의 80포트를 컨테이너의 3000포트와 연결 (가장 많이 쓰이는 구조)
docker run -d -p 80:3000 내_노드_이미지

# 또는 EXPOSE된 포트를 알아서 호스트의 랜덤 포트에 연결하고 싶을 때 (-P 대문자)
docker run -d -P 내_노드_이미지
```

[실습] 이미지 빌드 및 확인하기

Dockerfile 만들기

```
ARG UBUNTU_VERSION=22.04

FROM ubuntu:${UBUNTU_VERSION}

RUN apt-get update && apt-get install -y curl lsb-release nginx
ARG UBUNTU_VERSION
RUN echo "현재 빌드에 사용된 ubuntu version: ${UBUNTU_VERSION}"

LABEL maintainer="himang10@gmail.com"
LABEL description="SKALA Linux Version”

EXPOSE 8080
EXPOSE 80/tcp

CMD ["nginx", "-g", "daemon off;"]
```

빌드 및 라벨 확인하기

```
> docker build --tag linux-container:1.0 --build-arg UBUNTU_VERSION=24.04 .
> docker image inspect --format='{{.Config.ExposedPorts}}'  linux-container:1.0
```

manifest.json + Dockerfile (ENV, ENTRYPOINT, CMD, WORKDIR, EXPOSE, LABEL Etc)

COPY 명령어

```
COPY [--chown=<user>:<group>] <src>... <dest>
```

주요 핵심 특징

- **로컬 파일 복사**: 로컬 컴퓨터에 있는 파일이나 디렉토리를 컨테이너 내부 파일시스템으로 복사

- **빌드 콘텍스트 기준**: <src>는 docker build를 실행하는 디렉토리 기준의 상대 경로
  (예: ../ 등을 통해 상위 디렉토리의 파일은 가져올 수 없음)

- **--from 플래그 지원 (멀티 스테이지 빌드의 핵심)**: 이전 빌드 스테이지에서 생성된 산출물(FROM ... AS <name>)을 타겟 스테이지로 복사해 올 때 사용되는 유일한 명령어

COPY 명령어 활용 예시

소스 및 패키지 복사

```
FROM node:18-alpine
WORKDIR /app

# 1. 특정 소유자 권한으로 package.json 복사
COPY --chown=node:node package.json .

# 2. 멀티 스테이지 환경에서 builder 스테이지의 dist 폴더 내용을 복사
COPY --from=builder /app/dist ./dist
```

WORKDIR 명령어

```
WORKDIR <container 내 작업 디렉토리>
```

디렉토리는 절대 경로 및 상대 경로 가능

주요 핵심 특징

- **작업 디렉토리(기준점) 고정**: 리눅스의 cd 명령어처럼 이후에 나오는 명령어들이 실행될 컨테이너 내부의 기본 작업 경로를 설정. 이 명령어 이후의 RUN, CMD, ENTRYPOINT, COPY, ADD 등은 모두 이 경로를 기준으로 삼아 동작

- **자동 디렉토리 생성**: 지정한 경로가 이미지 내부에 존재하지 않더라도 도커가 빌드 과정에서 자동으로 폴더를 생성

- **환경변수(ENV) 지원**: WORKDIR 경로 설정 시 환경 변수 사용 가능

WORKDIR 명령어 활용 예시

작성 방법

```
# /usr/src/app 폴더가 자동으로 생성되고, 작업 기준점이 여기로 바뀝니다.
WORKDIR /usr/src/app

# 현재 작업 경로(.)인 /usr/src/app 폴더 내부로 로컬 파일들을 복사합니다.
COPY . .
```

경로 설정 방법

```
# 1. 환경변수로 루트 경로 정의
ENV APP_ROOT=/var/www

# 2. /var/www 폴더로 이동 (자동 생성)
WORKDIR $APP_ROOT

# 3. 상대 경로 입력 시, 직전 경로를 기준으로 이어 붙습니다. (/var/www/html)
WORKDIR html

# 현재 위치를 출력하면 /var/www/html 이 나옵니다.
RUN pwd
```

[실습] 이미지 빌드 및 확인하기

Dockerfile 만들기

```
ARG UBUNTU_VERSION=22.04

FROM ubuntu:${UBUNTU_VERSION}

RUN apt-get update && apt-get install -y curl lsb-release
ARG UBUNTU_VERSION
RUN echo "현재 빌드에 사용된 ubuntu version: ${UBUNTU_VERSION}"

LABEL maintainer="himang10@gmail.com"
LABEL description="SKALA Linux Version”

EXPOSE 8080/tcp

# nginx 기본 웹루트로 이동 후 index.html 복사 (default.conf 수정 없이 index.html만 교체)
WORKDIR /var/www/html
COPY index.html .

CMD ["nginx", "-g", "daemon off;"]
```

[실습] 이미지 빌드 및 확인하기

index.html

참고: 01.answer-code/02.command/index.html

```
<!DOCTYPE html>
<html lang="ko">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>SKALA Container</title>
</head>
<body>
    <h1>SKALA Container 에 오신것을 환영합니다.</h1>
</body>
</html>
```

[실습] 이미지 빌드 및 확인하기

빌드 및 확인하기

```
> docker build --tag linux-container:1.0 .
> docker image inspect --format='{{.Config.ExposedPorts}}'  linux-container:1.0
```

실행 및 확인하기

```
> docker run -d --name linux-container -p 8888:80 linux-container:1.0
CONTAINER ID   IMAGE                 COMMAND                  CREATED          STATUS          PORTS
NAMES
28ecbc410e64   linux-container:1.0   "nginx -g 'daemon of…"   17 seconds ago   Up 16 seconds   0.0.0.0:8888->80/tcp,
[::]:8888->80/tcp       linux-container


> docker ps
```

컨테이너 내 프로세스에 접속하기

Browser 를 통해 http://localhost:8888 접속하기

종료 및 삭제

```
> docker stop linux-container
> docker rm linux-container
```

USER 명령어

```
USER <user>[:<group>]
USER <uid>[:<gid>]
```

주요 핵심 특징

- **런타임 권한 제어 (보안의 핵심):** 이 명령어 이후에 배치되는 RUN, CMD, ENTRYPOINT 등이 실행될 컨테이너 내부의 계정 권한(사용자 및 그룹)을 변경

- **최소 권한 원칙 (Least Privilege):**
  - 도커 컨테이너는 기본적으로 root(최상위 관리자) 권한으로 실행되므로 보안에 취약
  - 실무에서는 권한이 제한된 일반 사용자 계정으로 전환하기 위해 USER를 필수로 사용

- **계정 자동 생성 없음**: 반드시 RUN 명령어로 OS에 맞는 계정 생성 명령(useradd 등)을 먼저 실행

USER 명령어 활용 예시

작성 방법

```
FROM alpine:latest
WORKDIR /app

# 1. 컨테이너 내부에서 사용할 일반 유저(appuser) 계정 생성
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

# 2. 애플리케이션 소스 복사 후 소유권을 appuser로 변경
COPY --chown=appuser:appgroup . .

# 3. root 계정에서 일반 유저 계정으로 전환 (보안 조치)
USER appuser

# 이제부터 실행되는 프로세스는 root 권한이 아닌 appuser의 제한된 권한으로 구동됩니다.
CMD ["node", "server.js"]
```

ENV 명령어

```
ENV <key>=<value>
ENV <key1>=<value1> <key2>=<value2> ...
```

주요 핵심 특징

- **영구적인 환경변수 설정**: 컨테이너 내부 시스템 전체에서 참조할 환경변수를 정의

- **빌드부터 런타임까지 유지**:

  ENV로 선언된 변수는 최종 완성된 이미지가 컨테이너로 실행되는 런타임 시점(docker run)까지 영구적으로 유지

- **실행 시점 오버라이딩**: 컨테이너를 실행할 때 docker run --env <key>=<value> 플래그를 주면 Dockerfile에 선언된 값을 동적으로 덮어쓸 수 있어 유연한 설정 관리가 가능

ENV 명령어 활용 예시

작성 방법

```
FROM openjdk:17-slim

# 1. 런타임 및 빌드 시점에 모두 유지될 환경변수 정의
ENV APP_HOME=/opt/myapp \
    SPRING_PROFILES_ACTIVE=prod \
    JAVA_OPTS="-Xms512m -Xmx1024m"

# 2. 위에서 정의한 ENV 변수를 다른 도커 지시어에서 즉시 사용
WORKDIR $APP_HOME

COPY target/*.jar app.jar

# 3. 자바 실행 시 환경변수($JAVA_OPTS)를 바인딩하여 기동
CMD ["sh", "-c", "java $JAVA_OPTS -jar app.jar"]
```

[실습] 이미지 빌드 및 확인하기

앞의 Dockerfile을 기준으로 Dockerfile.user 파일 복사하기

```
> cp Dockerfile Dockerfile.user
```

Dockerfile.user 확장하기

```
ARG UBUNTU_VERSION=22.04

FROM ubuntu:${UBUNTU_VERSION}
…

EXPOSE 8080/tcp
EXPOSE 80/tcp

# nginx 기본 웹루트로 이동 후 index.html 복사 (default.conf 수정 없이 index.html만 교체)
WORKDIR /var/www/html
COPY index.html .

# skala 그룹 및 사용자 추가
RUN groupadd skala && \
    useradd --create-home --shell /bin/bash --gid skala skala

# nginx가 런타임에 쓰는 디렉터리 소유권을 skala로 변경 (root 소유 상태로는 mkdir 권한 오류 발생)
RUN chown -R skala:skala /var/lib/nginx /var/log/nginx /run

# skala 사용자로 전환
USER skala

CMD ["nginx", "-g", "daemon off;"]
```

[실습] 이미지 빌드 및 확인하기

빌드 및 확인하기

```
> docker build --tag linux-container:1.0-USER -f Dockerfile.user .
```

실행하고 그 결과 확인해보기 (에러 발생)

```
> docker run -it --name linux-container -p 8888:80 linux-container:1.0
nginx: [alert] could not open error log file: open() "/var/log/nginx/error.log" failed (13: Permission denied)
2026/07/12 09:01:57 [warn] 1#1: the "user" directive makes sense only if the master process runs with super-user
privileges, ignored in /etc/nginx/nginx.conf:1
2026/07/12 09:01:57 [emerg] 1#1: mkdir() "/var/lib/nginx/body" failed (13: Permission denied)
> docker ps -a
```

USER skala를 # 한후 다시 실행해보자 (기존 Root로 전환)

```
…
# USER skala

CMD ["nginx", "-g", "daemon off;"]
```

[참고] USER skala로 에러 없이 실행하는 방법

Dockerfile.user 에 아래 내용을 추가해서 실행해보기 (동작 확인)

```
ARG UBUNTU_VERSION=22.04

FROM ubuntu:${UBUNTU_VERSION}
…

EXPOSE 8080/tcp
EXPOSE 80/tcp

# nginx 기본 웹루트로 이동 후 index.html 복사 (default.conf 수정 없이 index.html만 교체)
WORKDIR /var/www/html
COPY index.html .

# skala 그룹 및 사용자 추가
RUN groupadd skala && \
    useradd --create-home --shell /bin/bash --gid skala skala

# nginx가 런타임에 쓰는 디렉터리 소유권을 skala로 변경 (root 소유 상태로는 mkdir 권한 오류 발생)
RUN chown -R skala:skala /var/lib/nginx /var/log/nginx /run

# skala는 80번(privileged) 포트 바인딩이 불가하므로 기본 사이트 설정을 8080으로 변경
RUN sed -i \
    -e 's/listen 80 default_server;/listen 8080 default_server;/' \
    -e 's/listen \[::\]:80 default_server;/listen [::]:8080 default_server;/' \
    /etc/nginx/sites-enabled/default

USER skala

CMD ["nginx", "-g", "daemon off;"]
```

ENTRYPOINT 명령어

```
ENTRYPOINT ["executable", "param1", "param2"]
ENTRYPOINT executable param1 param2
```

주요 핵심 특징

- **실행 명령 고정** : 컨테이너가 시작될 때 무조건 실행되어야 하는 메인 프로세스를 설정

  컨테이너가 어떤 역할을 하는 도구인지 본질을 정의하는 명령어

- **오버라이딩(덮어쓰기)의 어려움**: 사용자가 실행 시 뒤에 입력한 값들은 모두 ENTRYPOINT 명령어 뒤에 인자로 추가 (강제로 바꾸려면 --entrypoint 플래그를 명시 필요)

ENTRYPOINT 명령어 활용 예시

작성 방법

```
FROM alpine:latest

# 1. 실행할 메인 프로그램(컨테이너의 본질)을 고정합니다.
ENTRYPOINT ["ping"]

# 2. 사용자가 아무것도 입력하지 않았을 때 적용될 '기본 인자값'을 설정합니다.
CMD ["-c", "3", "localhost"]
```

[실습] 이미지 빌드 및 확인하기

Dockerfile.entrypoint file 을 아래와 같이 작성해보자

```
FROM alpine:latest

# 1. 실행할 메인 프로그램(컨테이너의 본질)을 고정합니다.
ENTRYPOINT ["ping"]

# 2. 사용자가 아무것도 입력하지 않았을 때 적용될 '기본 인자값'을 설정합니다.
CMD ["-c", "3", "skala-fileserver.skala-ai.com"]
```

빌드하기

```
> docker build --tag check-ping -f Dockerfile.entrypoint .
```

[실습] 이미지 빌드 및 확인하기

명령어 처럼 실행해보기

```
# ping -c 3 localhost (CMD에 적어둔 기본값이 그대로 붙어서 작동)
> docker run --rm check-ping
```

google.com 에 poing 해보기

```
# 최종 실행 명령어: ping google.com
> docker run --rm check-poing google.com
```

사용자가 입력한 google.com이 Dockerfile의 CMD를 완전히 밀어내고(오버라이딩), 고정된 ENTRYPOINT 뒤에 붙어서 실행

ENTRYPOINT vs CMD

구분	ENTRYPOINT 지시어	CMD 지시어
개념적 의미	컨테이너가 실행할 **"메인 명령(본질)"**	컨테이너가 실행할 **"기본 인자(기본값)"**
docker run 뒤 인자 입력	입력한 값이 ENTRYPOINT 뒤에 **매개변수로 추가됨**	Dockerfile의 CMD 내용은 **완전히 무시되고 대체됨**
강제 오버라이딩 방법	--entrypoint 플래그를 명시해야만 가능	플래그 없이 이미지 이름 뒤에 명령어 입력 시 자동 대체
가장 흔한 실무 형태	ENTRYPOINT ["nginx"], ENTRYPOINT ["python"]	CMD ["--help"], CMD ["app.py"]

[검토] 눈으로 확인

```
FROM python:3.10-alpine

ENTRYPOINT ["echo", "This"]

# 기본 명령 설정

CMD ["is", "the default command"]
```

```
> docker run --rm test-image
```

출력 결과는 ?

```
> docker run --rm test-image  " is the custom CMD "
```

출력 결과는 ?

파일/레이어 vs 컨테이너 실행 설정 정보

이미지 레이어 생성 명령

읽기 전용 레이어(Layer / RootFS.Layers)를 만들어내는 명령어

명령	생성 대상/목적 설명
RUN	패키지 설치(apt-get, npm install), 소스코드 빌드, 스크립트 실행 등을 수행하여 **명령어 실행 결과로 생성·변경된 파일들을 새 레이어로 저장**
COPY	호스트 머신(빌드 컨텍스트)의 파일이나 디렉터리를 이미지 내부 경로로 복사하여 **복사된 파일들로 구성된 새 레이어를 생성**
ADD	COPY와 유사하게 호스트 파일을 복사하며, 추가로 **압축 파일(tar) 자동 해제** 및 **원격 URL 파일 다운로드**를 수행하여 **파일시스템에 새 레이어로 추가**

컨테이너 실행 설정 명령

컨테이너 실행을 위한 환경 변수, 실행 명령어, $HOME, user 등
container 실행 config를 생성 명령어 (config.json)

ENTRYPOINT, CMD, ENV, WORKDIR, EXPOSE, USER, VOLUME, LABEL ...

컨테이너 볼륨 연결하기



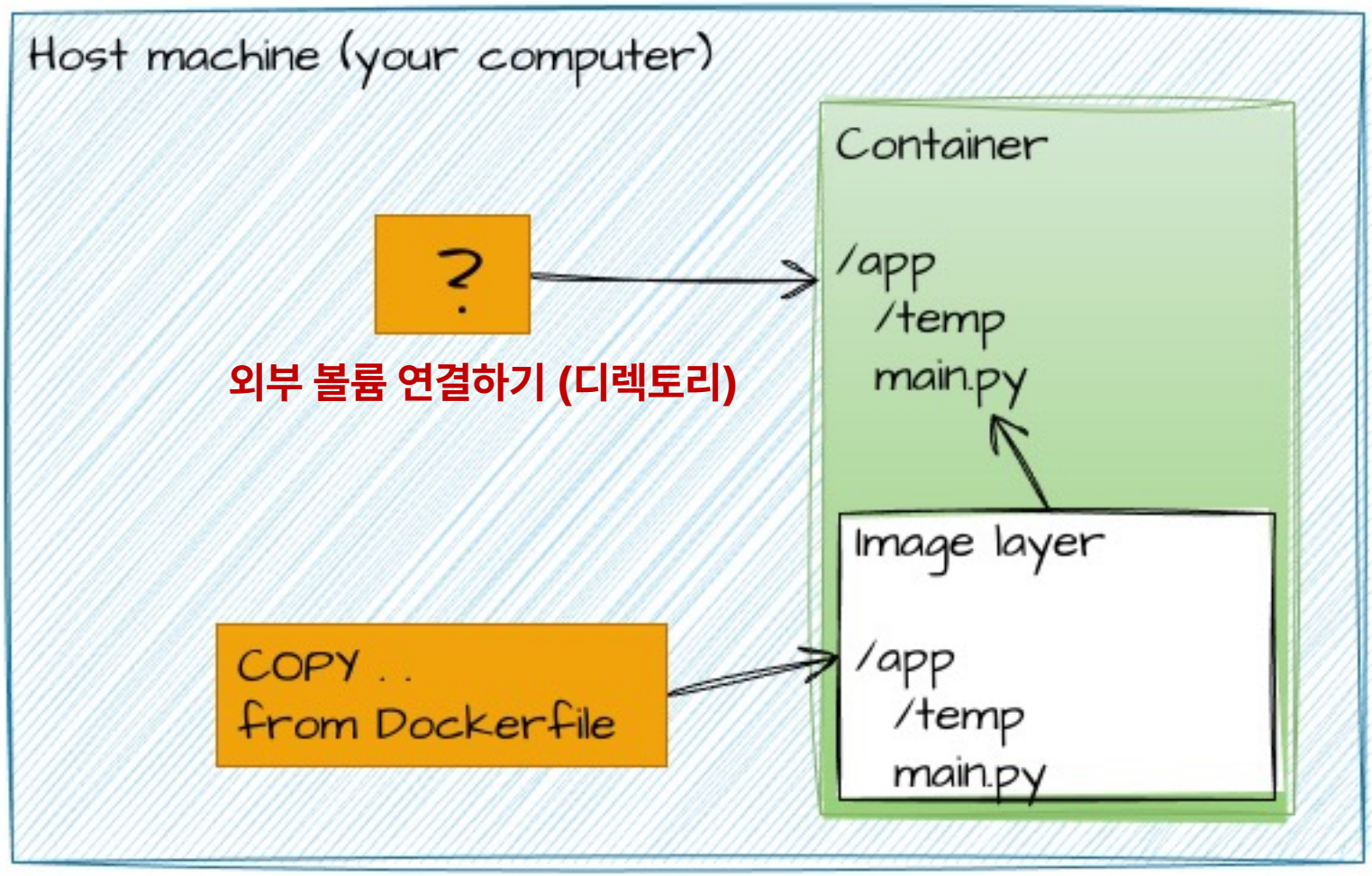
본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

컨테이너에 Persistent Volume 연결하기

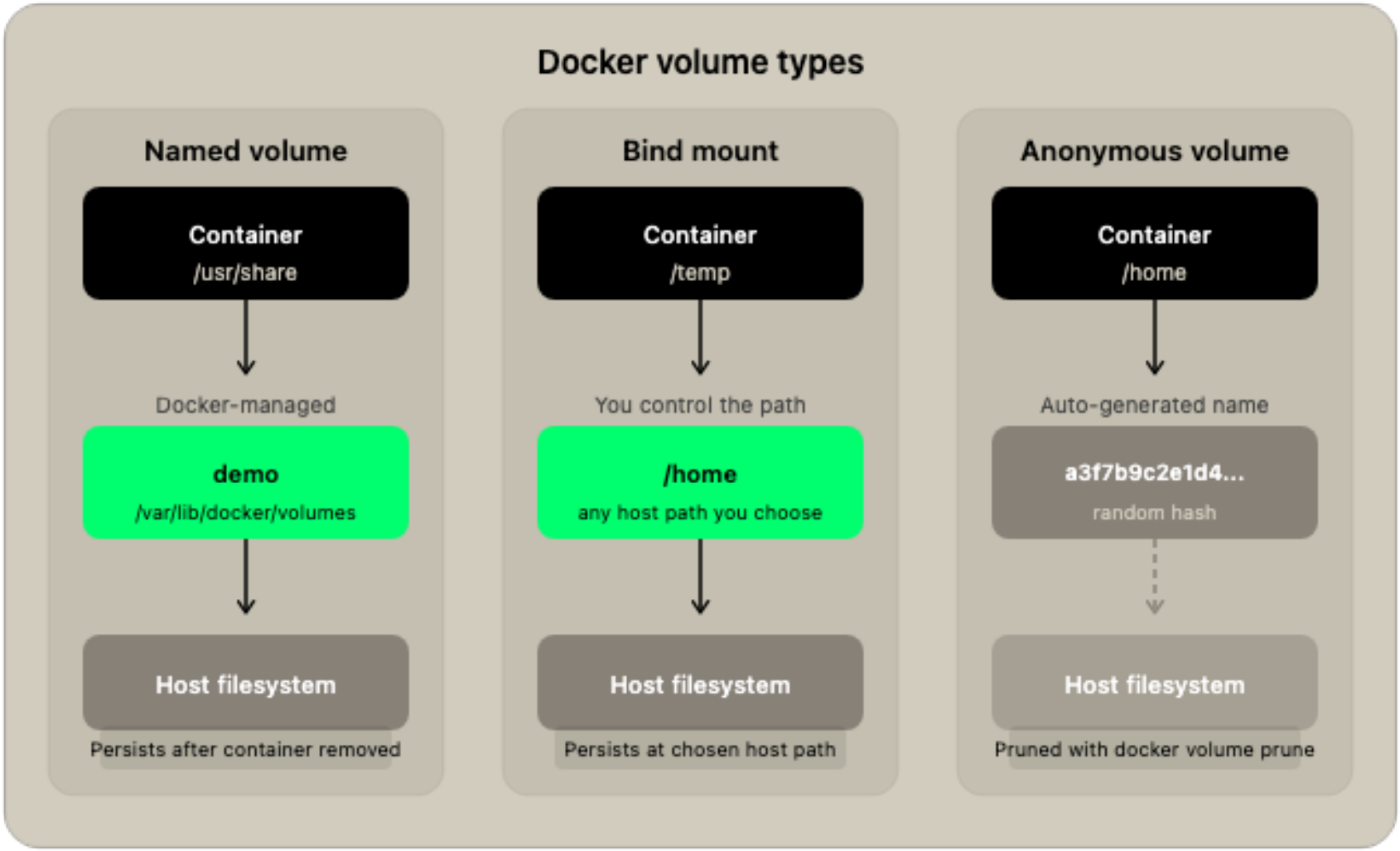
컨테이너는 언제든 삭제 될수 있는 일회성 (Ephermeral) 환경

컨테이너가 종료 후 재실행되더라도 기존 데이터를 유지하기 위한 방법 필요



볼륨 타입

- **Named Volume:** Docker를 포함한 컨테이너 런타임에서 관리하는 이름 있는 Volume
- **Bind Mount :** 외부 호스트 디렉토리와 Container 내부 디렉토리를 마운트하는 방식
- **Anyonymous Volume:** 이름없이 임시 생성 (Dockerfile 내부 VOLUME 명령어로 생성)



볼륨 타입 : Named Volume

```
> docker run -it --name busybox -v demo:/usr/share busybox
```

busybox container 내부 들여다 보기

```
/ # mount | grep /usr/share
/dev/vda1 on /usr/share type ext4 (rw,relatime,discard)
/ # exit
```

busybox config 내 볼륨 마운트 확인하기

```
> docker inspect busybox --format '{{json .Mounts}}' | jq
[
  {
    "Type": "volume",
    "Name": "demo",
    "Source": "/var/lib/docker/volumes/demo/_data",
    "Destination": "/usr/share",
    "Driver": "local",    # local : host, ex) nfs, aws efs
    "Mode": "z",
    "RW": true,
    "Propagation": ""
  }
]


> docker rm busybox
```

볼륨 타입 : Bind Mount

```
> docker run -it --name busybox -v $(pwd):/usr/share busybox
```

busybox container 내부 들여다 보기 (macbook 의 /Users 아래 디렉토리에 마운트되어 있음)

```
/ # mount | grep /usr/share
/run/host_mark/Users on /usr/share type fakeowner (rw,nosuid,nodev,relatime,fakeowner)
/ # touch /usr/share/bind-mount.txt
/ # exit
```

busybox config 내 볼륨 마운트 확인하기

```
> docker inspect busybox --format '{{json .Mounts}}' | jq
[
 {
    "Type": "bind",
    "Source": "/Users/himang10/mydev/skala/skala-cloud/skala-4/cloud/01.skala-container/02.answer-code/04.volumes",
    "Destination": "/usr/share",
    "Mode": "",
    "RW": true,
    "Propagation": "rprivate"
  }
]

> ls bind-mount.txt
> docker rm busybox
```

[실습 준비] 컨테이너 실습 환경 구성

참고: 04.volumes/Dockerfile.python

현재 작업 디렉토리에 다음 사항을 사전에 준비하기

1. 01.answer-code/04.volumes/webserver.py 파일을 자신의 현재 디렉토리로 복사하자

```
> cp ${01.answer-code/04.volumes/webserver.py}  .
> ls webserver.py
webserver.py
```

2. 실습 디렉토리 만들기

```
> mkdir db-data
> mkdir mydata
```

[실습] Mariadb 에 BindMount 로 볼륨 연결

mariadb에 volume 연결하기

```
> mkdir db-data
 > docker run -d \
   --name mariadb \
   -e MYSQL_ROOT_PASSWORD=password \
   -e MYSQL_DATABASE=skala \
   -e MYSQL_USER=user \
   -e MYSQL_PASSWORD=password \
   --network skala \
   -p 3306:3306 \
   -v $(pwd)/db-data:/var/lib/mysql \
  mariadb:latest
>  docker ps
CONTAINER ID   IMAGE           COMMAND                  CREATED         STATUS         PORTS                                   NAMES
4d58e441d58d   mariadb:latest   "docker-entrypoint.s..."   2 seconds ago   Up 2 seconds   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp   mariadb
> ls ./db-data
aria_log.00000001  ddl_recovery.log  ib_logfile0  ibtmp1              multi-master.info  performance_schema  sys     undo001  undo003
aria_log_control   ib_buffer_pool    ibdata1      mariadb_upgrade_info  mysql              skala               tc.log  undo002
```

python 실행 컨테이너 만들기

앞에서 실행했던 linux-container:1.0을 사용해보자

```
# container와 공유하기 위한 디렉토리 만들기
➢ mkdir -p ./mydata

# docker 실행  -- windows에서 아래 방식이 동작하지 않으면 : ${PWD} 적용
➢ docker run -d \
    --name linux-container \
    -p 8080:8080 \
    -p 8888:80 \
    -v $(pwd)/mydata:/mydata \
    linux-container:1.0
94f92313eb5698e53180083f63b06307f1b283e2e3425d8a3c43060b3e944f38.   ←-  Container ID (64 자리)
# 실행되고 있는 컨테이너에 접속하기
➢  docker ps
CONTAINER ID  IMAGE                          COMMAND               CREATED      STATUS          PORTS       NAMES
94f92313eb56  localhost/ skala-ubuntu:1.0  sh -c echo 'start...  3 hours ago  Up 3 hours ago              my-first-container
```

python 실행 컨테이너 만들기

container 내 mydata에 webserver.py 복사하기

terminal 1 (컨테이너 내부)

1) linux-container 내에 접속하기

```
# 실행되고 있는 컨테이너에 접속하기
>  docker exec -it linux-container  /bin/bash
-----------------------------------------------

# 컨테이너 내부
himang10-skala-pro-docker-rde-0:/# cd mydata
himang10-skala-pro-docker-rde-0:/mydata# ls
=== 아무것도 없다.
```

3) linux-container 내에 webserver.py가 복사 여부 확인

```
himang10-skala-pro-docker-rde-0:/# ls
webserver.py
```

terminal 2 로컬의 mydata 디렉토리)

2) 로컬에서 webserver.py를 ./mydata에 복사하기

```
## 공유 디렉토리 /mydata에 webserver.py를 복사하기
> cp    webserver.py ./mydata
> cd   ./mydata
> ls
webserver.py
```

python 실행 컨테이너 만들기

./mydata 내의 webserver.py를 실행시키고 외부에서 접속하기

```
root@83c1e07f154b:/mydata# apt-get update
Hit:1 http://ports.ubuntu.com/ubuntu-ports jammy InRelease
Get:2 http://ports.ubuntu.com/ubuntu-ports jammy-updates InRelease [128 kB]
…


root@83c1e07f154b:/mydata# apt-get install -y python3 python3-pip
…
which you live ..
5. asia
..
68.seoul 선택

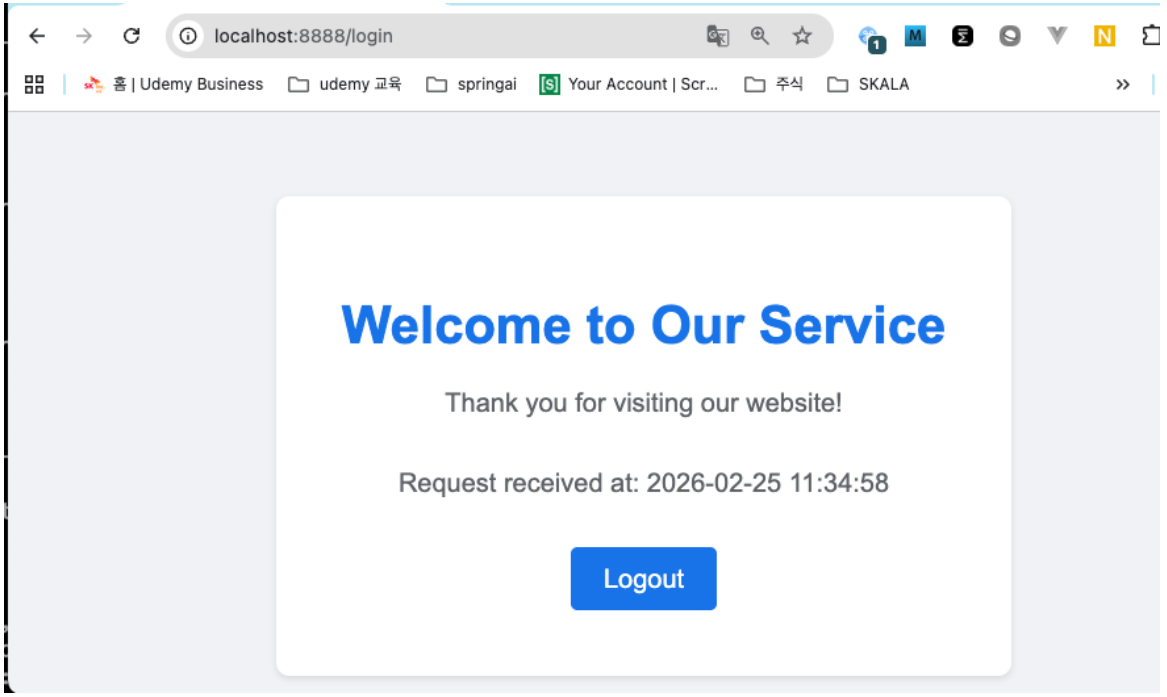
root@83c1e07f154b:/# cd /mydata
root@83c1e07f154b:/mydata# ps –ef

root@83c1e07f154b:/# python3 /mydata/webserver.py
Starting server on port 8080...
Access the server at http://localhost:8080/login
```

python 실행 컨테이너 만들기

컨테이너 내부에서 Webserver 실행 결과 확인하기

http://localhost:8080/ 로 browser로 접속해보기



[실습] 파이썬 웹서버용 Dockerfile 만들기

앞의 수동 실습 환경 구성을 Dockerfile에 명령어로 추가해보자

1. 기존 Dockerfile에 다음 사항을 추가해보자

```
# WORKDIR /var/www/html
# COPY index.html .

RUN apt-get install -y python3 python3-pip

WORKDIR /app
COPY webserver.py .

#CMD ["nginx", "-g", "daemon off;"]
CMD ["python3", "webserver.py"]
```

2. Dockerfile을 빌드하고 컨테이너를 8080 포트로 실행해보자

```
> docker build --tag linux-container:1.0  .
> docker run -d --name linux-container -p 8080:8080 linux-container:1.0
```

http://localhost:8080/ 로 browser로 접속해보기



CMD 명령어 최적화

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

컨테이너 프로세스의 실행 방법

1. 컨테이너가 즉시 종료되지 않고 멈춰 있다 강제 종료되는 문제
2. 트래픽을 받고 처리 중 Graceful shutdown 되지 않고 KILL 되는 현상
3. 이로인한 트래픽 유실 발생 및 배포 속도 저하

이러한 문제는 ?

CMD 한줄 잘못 사용하는 경우 발생

CMD 최적화 방법

표준 명칭	사용 방법	특징
Exec Form (직접 실행)	CMD ["python3", "webserver.py"]	쉘 없이 앱이 직접 **PID 1 로 실행** **(강력 권고)**
Shell Form (포크 실행)	CMD ["/bin/sh", "-c", "python3 webserver.py"] *(또는 CMD python3 webserver.py)*	/bin/sh가 PID 1로 실행 실행 앱은 자식 프로세스로 분기 (**비권고**)
Shell with Exec (치환 실행)	CMD ["/bin/sh", "-c", "exec python3 webserver.py"]	쉘 환경변수 등을 처리한 뒤 exec로 부모 쉘을 앱(PID 1)으로 치환 (**조건부 권고**)

SIGTERM 신호 수신 코드 넣기

webserver.py에 SIGTERM 처리 handler를 등록

```
import signal

def run_server(port=8080):
    server_address = ('', port)
    httpd = HTTPServer(server_address, SimpleHTTPRequestHandler)

    # SIGTERM 신호 처리 함수 정의 및 등록
    def handle_sigterm(signum, frame):
        print(f'[{datetime.now().strftime("%Y-%m-%d %H:%M:%S")}] SIGTERM 신호 수신')
        time.sleep(1)
        print(f'[{datetime.now().strftime("%Y-%m-%d %H:%M:%S")}] SIGTERM 처리 종료')
        httpd.server_close()
        sys.exit(0)

    signal.signal(signal.SIGTERM, handle_sigterm)

    print(f'Starting server on port {port}...')
    print(f'Access the server at http://localhost:{port}/login')
    try:
        httpd.serve_forever()
    except KeyboardInterrupt:
        print('\nShutting down server...')
        httpd.server_close()
```

[실습 1] Exec Form (직접 실행)

기존 Dockerfile의 Command 부분은 python3 프로세스를 직접 호출하는 방식

```
ARG UBUNTU_VERSION=22.04

FROM ubuntu:${UBUNTU_VERSION}

...

RUN apt-get install -y python3 python3-pip

WORKDIR /app
COPY webserver.py .

CMD ["python3", "webserver.py"]
```

[실습 1] Exec Form (직접 실행)

컨테이너 접속하기

```
# docker image 검색
➢  docker images
# docker 실행
➢  docker run --rm -d --name linux-container linux-container:1.0
94f92313eb5698e53180083f63b06307f1b283e2e3425d8a3c43060b3e944f38.   ←-  Container ID (64 자리)

# 실행되고 있는 컨테이너에 접속하기
➢  docker ps
➢ docker exec -it linux-container /bin/bash
# linux-container 내부에서 동작하고 있는 지 확인
 # ps –ef
UID        PID  PPID  C STIME TTY          TIME CMD
root         1     0  0 23:34 ?        00:00:00 python3 webserver.py
root         7     0  0 23:35 pts/0    00:00:00 /bin/bash
root        15     7  0 23:35 pts/0    00:00:00 ps –ef

 # exit
```

[실습 1] Exec Form (직접 실행): SIGTERM 수신 확인

기존 Dockerfile의 Command 부분은 python3 프로세스를 직접 호출하는 방식

terminal 1

```
> docker logs -f linux-container
Starting server on port 8080...
Access the server at http://localhost:8080/login




# 즉시 수신
[2026-08-19 05:11:11] SIGTERM 신호 수신
[2026-08-19 05:11:12] SIGTERM 처리 종료
```

terminal 2

```
> docker stop linux-container
```

spring boot 과 fastapi 등 framework는 SIGTERM SIGNAL 수신시

- GRACEFull Shutdown 실행
- 자원 해지, 연결 해지, 수신 처리중 요청 처리 완료 등

[실습 2] Shell Form (Fork 실행)

Dockerfile의 Command를 아래와 같이 변경하고 컨테이너를 실행해보자

```
ARG UBUNTU_VERSION=22.04

FROM ubuntu:${UBUNTU_VERSION}

...

RUN apt-get install -y python3 python3-pip

WORKDIR /app
COPY webserver.py .

#CMD ["python3", "webserver.py"]
CMD ["/bin/sh", "-c", "python3 -u webserver.py"]
```

- docker stop 시 10초 대기
- kubectl 처리 시 : 30초 대기

기존에 동작하던 linux-container 중단 및 제거

```
> docker stop linux-container
> docker rm linux-container
```

[실습 2] Shell Form (Fork 실행)

컨테이너 접속하기

```
# docker image 빌드
➢  docker build --tag linux-container:1.1 .

# docker 실행
➢  docker run --rm -d --name linux-container linux-container:1.1
94f92313eb5698e53180083f63b06307f1b283e2e3425d8a3c43060b3e944f38.   ←-  Container ID (64 자리)

# 실행되고 있는 컨테이너에 접속하기
➢  docker ps
➢ docker exec -it python-container /bin/bash
# inux-container 내부에서 동작하고 있는 지 확인
 # ps –ef
UID        PID  PPID  C STIME TTY          TIME CMD
root         1     0  0 23:27 ?        00:00:00 /bin/sh -c python3 webserver.py
root         7     1  0 23:27 ?        00:00:00 python3 webserver.py
root         8     0  0 23:28 pts/0    00:00:00 /bin/bash
root        16     8  0 23:28 pts/0    00:00:00 ps –ef
 # exit
```

[실습 2] Shell Form (Fork 실행): SIGTERM 수신 확인

기존 Dockerfile의 Command 부분은 python3 프로세스를 직접 호출하는 방식

terminal 1

```
> docker logs -f linux-container
Starting server on port 8080...
Access the server at http://localhost:8080/login




# 아래의 로그 미 수신. 바로 SIGKILL 실행 됨
[2026-08-19 05:11:11] SIGTERM 신호 수신
[2026-08-19 05:11:12] SIGTERM 처리 종료
```

terminal 2

```
> docker stop linux-container
```

spring boot 과 fastapi 등 framework는 SIGTERM SIGNAL 미 수신

이로 인해 비정상 종료 (Graceful shutdown 미지원)

[실습 3] Shell with Exec (치환 실행)

Dockerfile의 Command를 아래와 같이 변경하고 컨테이너를 실행해보자

```
ARG UBUNTU_VERSION=22.04

FROM ubuntu:${UBUNTU_VERSION}

...

RUN apt-get install -y python3 python3-pip

WORKDIR /app
COPY webserver.py .

#CMD ["python3", "webserver.py"]
CMD ["/bin/sh", "-c", "exec python3 -u webserver.py"]
```

기존에 동작하던 linux-container 중단 및 제거

```
> docker stop linux-container
> docker rm linux-container
```

[실습 3] Shell with Exec (치환 실행)

컨테이너 접속하기

```
# docker image 빌드
➢  docker build --tag linux-container:1.2 .

# docker 실행
➢  docker run --rm -d --name linux-container linux-container:1.2
94f92313eb5698e53180083f63b06307f1b283e2e3425d8a3c43060b3e944f38.   ←-  Container ID (64 자리)
# 실행되고 있는 컨테이너에 접속하기
➢  docker ps
➢ docker exec -it python-container /bin/bash
# inux-container 내부에서 동작하고 있는 지 확인
 # ps –ef
UID        PID  PPID  C STIME TTY          TIME CMD
root         1     0  0 23:34 ?        00:00:00 python3 webserver.py
root         7     0  0 23:35 pts/0    00:00:00 /bin/bash
root        15     7  0 23:35 pts/0    00:00:00 ps –ef

 # exit
```

[실습 3] Shell with Exec (치환 실행): SIGTERM 수신 확인

기존 Dockerfile의 Command 부분은 python3 프로세스를 직접 호출하는 방식

terminal 1

```
> docker logs -f linux-container
Starting server on port 8080...
Access the server at http://localhost:8080/login




# SIGTERM 수신 처리
[2026-08-19 05:11:11] SIGTERM 신호 수신
[2026-08-19 05:11:12] SIGTERM 처리 종료
```

terminal 2

```
> docker stop linux-container
```

[참고] Terminal을 Open 하는 의미?

Terminal을 오픈한다는 의미는 Stdin 과 Stdout과 연결된 Shell Process를 실행한다는 의미

~/ # python fastapiserver.py

--> Shell: Child fork + exec(치환) + wait (대기)

fork는 자기와 닮은 아이가 태어남

exec는 에어리언!!!

~/ # python fastapiserver.py &

→ Child fork + exec

~/ # cat “hello”

~/ # docker exec -it container /bin/bash

- **-i (Keep STDIN open):**
  - 로컬 터미널의 Shell Process STDIN을 컨테이너 내부 프로세스의 STDIN으로 연결
  - 이 옵션이 없으면 입력이 컨테이너 안으로 전달되지 않음

- **-t (Allocate a pseudo-TTY):**
  - 컨테이너 내부에 가상 터미널(pty)을 생성



웹 서비스 실행 컨테이너 만들기

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

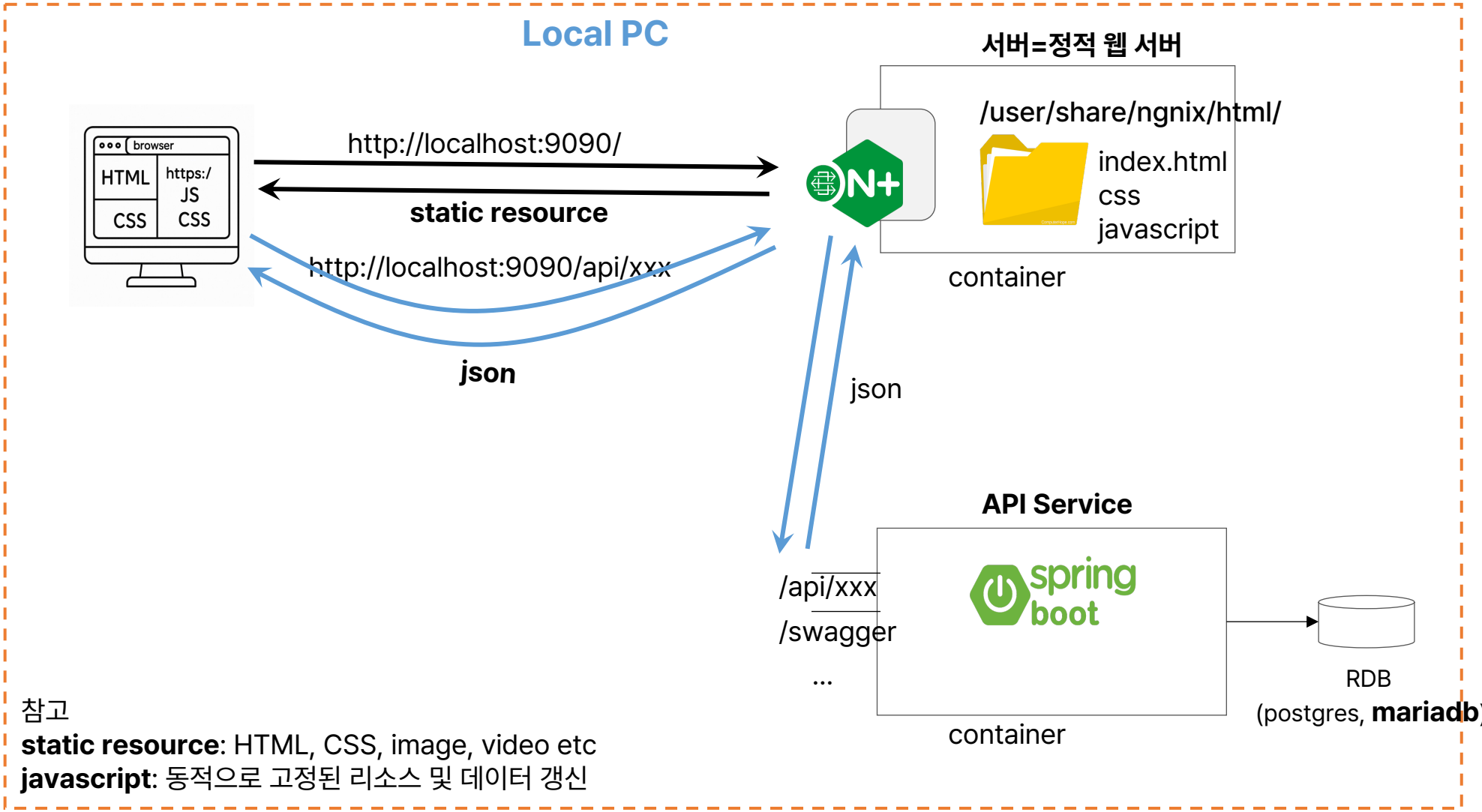
SK 주식회사 C&C

실습 시나리오

1. spring boot 컨테이너 빌드 및 실행
2. container network skala bridge 기반 Mariadb 연동하기
3. frontend 컨테이너 빌드 및 실행
4. vue.js Frontend 컨테이너 빌드 및 실행
5. 자신의 웹 서비스를 구성해보자

컨테이너 기반 웹 서비스 구성하기

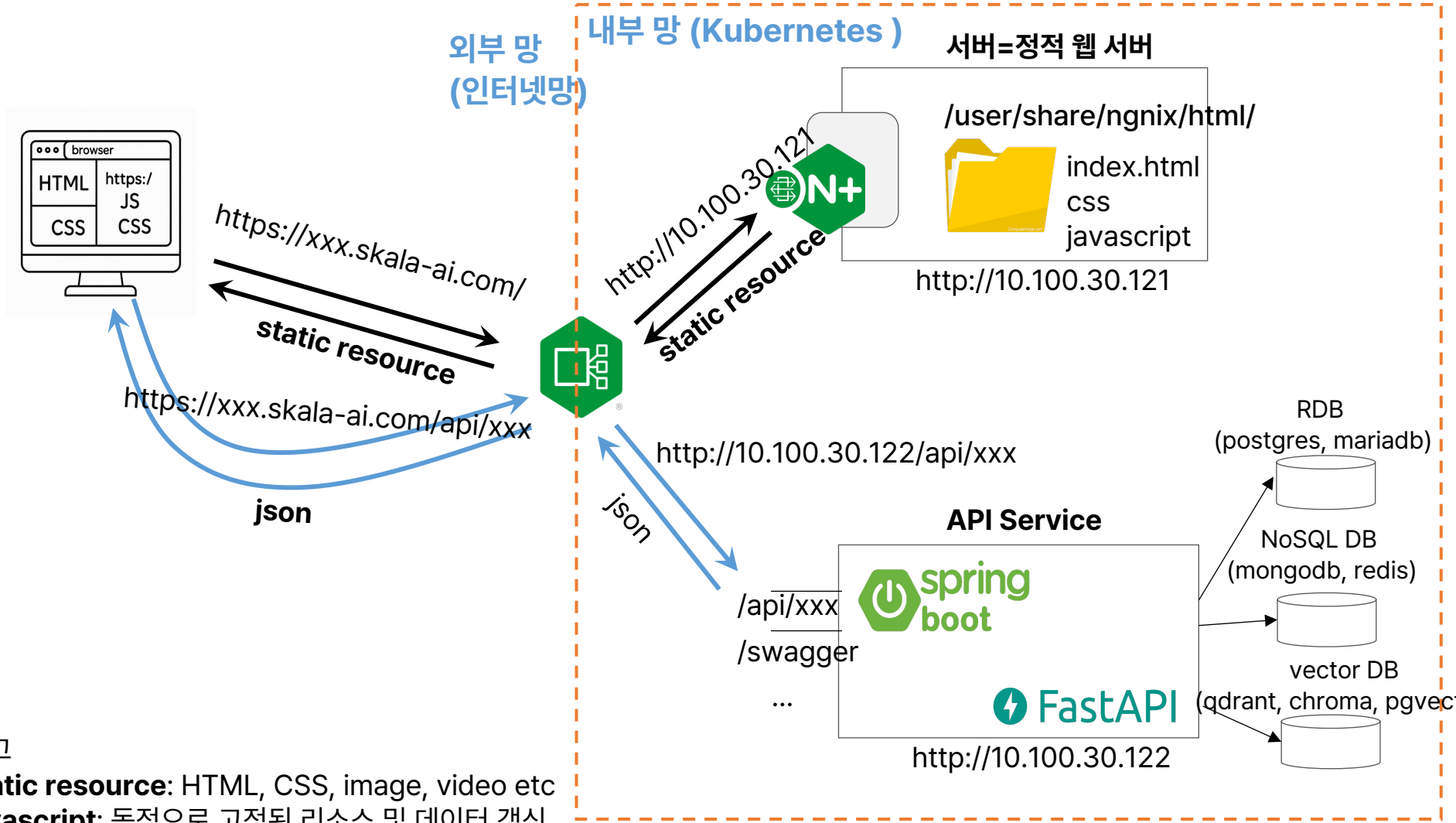
Frontend + Backend + Mariadb



참고
static resource: HTML, CSS, image, video etc
javascript: 동적으로 고정된 리소스 및 데이터 갱신

컨테이너 기반 웹 서비스 구성하기

Frontend + Backend + Mariadb



참고
static resource: HTML, CSS, image, video etc
javascript: 동적으로 고정된 리소스 및 데이터 갱신

[실습] spring boot 컨테이너 만들기

1.spring boot 소스 코드 위치

```
00.sample-container/01.spring-backend-v1.0
```

2. Dockerfile 확인하기

```
FROM eclipse-temurin:21-jre

# 작업 디렉토리 설정
WORKDIR /app

# JVM 힙 메모리 고정 설정 (load-memory 64MB 할당 + JVM 오버헤드 확보)
ENV JAVA_OPTS="-Xms256m -Xmx512m"

# 애플리케이션의 jar 파일을 컨테이너에 추가
ADD ./target/*.jar  app.jar

# 애플리케이션 실행 (JAVA_OPTS 환경변수 적용)
CMD ["sh", "-c", "exec java $JAVA_OPTS -jar app.jar"]
```

3. spring boot 이미지를 빌드해보자

```
> mvn clean install -DskipTests
> docker buildx build --tag spring-backend:1.0  .
```

[실습] spring boot 컨테이너 만들기

3. mariadb 가 실행되고 있는지 확인하기

```
> docker ps
865c08b411ca   mariadb:latest          "docker-entrypoint.s…"   28 minutes ago   Up 28
minutes   0.0.0.0:3306->3306/tcp, [::]:3306->3306/tcp   mariadb

> docker network ls
8c04df2f4086   skala       bridge    local
```

4. spring boot container 실행하기

```
> docker run -d \
  --name spring-backend \
  --network skala \
  -p 8080:8080 \
  -e SPRING_PROFILES_ACTIVE=local-mariadb \
  spring-backend:1.0
```

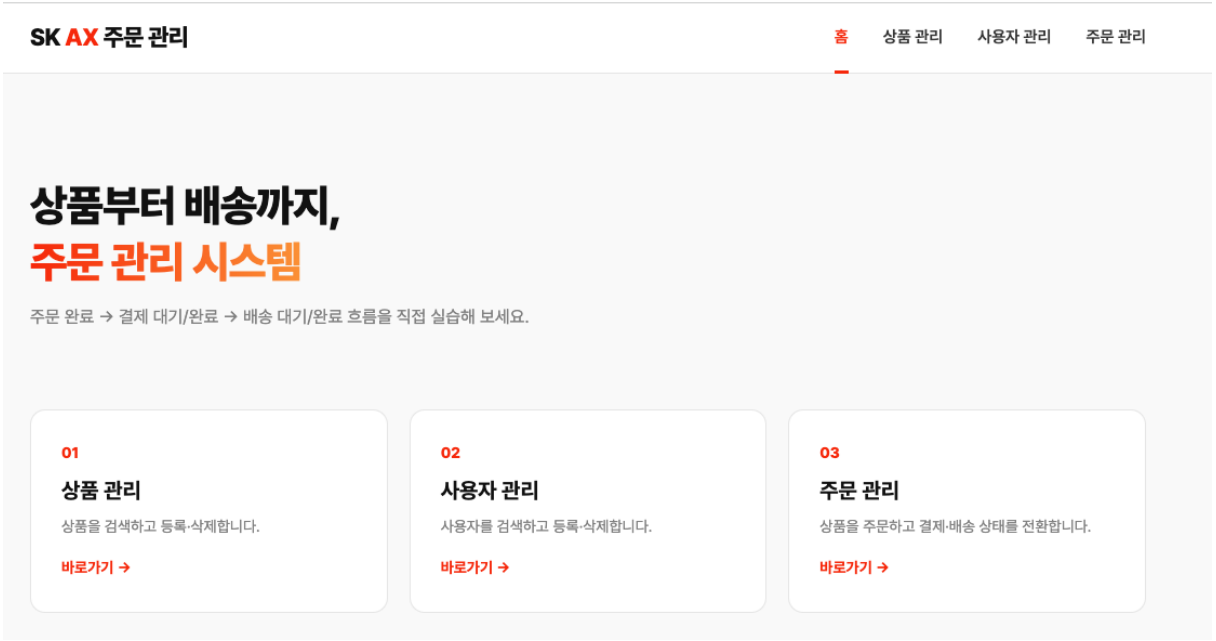
5. 확인하기

```
> docker ps
> docker logs spring-backend
```

[실습] spring boot Dockerfile 만들기

6. http://localhost:8080/ 로 browser로 접속해보기

사용자 관리 > 사용자 등록



7. 사용자 등록 후 DBeaver 를 통해 users table에 사용자 등록 현황 확인하기

- http://localhost:3306
- user name: user
- password: password
- database: skala

[참고] spring build & Container Build Dockerfile

mvn build 와 container build를 모두 지원하는 **Multi-Stage 방식**

```
FROM ubuntu:24.04 AS builder

RUN apt-get update && apt-get install -y openjdk-21-jdk maven \
    && apt-get clean && rm -rf /var/lib/apt/lists/*

WORKDIR /build

COPY pom.xml .
RUN mvn dependency:go-offline -q

COPY src ./src
RUN mvn package -DskipTests -q

FROM eclipse-temurin:25-jre

WORKDIR /app

ENV SPRING_PROFILES_ACTIVE=local

# builder 스테이지의 jar만 복사 (소스, JDK, Maven 캐시 제외)
COPY --from=builder /build/target/*.jar app.jar

ENTRYPOINT ["java", "-jar", "app.jar"]
```

참고: 01.training-code/execise-source/Dockerfile.multi-stage

[실습] Frontend 만들기

spring boot의 thymeleaf 코드를 외부 SPA 구조의 컨테이너로 전환해서 별도 배포해보자

참고: 00.sample-container/03.frontend/default.conf

1. default.conf

```
server {
    listen 80;

    # 정적 파일 서빙
    location / {
        root /usr/share/nginx/html;
        index index.html;
        try_files $uri $uri/ /index.html;

        expires -1;
        add_header Cache-Control "no-store, no-cache, must-revalidate";
        add_header Pragma "no-cache";
    }

    # Spring Boot API 프록시
    location /api {
        resolver 127.0.0.11 valid=10s;
        set $backend "spring-backend:8080";
        proxy_pass http://$backend;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
    }
}
```

[실습] Frontend 만들기

2. Dockerfile

참고: 00.sample-container/03.frontend/Dockerfile

```
FROM nginx:alpine

# 기본 nginx 설정 파일 제거
RUN rm /etc/nginx/conf.d/default.conf

# 커스텀 nginx 설정 파일 복사 (정적 서빙 + /api 프록시)
COPY default.conf /etc/nginx/conf.d/

# 정적 파일들을 nginx 기본 디렉토리로 복사
COPY src/ /usr/share/nginx/html/

# nginx 포트 노출
EXPOSE 80

# nginx 실행 (기본 CMD가 이미 설정되어 있음)
CMD ["nginx", "-g", "daemon off;"]
```

3. nginx with static resource 를 빌드해보자

```
> docker buildx build --tag frontend:1.0  .
```

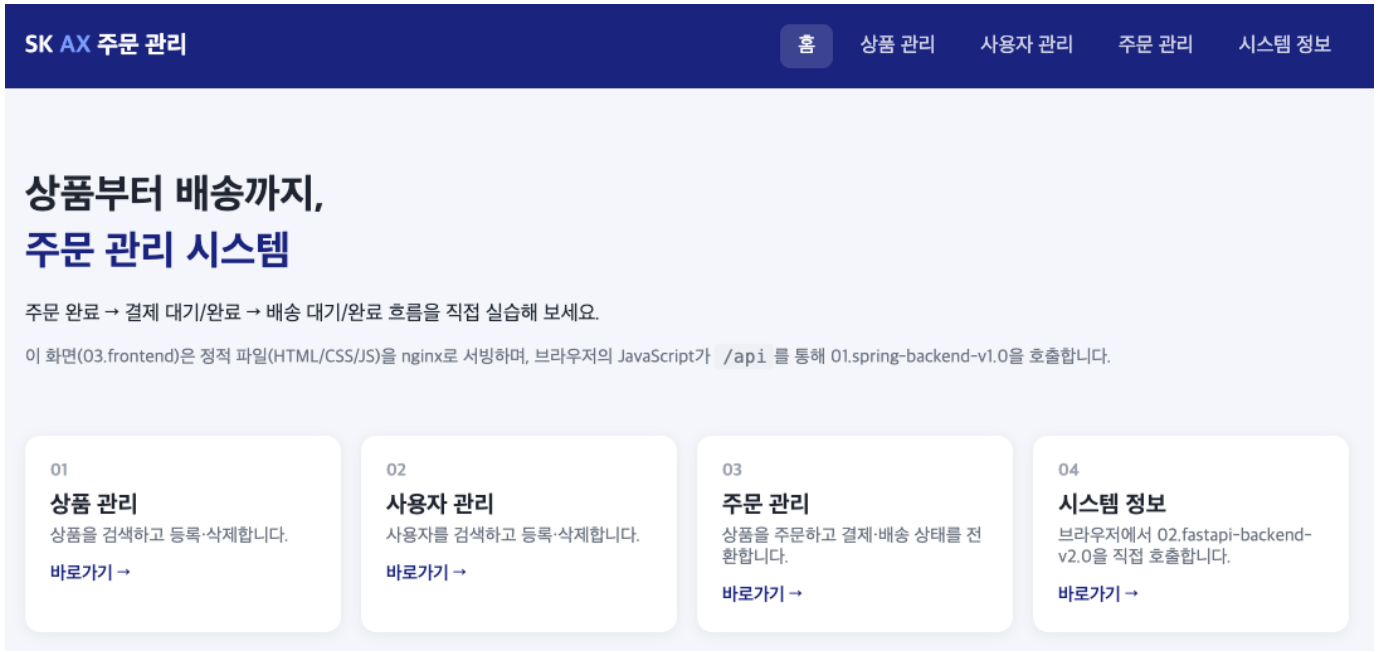
[실습] Frontend 만들기

4. skala docker bridge network 연결된 frontend container 실행하기

```
docker run -d \
  --name frontend \
  --network skala \
  -p 9090:80 \
  frontend:1.0
```

6. http://localhost:9090/ 로 browser로 접속해보기

사용자 관리 > 사용자 등록



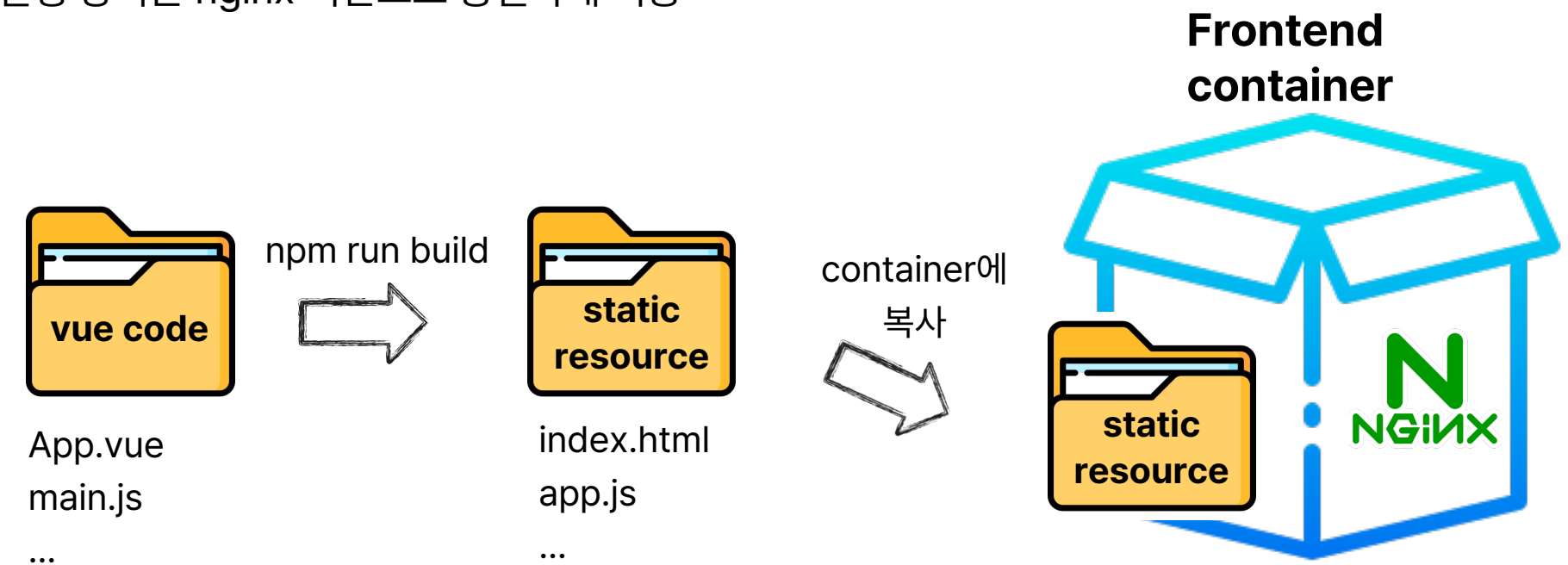
[실습] vue.js Frontend 만들기

vue.js 소스 코드 위치

```
00.sample-container/04.vue-frontend
```

default.conf는 fontend와 동일하게

- vue.js 구조는 실제 npm run build 과정을 통해 static resource 생성
- 생성된 파일은 앞의 frontend 와 동일하게 Container 에 복사
- 실행 방식은 nginx 기반으로 동일하게 적용



[실습] Frontend 만들기

1. Dockerfile

참고: 00.sample-container/04.vue-frontend/Dockerfile

```
# ── 빌드 스테이지 ──────────────────────────────
FROM node:20-alpine AS builder

WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build

# ── 배포 스테이지 ──────────────────────────────
FROM nginx:alpine

# 기본 nginx 설정 제거 후 커스텀 설정 복사
RUN rm /etc/nginx/conf.d/default.conf
COPY default.conf /etc/nginx/conf.d/

# 빌드 결과물 복사
COPY --from=builder /app/dist /usr/share/nginx/html

EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```

static resource 생성 과정

static resourc를 포함한 nginx container 생성 과정

[실습] Frontend 만들기

2. nginx with static resource 를 빌드해보자

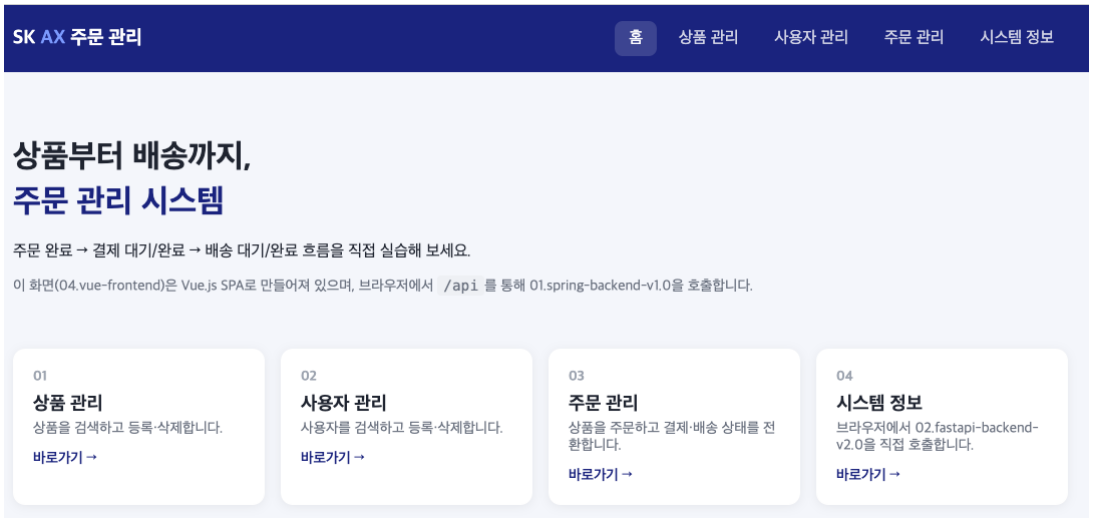
```
> docker buildx build --tag vue-frontend:1.0  .
```

3. skala docker bridge network 연결된 frontend container 실행하기

```
docker run -d \
  --name vue-frontend \
  --network skala \
  -p 8090:80 \
  vue-frontend:1.0
```

4. http://localhost:8090/ 로 browser로 접속해보기

사용자 관리 > 사용자 등록





KDT Cloud – 3. Container의 이해

컨테이너 이미지 구조

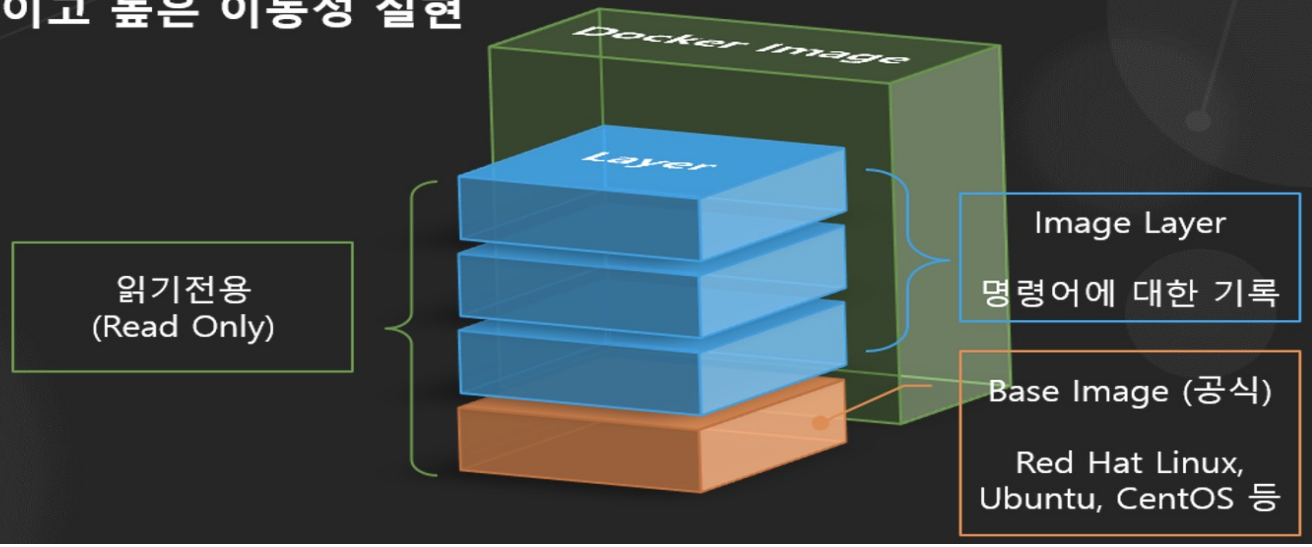
본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

컨테이너 이미지 구조

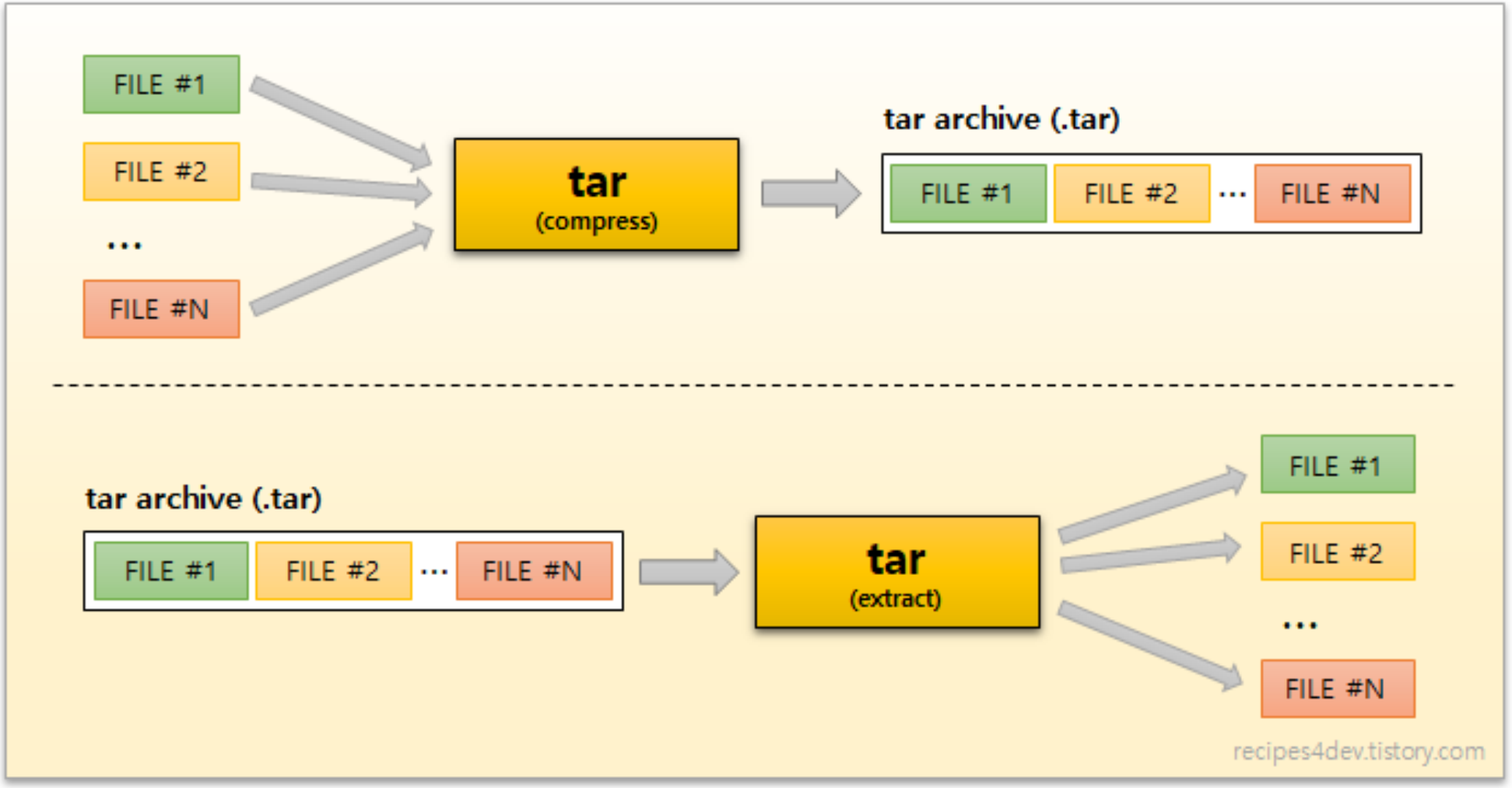
나의 프로세스를 실행하기 위한 OS 루트 파일시스템(rootfs)과 앱 실행 환경을 tar 레이어 단위로 직렬화하여 메타데이터와 함께 묶어놓은 배포용 패키지 아카이브

- 컨테이너를 실행할 때 필요한 파일시스템
  - 이미지 레이어의 집합체
    - 파일 내용과 메타 정보를 포함
  - 레이어는 부모와 자식 관계
  - 변경분만 기록
  - Read Only (읽기 전용) 으로 쓸 수 없음
- 공통 레이어를 이미지 간에 공유
  - 디스크 용량을 줄이고 높은 이동성 실현



tar (Tape ARchiver) 명령어

여러개의 파일과 디렉토리를 하나의 파일로 묶거나 풀때 사용하는 명령



tar로 보는 컨테이너 이미지

docker image는 실행을 위한 파일들을 계층적으로 묶어서 제공하는 방식으로 tar 묶음과 유사하다

```
# tar 로 파일을 패키징 해보자
> cd 00.sample-container
> tar cvf sample-container.tar *

# 묶여진 파일 목록을 확인해보자
> tar tvf sample-container.tar
```

tar는 무엇으로 구성되어 있나?

컨테이너의 이미지 내부 구조 확인하기

컨테이너 이미지를 만들고 이것을 하나씩 분해해보자

소스 코드 위치

```
01.answer-code/07.in-images
```

Dockerfile

붉은색: 이미지 레이어로 만들어지는 대상

```
ARG UBUNTU_VERSION=22.04
FROM ubuntu:${UBUNTU_VERSION}

RUN apt-get update && apt-get install -y curl lsb-release nginx
ARG UBUNTU_VERSION
RUN echo "현재 빌드에 사용된 ubuntu version: ${UBUNTU_VERSION}"

LABEL maintainer="himang10@gmail.com"
LABEL description="SKALA Linux Version"

EXPOSE 8080/tcp
EXPOSE 80/tcp

WORKDIR /var/www/html
COPY index.html .

CMD ["nginx", "-g", "daemon off;"]
```

컨테이너의 이미지 내부 구조 확인하기

이미지 빌드하기

```
> docker build --tag indepth-container:1.0  .
```

컨테이너 이미지를 로컬에 저장하기

```
> mkdir indepth-container
# Docker Image를 저장하기
> cd indepth-container
> docker save indepth-container:1.0 -o indepth-container.tar
```

이미지 내용 확인하기

```
> tar  tvf indepth-container.tar
> tar   xvf indepth-container.tar
> ls
# 타일 유형 확인
> file *
```

컨테이너의 이미지 내부 구조 확인하기

이미지 내에 여러 Layer가 존재하는 것을 확인해본다

컨테이너 이미지 계층 확인하기

```
> cat  manifest.json  |  jq
[
  {
    "Config": "blobs/sha256/19e1da3667ffa7305b54fb0ec640b156d87e4b948e1614f2e6a5eee8b89fe3ed",
    "RepoTags": [
      "container-linux:1.0"
    ],
    "Layers": [
      "blobs/sha256/119d19e001bafa21919289095e1dbfac64f1e16d2469dd14c2d2a520039d26d9",
      "blobs/sha256/0655737b94e69d8feaadb404d3c32bf6054788b8a7bd799836335a13252e7c1f",
      "blobs/sha256/4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1",
      "blobs/sha256/4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1",
      "blobs/sha256/a1b97ad067fc4da50b167f14c5b3f46da3db6906061e5c187fd30ea57bfb9a40"
    ]
  }
]
```

[참고] blobs/sha256 내부 확인해보기

이미지 내에 여러 Layer가 존재하는 것을 확인해본다

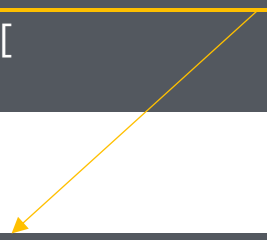
컨테이너 이미지 계층 확인하기

```
> cd indepth-container/blobs/sha256
> file *
1f2b5b930d8764b95d517bb04f3a3156d061e938ac80350cceb635d0b6e11d06: JSON data
231032373bb3b10375f5940876c5c9a4a630f4e007e3aee895f8e1687b00295e: gzip compressed data, was "bf05b9274099653dfa3f182aeace
25591e8de4215f625d8f58da1fba0e34036a.tar", last modified: Mon Aug 10 17:38:40 2026, max compression, original size modulo
4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1: gzip compressed data, truncated
57068f897ddff6d4902e5db38a7aa09c905fc671a82b4075034d70e5800918da: JSON data
60235ca8ca5d18939634354ed4a464376a0959c73d0f260fbd6554a3dd10a034: gzip compressed data, original size modulo 2^32 1679068
67d0c8eab99d2170a01a86a402b5c2bd7dd425c6630c70b98582341817265d77: JSON data
84fd2ce5dec31163b42d5480779f672f556c0dce990fbf42d5801b401d847b14: JSON data
a757628d7a057834ab3f622d8d4902c456785ec677148870b209fe20fff8d395: JSON data
aae59993f0495783a69d86fc9e11f8f365a4d136c5a824441d2a295acc985d9a: gzip compressed data, original size modulo 2^32 3584
c56b5750cb2b38d26fa199072dece168efb35faac501544bb4e1beab67177b64: JSON data
> cat c56b5750cb2b38d26fa199072dece168efb35faac501544bb4e1beab67177b64
```

컨테이너의 이미지 내부 구조 확인하기

이미지 메타 데이터

```
[
  {
    "Config": "blobs/sha256/5b666a89cbc7dd0abf11902346cc178118d055d8b8000fb29ecaaea6592d0491",
    "RepoTags": [
```



```
> cat  blobs/sha256/5b…  |  jq
```

컨테이너의 이미지 내부 구조 확인하기

아래의 각 Layer 를 하나씩 풀어보면서 Dockerfile에 어떤 명령어가 실행되었느지 확인해보자

```
"Layers": [
     "blobs/sha256/119d19e001bafa21919289095e1dbfac64f1e16d2469dd14c2d2a520039d26d9",
     "blobs/sha256/0655737b94e69d8feaadb404d3c32bf6054788b8a7bd799836335a13252e7c1f",
     "blobs/sha256/4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1",
     "blobs/sha256/4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1",
     "blobs/sha256/a1b97ad067fc4da50b167f14c5b3f46da3db6906061e5c187fd30ea57bfb9a40"
   ]
```

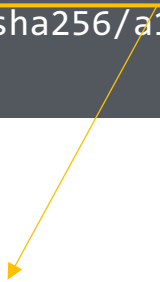
```
> tar tvf  blobs/sha256/5528fe4…
```

COPY index.html .

컨테이너의 이미지 내부 구조 확인하기

아래의 각 Layer 를 하나씩 풀어보면서 Dockerfile에 어떤 명령어가 실행되었느지 확인해보자

```
"Layers": [
     "blobs/sha256/119d19e001bafa21919289095e1dbfac64f1e16d2469dd14c2d2a520039d26d9",
     "blobs/sha256/0655737b94e69d8feaadb404d3c32bf6054788b8a7bd799836335a13252e7c1f",
     "blobs/sha256/4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1",
     "blobs/sha256/4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1",
     "blobs/sha256/a1b97ad067fc4da50b167f14c5b3f46da3db6906061e5c187fd30ea57bfb9a40"
   ]
```



```
WORKDIR /var/www/html
```

컨테이너의 이미지 내부 구조 확인하기

아래의 각 Layer 를 하나씩 풀어보면서 Dockerfile에 어떤 명령어가 실행되었느지 확인해보자

```
"Layers": [
     "blobs/sha256/119d19e001bafa21919289095e1dbfac64f1e16d2469dd14c2d2a520039d26d9",
     "blobs/sha256/0655737b94e69d8feaadb404d3c32bf6054788b8a7bd799836335a13252e7c1f",
     "blobs/sha256/4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1",
     "blobs/sha256/4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1",
     "blobs/sha256/a1b97ad067fc4da50b167f14c5b3f46da3db6906061e5c187fd30ea57bfb9a40"
   ]
```



```
RUN echo "현재 빌드에 사용된 ubuntu version: ${UBUNTU_VERSION}"
```

컨테이너의 이미지 내부 구조 확인하기

아래의 각 Layer 를 하나씩 풀어보면서 Dockerfile에 어떤 명령어가 실행되었느지 확인해보자

```
"Layers": [
     "blobs/sha256/119d19e001bafa21919289095e1dbfac64f1e16d2469dd14c2d2a520039d26d9",
     "blobs/sha256/0655737b94e69d8feaadb404d3c32bf6054788b8a7bd799836335a13252e7c1f",
     "blobs/sha256/4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1",
     "blobs/sha256/4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1",
     "blobs/sha256/a1b97ad067fc4da50b167f14c5b3f46da3db6906061e5c187fd30ea57bfb9a40"
   ]
```

```
RUN apt-get update && apt-get install -y curl lsb-release nginx
```

컨테이너의 이미지 내부 구조 확인하기

아래의 각 Layer 를 하나씩 풀어보면서 Dockerfile에 어떤 명령어가 실행되었느지 확인해보자

```
"Layers": [
     "blobs/sha256/119d19e001bafa21919289095e1dbfac64f1e16d2469dd14c2d2a520039d26d9",
     "blobs/sha256/0655737b94e69d8feaadb404d3c32bf6054788b8a7bd799836335a13252e7c1f",
     "blobs/sha256/4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1",
     "blobs/sha256/4f4fb700ef54461cfa02571ae0db9a0dc1e0cdb5577484a6d75e68dc38e8acc1",
     "blobs/sha256/a1b97ad067fc4da50b167f14c5b3f46da3db6906061e5c187fd30ea57bfb9a40"
   ]
```

```
FROM ubuntu:${UBUNTU_VERSION}
```

mariadb image layer 확인해보기

Mariadb Container Dockerfile

https://hub.docker.com/_/mariadb

https://github.com/MariaDB/mariadb-docker/blob/master/10.11/Dockerfile

mariadb-docker / 10.11 / Dockerfile



grooverdan 10.11.13 and 11.4.7 re-release

Code Blame 142 lines (128 loc) · 6.39 KB

```
1   # vim:set ft=dockerfile:
2   FROM ubuntu:jammy
3
4   # add our user and group first to make sure their IDs get assigned consistently, regardless of whatever dependencies get added
5   RUN groupadd -r mysql && useradd -r -g mysql mysql --home-dir /var/lib/mysql
6
7   # add gosu for easy step-down from root
8   # https://github.com/tianon/gosu/releases
9   # gosu key is B42F6819007F00F88E364FD4036A9C25BF357DD4
10  ENV GOSU_VERSION 1.17
11
12  ARG GPG_KEYS=177F4010FE56CA3336300305F1656F24C74CD1D8
```

[실습] mariadb 풀어보자

이미지 빌드하기

```
> docker pull mariadb:10.11
```

컨테이너 이미지를 로컬에 저장하기

```
> mkdir indepth-mariadb
# Docker Image를 저장하기
> cd indepth-container
> docker save mariadb:10.11 -o mariadb.tar
```

이미지 내용 확인하기

```
> tar   xvf mariadb.tar
> ls
# 타일 유형 확인
> file *
```

[실습] mariadb image layer 확인해보기

mariadb image layer 구성 확인하기

```
➢ cat manifest.json | jq
[
 {
  "Config": "blobs/sha256/b5898e2f865470ead4d653b985c74c640ea00be8e384d7c84d5eade2c58ff705",
  "RepoTags": [
   "mariadb:latest"
  ],
  "Layers": [
   "blobs/sha256/69c262fc30fc134b6d373dee8db695319c41d8b9489deb0f682565473bf29748",
   "blobs/sha256/4a585ea2a801a7e852a88607392a2682abe6163f7c62138bc8d2d7387ea51501",
   "blobs/sha256/986b7028e52e39fadc4122e969e818c2b9a57ad0d041026ea995dd6310da15be",
   "blobs/sha256/9bf0665a0c3d191cb19d9f0027302e47db304a8c37fc255b0e842ff49df3fb83",
   "blobs/sha256/94b6ebcad19f3488fa4c4b89d30aa75faa444d86a85f1821f148fe75c8ea949e",
   "blobs/sha256/570cbed76d34d44a0a098a84f2505d1fc8b985225a71dc15599eb394c6d1b859",
   "blobs/sha256/de133d0ba73769c923f524a0d704fff2f8a88b3cd32981200611eefc83b078a7",
   "blobs/sha256/6eb57503768609400 38ecdc9dc28a18962385a25331e2986326f0d2462a0a50c"
  ]
 }
]
...
```

Dockerfile 내용

FROM ubuntu:jammy

RUN groupadd ...

RUN set –eux;l \
apt-get ..

RUN mkdir
/docker-entrypoint-initdb.d

RUN set –e;\
echo “dev

[실습] mariadb image layer 확인해보기

mariadb image layer 구성 확인하기

```
➢ cd ./blobs/sha256

➢ tar tvf 6eb57503768609400 38ecdc9dc28a18962385a25331e2986326f0d2462a0a50c
drwxr-xr-x  0 0      0           0  5 29 11:22 usr/
drwxr-xr-x  0 0      0           0  5 29 11:22 usr/local/
drwxr-xr-x  0 0      0           0  6 11 07:36 usr/local/bin/
-rwxr-xr-x  0 0      0       26472  6 11 07:35 usr/local/bin/docker-entrypoint.sh
```



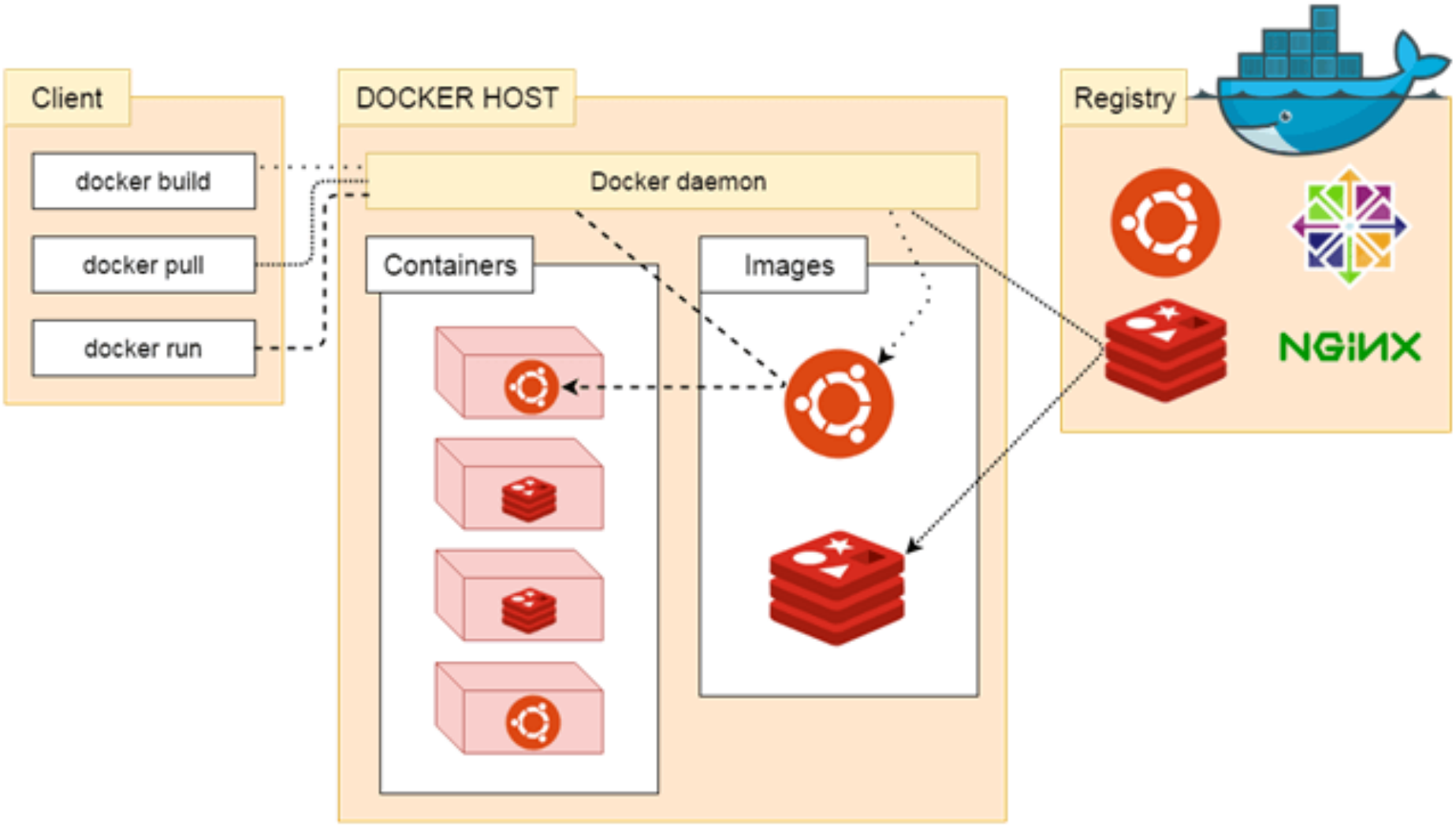
컨테이너 구조 이해

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

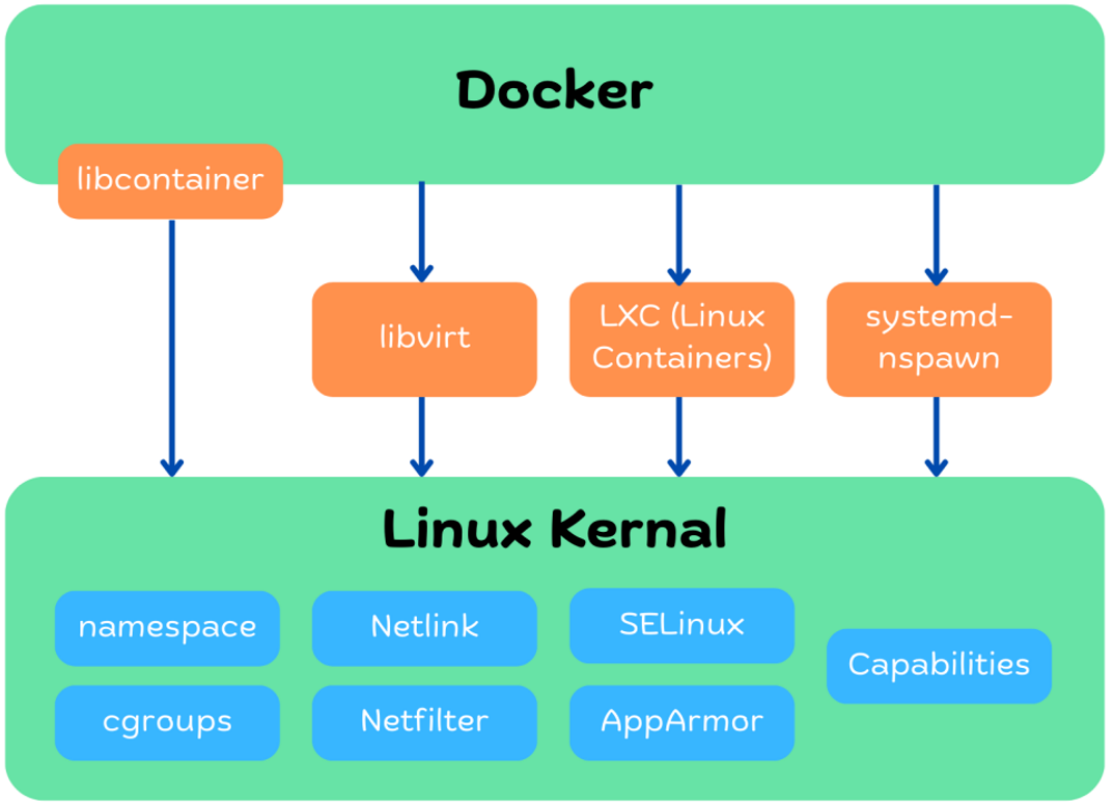
Docker Architecture

컨테이너 생성 흐름



Docker에서 사용되는 기술

Docker 데몬 → containerd → containerd-shim → **runc** (바이너리) —내장→ **libcontainer** (라이브러리) → Linux Kernel



libcontainer

리눅스 커널의 네임스페이스 생성, Cgroups 자원 할당, pivot_root, Capabilities 드롭 등 커널 시스템 콜을 직접 호출하는 순수 Go 언어 패키지(내부 엔진)

Apache License 2.0 하에 공개
리눅스 재단 산하의 OCI 프로젝트

runc

- OCI(Open Container Initiative) 런타임 규격에 맞게 만들어진 CLI 커맨드라인 도구
- OCI 스펙의 config.json을 읽어 컨테이너 생명주기 관리

Container Feature

컨테이너는 가상 머신이 아니라 Linux Kernel 기능들의 조합으로 만들어진 격리 실행 환경

Namespace
- 프로세스 마다 서로 다른 시스템 뷰 제공
- Container 내부 뷰/테이, Host 뷰/테이블
- PID, NET, MNT, IPC, USER, UTS/HOSTNAME

cgroups
- 자원 사용을 통제 (하나의 컨테이너가 폭주해도 노드 전체에 영향을 최소화)
- CPU, Memory, Disk I/O throttle, 프로세스 개수,

rootfs + OverlayFS
- 이미지 레이어를 모든 컨테이너와 재사용
- 컨테이너마다 독립 파일 시스템으로 격리

Netfliter
- 네트워크 패킷을 가로채고, 검사하고, 변경 지원
- Port-forwarding, NAT, 컨테이너간 통신 격리

Linux Capabilities
프로세스가 어떤 시스템 작업(기능)을 수행할 수 있는가?
- root 권한을 세분화
- CAP_NET_ADMIN (네트워크 설정 권한),
  CAP_SYS_ADMIN (mount, 커널 제어 권한),
  CAP_KILL (프로세스 kill 권하)

SELinux/AppArmor :
프로세스가 어떤 대상(파일, 포트, 프로세스)에 접근할 수 있는가
- 컨테이너가 호스트의 불필요한 시스템 자원에 접근하는 것을 차단

권한 체계

전통적 리눅스 모델에서의 권한 체계: UID 0 (root)와 일반 사용자 (UID > 0) 로 구분

문제점

과도한 권한 부여

- 웹 서버가 80번 포트에 바인딩(Privileged Port)하려면 root 권한으로 실행
- root 권한만 있으면 시스템 전체의 실행 가능

소유자 기반 통제 (DAC)의 무력화

- 파일 소유자나 root는 파일 권한을 임의로 변경 가능, 악성 코드 침투 시 시스템 보호 불가

솔루션

Linux Capabilities : 주체 행위 특권의 세분화

root 특권을 약 40개로 나눔 (최소 권한 원칙 적용)

- 특정 프로세스가 탈취되더라고 root 전체 권한으로 승격되는 것을 방지
  - ✓ CAP_NET_BIND_SERVICE: 1024 미만 포트 바인딩만 허용
  - ✓ CAP_SYS_ADMIN: 시스템 관리성 작업 허용 (컨테이너 등에서 격리용으로 사용)
  - ✓ CAP_NET_ADMIN: 네트워크 인터페이스 설정 및 라우팅 제어 허용

SELinux: 객체 접근 및 행위에 대해 주체에게 강제 통제

프로세스가 파일, 디렉토리, 포트 등에 대해 수행 할수 있는 작업을 강제로 정의

예) httpd 는 /var/www/html 파일만 읽기 지정 가능

- 프로세스가 root(UID 0)이고 모든 Capability를 가지고 있더라도, SELinux 정책에 명시적으로 allow가 정의되어 있지 않으면 커널 단에서 접근을 차단

[예시] Capability & SELinux 적용

전통적 리눅스 모델에서의 권한 체계: UID 0 (root)와 일반 사용자 (UID > 0) 로 구분

쿠버네티스 Capability 예시

```
apiVersion: v1
kind: Pod
metadata:
  name: capability-demo
spec:
  containers:
    - name: nginx
      image: nginx:latest

      securityContext:
        runAsUser: 1000
        runAsNonRoot: true

        capabilities:
          drop:
            - ALL
          add:
            - NET_BIND_SERVICE
```

쿠버네티스 SELinux 예시

```
apiVersion: v1
kind: Pod
metadata:
  name: selinux-demo
spec:

  securityContext:
    seLinuxOptions:
      user: system_u
      role: system_r
      type: container_t
      level: "s0:c123,c456"

  containers:
    - name: nginx
      image: nginx:latest
```

1024 이하 포트 사용 권한 할당
예) nginx 80 포트 할당 가능

컨테이너 내 프로세스 격리 : namespace

PID, Network, Mount 등의 커널 데이터 구조를 네임스페이스로 분리해서 컨테이너 내 실행되는 프로세스가 지정된 네임스페이스의 커널 데이터를 참조하도록 구성하는 방식

핵심

- **커널 레벨**에서 **보이는 세상(view)을 분리**
- 컨테이너 마다 서로 다른 리소스 공간을 가진것 처럼 보이게 제공
- 하나의 커널에서 여러 논리적 OS 환경을 제공 → **실행되는 프로세스 입장에서는 논리적 OS만 볼 수 있음**

namespace	기능
PID	컨테이너마다 별도의 프로세스 테이블을 관리 (프로세스 트리 분리: 컨테이너 안에서 PID 1부터 시작) 특정 컨테이너의 프로세스에서 다른 컨테이너의 프로세스로 접근할 수 없음
Network	Virtual Ethernet (veth0), IP Address, 포트 번호, 라우팅 테이블, 필터링 테이블 등 네트워크 리소스를 namespace별로 할당 가능
UID/GID	독립적인 사용자 할당 User ID(UID)와 Group ID(GID) 공간을 격리 namespace 외부에 있는 user나 group이 내부의 공간을 보거나 건드릴 수 없음
Mount	마운트를 조작하여 namespace 내에 파일 시스템 트리를 구성하여, 컨테이너마다 특정 디렉토리를 루트 파일 시스템으로 보이게 함 (rootfs)
UTS	namespace별로 hostname과 domain-name을 분리 (file 분리: /etc/hosts, /etc/resolve.conf, /etc/hostname)
IPC	공유 메모리, 큐, 세마포어 등 프로세스 간의 통신(IPC, Inter-Process Communication) 자원을 분리 할 수 있음

프로세스에 할당된 리소스 Control Group: cgroup

동일 namespace에 할당된 프로세스 집합에 대해 CPU, Memory, I/O 등의 사용량을 제한, 격리, 모니터링하는 기능

- 컨테이너에 할당하는 자원을 조정하는데 사용
- 프로세스 및 Thread를 그룹화하여 관리하는 기능
- Host OS의 CPU, 메모리와 같은 리소스를 그룹별로 제한 가능
- 다음과 같은 주요 서브 시스템 관리 가능
- 계층구조로 프로세스를 그룹화하여 관리가능
- 하위 cgroup은 상위 cgroup의 제한 설정이 그대로 적용

자원	예
CPU	quota, shares
Memory	limit, oom
Block I/O	read/write throttle
PIDs	프로세스 수 제한

항목	기능
cpu	CPU 사용량 제한
cpuacct	CPU 사용량 통계 정보 제공
cpuset	CPU와 메모리 배치 제어
memory	메모리와 swap 사용량 제한
devices	디바이스 엑세스 허가 및 제한
freezer	그룹에 속한 프로세스 정지 및 재개
net_cls	네트워크 제어 태그 추가
blkio	블록 디바이스 입출력 량 제어

```
docker run -d \
  --name my-resource-container \
  --cpus="2.0" \
  --memory="512m" \
  nginx
```

[참고] container 내 process가 자원 초과시 처리 과정

컨테이너에서 CPU, Memory, Disk I/O 등을 초과하는 경우 커널이 자원 초과 상황을 감지후 제어 실행

메모리 설정 값 컨테이너는 cgroup으로 memory limit: 512M

상태: init process가 700MiB 사용 → limit 초과

실행 절차

1. Linux 커널의 **OOM Killer**가 동작 → 프로세스 강제 종료
2. shim이 PID 1 종료 감지 → container 종료 상태를 containerd에 알림
3. containerd가 container 상태 갱신 (Docker는 Exited/137, k8s는 OOMKilled)

자원 종류	컨트롤러	제어 방식 예시
CPU	cpu / cpuacct	cfs quota, period 로 제한
Memory	memory	limit 초과 시 OOM kill
I/O	blkio	읽기/쓰기 속도 제한
PID 수	pids	프로세스 개수 제한
Network	(없음)	cgroup 자체로는 직접 제한 불가 (tc 등 필요)

이미지 관리 기술 : OverlayFS 기반 계층 파일 시스템

컨테이너 이미지를 가볍게 유지하고 빠른 속도로 컨테이너를 생성할 수 있는 핵심 기술

여러개의 폴더를 하나의 디렉토리로 합쳐서 (Overlay) 보여주는 기술

리눅스 커널 3.18 이후 정식 추가

구성 요소

LowerDir (하단 디렉토리 / Read-Only)

- 읽기 전용 레이어 (풀려진 컨테이너 이미지)
- 컨테이너 이미지 원본 레이어들 (수정/삭제 불가)

UpperDir (상단 디렉토리 / Read-Write)

- 쓰기 가능 레이어
- 컨테이너 실행 후 사용자 생성, 수정, 삭제 파일의 기록들이 모두 이 디렉토리에 물리적으로 저장
- 원래 lowerdir에 있던 파일을 수정할 경우 upperdir로 복사된 후 변경

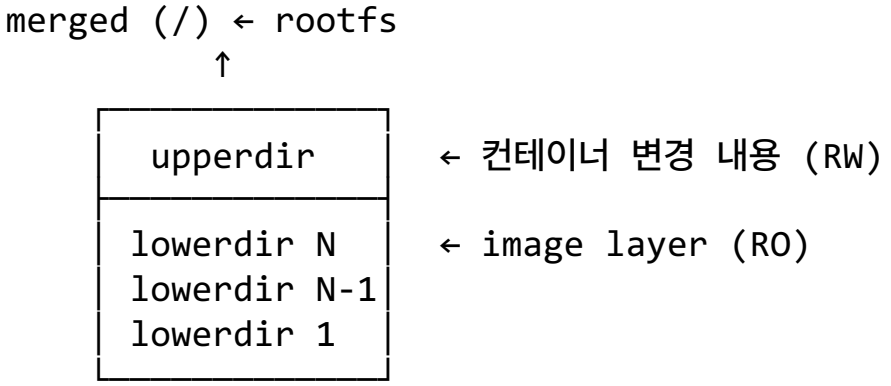
python 3.10 (LowerDir) → python 3.11 (UpperDri)
/usr/bin/python (3.10) → /usr/bin/python (3.11)

MergedDir (=rootfs)

- LowerDir과 UpperDir을 OverlayFS로 병합하여 구성한 실제 호스트 상의 디렉토리
- 해당 컨테이너의 독립된 루트 파일 시스템(/)을 의미

workdir (작업 디렉토리)

- 디렉터리 생성, 삭제, rename 작업 등에 필요한 임시 작업 공간



이미지 관리 기술 : OverlayFS 기반 계층 파일 시스템

주요 특성

레이어링 (Layering)

- 여러 디렉터리를 "레이어(layer)"로 취급
- **읽기 전용(lowerdir)** 과 **읽기-쓰기 가능(upperdir)** 디렉터리를 하나의 통합된 파일 시스템처럼 사용
- 사용자가 파일을 수정하면 변경 사항은 upperdir에 기록되며, lowerdir의 원본은 변경되지 않음

Copy-on-Write (CoW) 전략

파일 수정 시 원본(lowerDir)을 그대로 두고
upperdir에 수정 파일을 복사하여 변경 관리
→ 원본 불변성(Immutable) 보장

디렉토리 병합 및 우선순위

- 여러 디렉토리 (lowerdir, upperdir, workdir)를 container rootfs로 병합하여 마운트

```
mount -t overlay overlay \
  -o lowerdir=/image/layers,\
     upperdir=/container/writable,\
     workdir=/container/work \
  /container/rootfs
```

- 동일한 파일이 존재할 경우, upperdir 파일이 우선

파일 시스템 투명성

- 사용자에게는 하나의 디렉터리처럼 보임
- 실제로는 여러 레이어를 OverlayFS가 조합하여 동작

[참고] 디렉토리 구성 흐름

runC는 pivot_root()를 이용해서 이 병합된 디렉토리를 **컨테이너의 루트 파일 시스템(/)으로 전환**

```
/var/lib/containerd/io.containerd.snapshotter.v1.overlayfs/snapshots/<id>/fs/
├── lowerdir/        ← 읽기 전용 레이어들 (이미지)
├── upperdir/        ← 컨테이너 실행 중 변경사항 저장
├── workdir/         ← OverlayFS 내부 작업 디렉토리
└── merged/          ← OverlayFS 병합 mount 결과 → 이것이 /run/containerd/.../rootfs
```

- merged = lowerdir + upperdir + workdir
- runC는 이 merged 디렉토리를 /로 설정

[실습] Overlayfs 기반 rootfs 만들어보기

ubuntu 컨테이너 내부에서 overlayfs 동작 원리를 이해해보자

1. ubuntu container 실행하기

```
> docker run --rm -it \
  --privileged \
  ubuntu:24.04 \
  /bin/bash
```

2. 컨테이너 내부에서 tmpfs를 마운트하기

```
> mkdir -p /mnt/ovtest
> mount -t tmpfs tmpfs /mnt/ovtest          # p1: 파일시스템 종류, p2: 장치식별자
> df -T /mnt/ovtest
Filesystem     Type  1K-blocks  Used Available Use% Mounted on
tmpfs          tmpfs   4012648    12   4012636   1% /mnt/ovtest
```

[실습] Overlayfs 기반 rootfs 만들어보기

3. OverlayFS 디렉토리 만들기

```
mkdir -p /mnt/ovtest/{lower,upper,work,merged}
```

4. 파일 생성하기

```
echo "AAA from lower" > /mnt/ovtest/lower/a.txt
echo "BBB from lower" > /mnt/ovtest/lower/b.txt
```

5.여러 파일을 중첩해서 마운트하기

```
mount -t overlay overlay \
  -o lowerdir=/mnt/ovtest/lower,upperdir=/mnt/ovtest/upper,workdir=/mnt/ovtest/work \
  /mnt/ovtest/merged
```

[실습] Overlayfs 기반 rootfs 만들어보기

6./mnt/ovtest/merged mount 확인하기

```
> mountpoint /mnt/ovtest/merged
/mnt/ovtest/merged is a mountpoint
> ls /mnt/ovtest/lower
a.txt  b.txt
> ls /mnt/ovtest/merged              # lower, upper merged 된 상태
a.txt  b.txt
```

7. /mn/ovtest/merged 의 a.txt에 값을 변경해보자

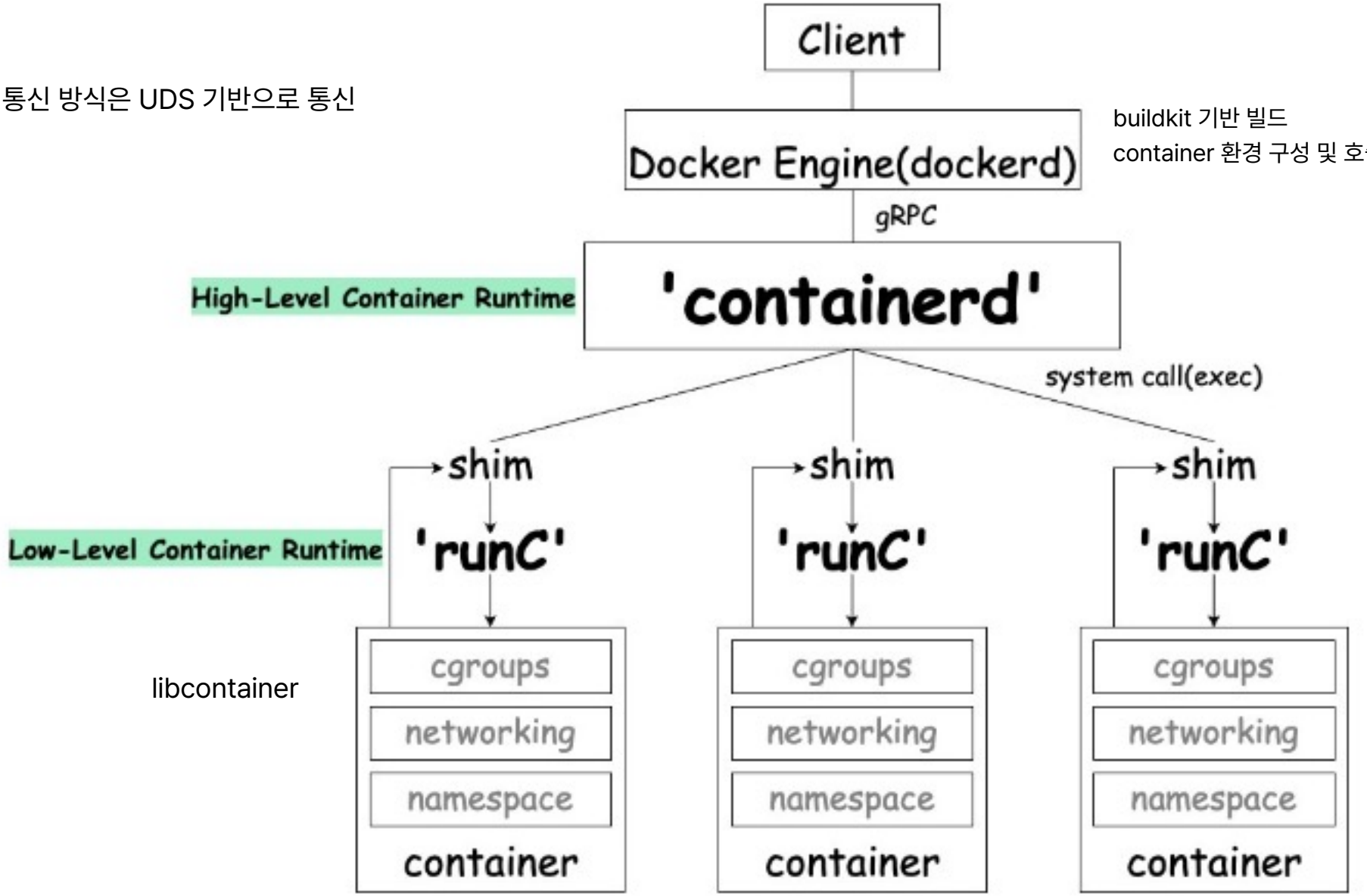
```
> echo "add line test" >> /mnt/ovtest/merged/a.txt
```

8. /mnt/ovtest/merged/a.txt를 수정 후

- /mnt/ovtest/upper/a.txt가 생성되었는지 확인하기
- /mnt/ovtest/lower/a.txt의 내용 확인하기 (cat /mnt/ovtest/lower/a.txt)
- /mnt/ovtest/upper/a.txt 의 내용을 /lower와 비교해보자 (cat /mnt/ovtest/upper/a.txt)

컨테이너 실행 구조

모든 통신 방식은 UDS 기반으로 통신



buildkit 기반 빌드
container 환경 구성 및 호출

libcontainer

docker v1.11 이후부터 적용된 docker architecture

Docker Engine, containerd, runc

shim: 끼움쇠, 쐐기

dockerd (docker engine)

- Docker의 진입점 이면 Docker 전체 운영을 관리
- Docker 엔진에 이미지 실행을 요청하면 containerd 데몬에 책임을 위임

↓ docker run 파라메터 →

```
docker run -d \
  --name myapp \
  --memory 512m \
  --cpus 1 \
  -p 8080:80 \
  -e PROFILE=prod \
  nginx:latest
```

containerd (=cri-o) - kubernetes kubelet과 연동

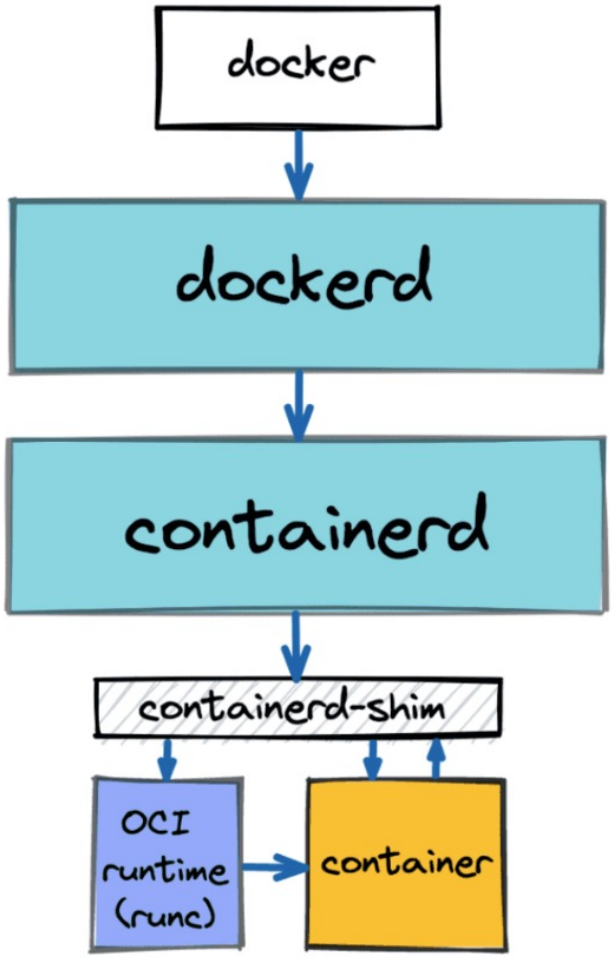
OCI 표준 번들 (**config.json + rootfs**) 직접 생성

- OverlayFS 기반으로 rootfs (MergedDir) 준비
- dockerd로 부터 받은 컨테이너 실행 설정을 config.json으로 구성
- containerd-shim fork 및 exec 후 OCI 번들 전달

↓ config.json, rootfs 경로

containerd-shim (=podman/crio는 conmon 경량 프로세스)

- runC 프로그램 실행 (Args: config.json, rootfs 경로)
  1. namespace 생성, cgroups 설정
  2. pivot_root 로 / 교체
  3. execve() init process 실행 및 init process PID shim에 전달
- 해당 PID를 감시 및 I/O 파이프라인 연결/대기 (docker exec | logs)



OCI (Open Container Initiative) ?

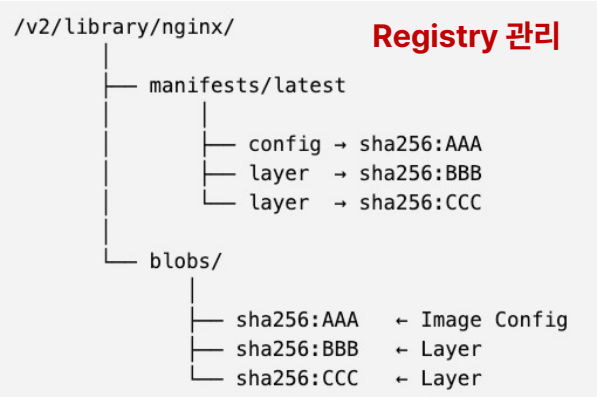
컨테이너 이미지와 컨테이너 실행 방법을 표준화한 오픈 표준 프로젝트

1. **OCI Image Spec:** 어떤 도구든 이미지를 다운받아 해석할 수 있는 패키지 표준 규격
   - 컨테이너 이미지 구조, 압축 파일(tar) 레이어
   - manifest.json 표준

2. **OCI Runtime Spec:** 컨테이너 실행을 위한 **OCI Bundle**을 정의하는 표준 규격

```
OCI Bundle
├── config.json        ← 어떻게 실행할 것인가
└── rootfs/            ← 무엇을 실행할 것인가
    ├── bin/
    ├── usr/
    ├── lib/
    └── ...
```

3. **OCI Distribution Spec:** 컨테이너 이미지를 Registry에 Push/Pull 하기 위한 API 규격



manifest GET `GET /v2/library/nginx/manifests/latest`

layer GET `GET /v2/library/nginx/blobs/sha256:AAA...`

[참고] OCI를 통해 이종(Heterogeneous) 도구간 호환성 보장

OCI 호환 이미지를 생성한다는 의미는?

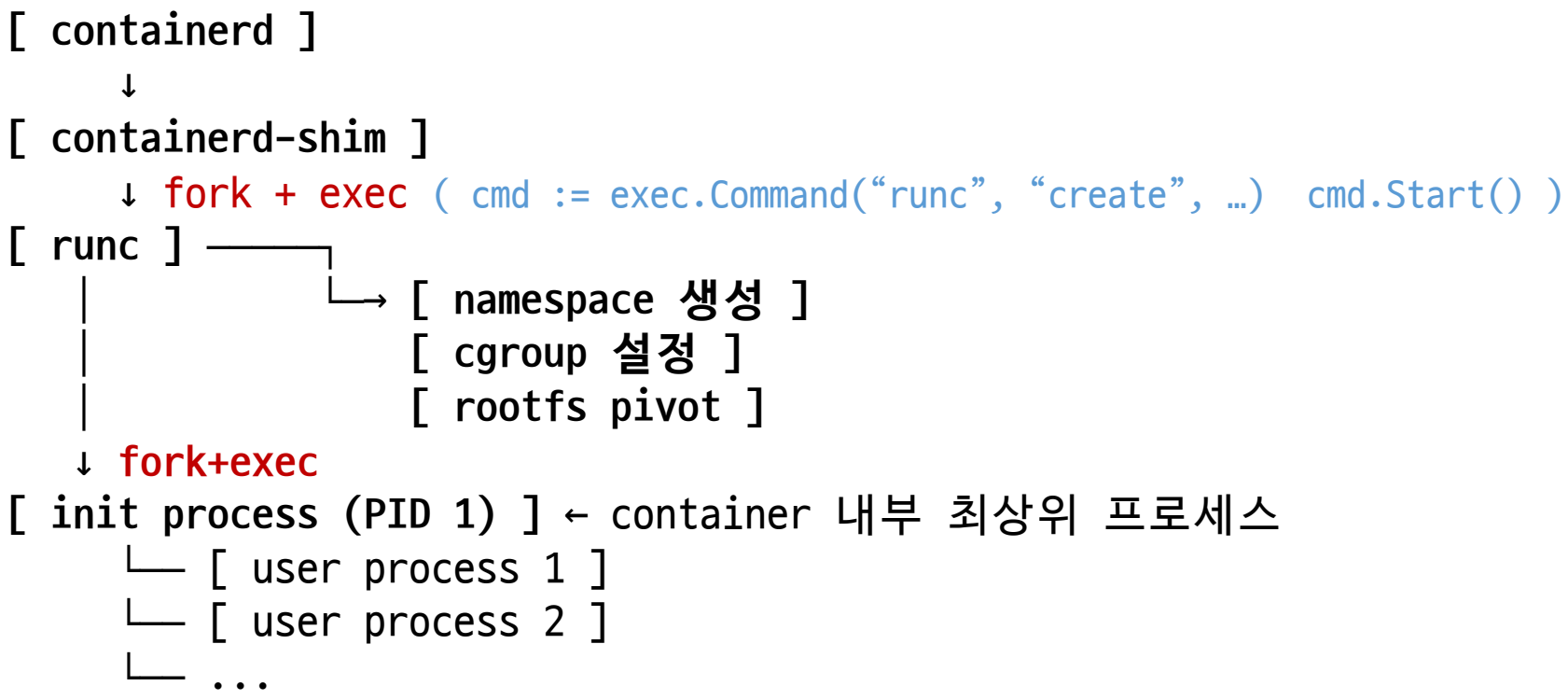
Docker가 만든 이미지가 **OCI에서 정한 표준 형식에 맞는 컨테이너 이미지라는 의미**

- 이미지 메타데이터 구조
- 레이어 구조
- Image Spec (mainfest.json)
- Runtime Spec (config.json)

공통 형식(OCI 표준)을 사용하기 때문에 **특정 벤더에 종속되지 않는** 에코시스템 빌드가 가능

- **로컬 검증 (CLI)**: 개발자 로컬 PC에서 Docker나 Podman으로 즉시 실행 및 테스트 가능
- **빌드 (Build)**: buildkit, Buildah, Kaniko로 이미지를 빌드하고,
- **저장 (Registry)**: Harbor나 Docker Hub 같은 OCI 레지스트리에 저장(Push/Pull)하며,
- **배포 및 실행 (Runtime)**: docker, 쿠버네티스의 containerd나 CRI-O에서 이 이미지 pull /run 하고,

[참고] 컨테이너 프로세스 (CMD init Process) 실행 구조

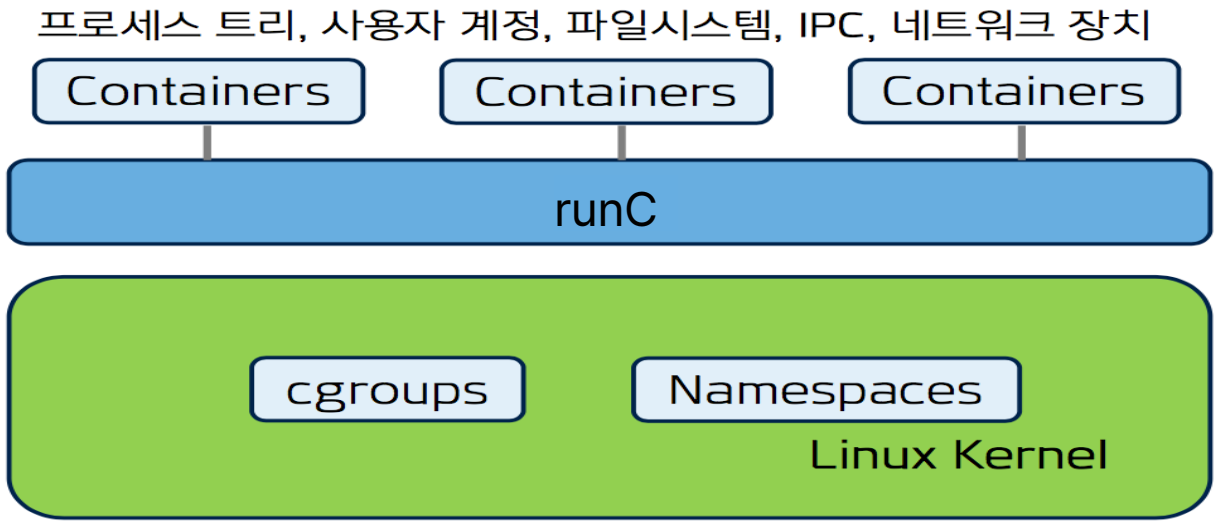


RunC

LXC (LinuX Contianer**) → libcontainer**

→ OCI 표준의 Reference 구현체 RunC (libcontainer wrapping)

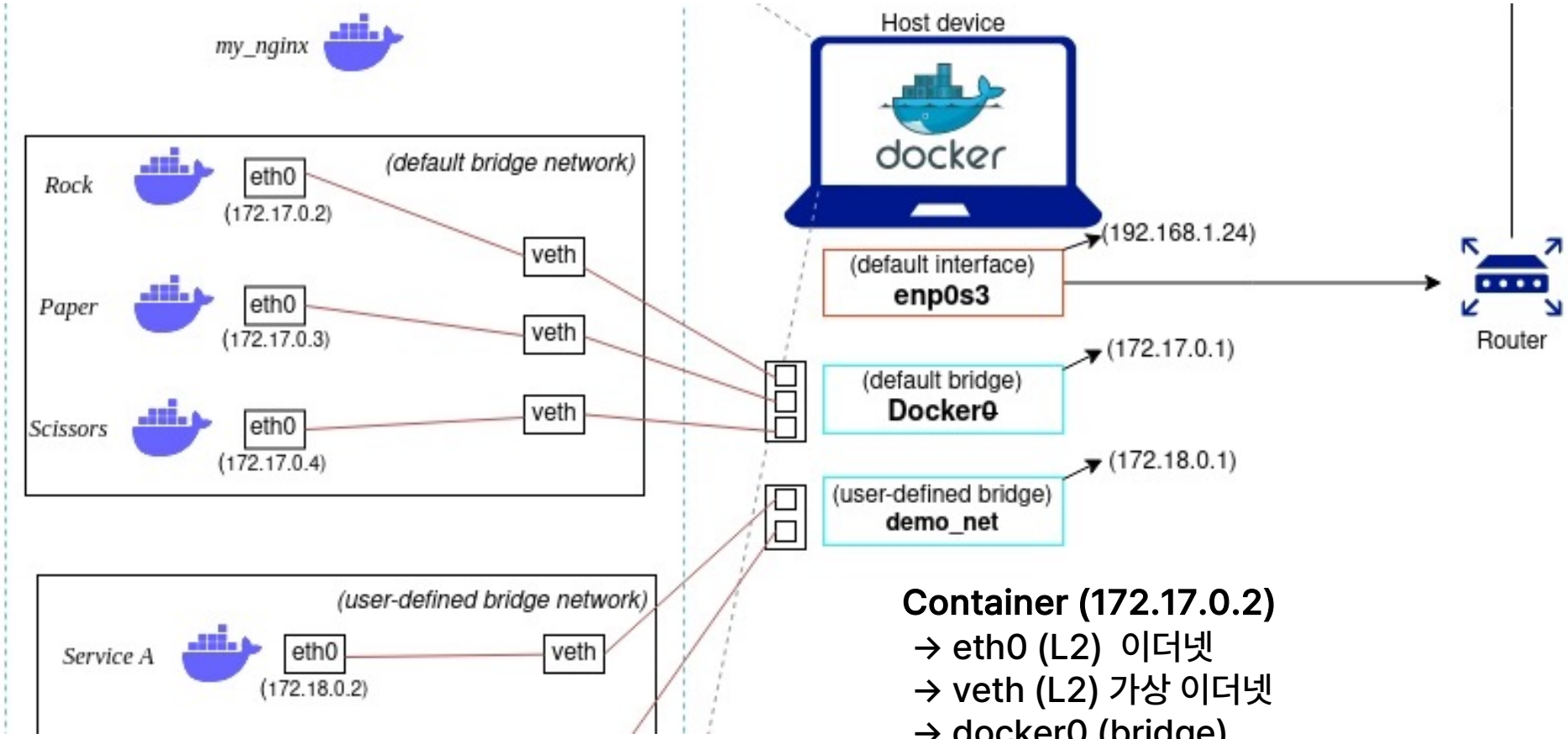
- namespace 생성 & cgroup 설정
- rootfs 마운트 및 교체
- UID/GID 매핑
- Capability, seccomp, AppArmor 적용
- init 프로세스 (PID 1) 실행 (CMD, ENTRYPOINT) 기준



Docker Network 유형

유형	생성 방식	목적
bridge	Default (docker0)	리눅스 커널 에 생성된 가상 브리지 (L2 switch) docker0 bridge network 기반 통신
host	docker run --network host	호스트 네트워크 직접 사용 - Bridge 생성하지 않음 - Veth0 없음 - NAT 없음
custom bridge	docker network create my-bridge docker run --network my-bridge	새로운 Bridge Network를 별도 생성

Docker Network 유형



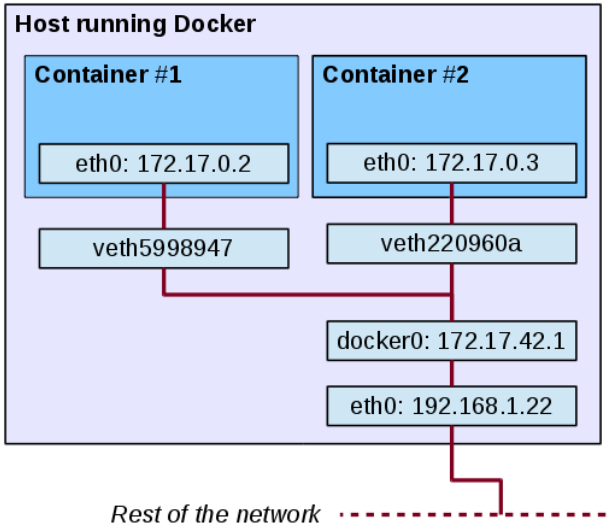
Container (172.17.0.2)

→ eth0 (L2) 이더넷
→ veth (L2) 가상 이더넷
→ docker0 (bridge)
→ Host IP stack (NAPT 처리)
→ enp0s3 (L2 NIC)
→ Router
→ Internet

Docker Networking (bridge 및 가상 NIC)

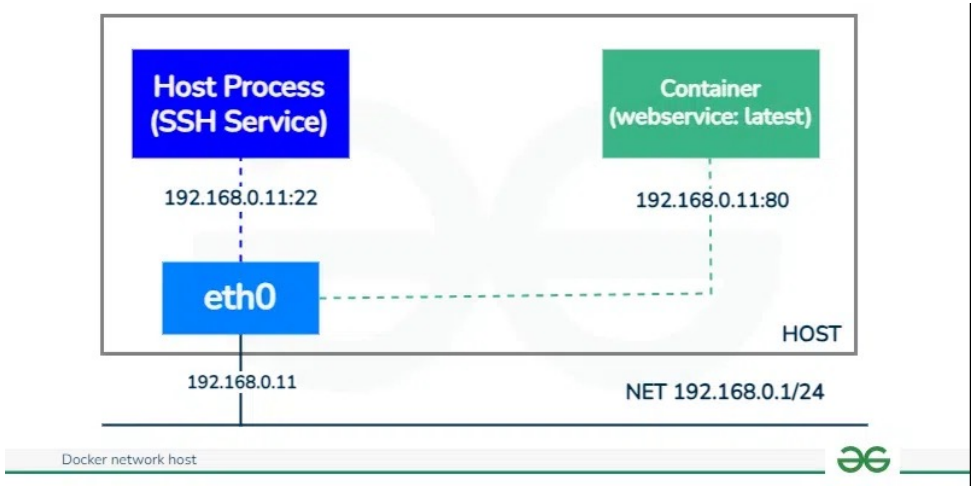
- Docker 컨테이너는 서버의 물리 NIC와 별도로 각 컨테이너 마다 가상 NIC 할당
- Default gateway로 Linux bridge 인 docker0를 만듦
  - ✓ 외부 네트워크와 교환하는 페킷에 NAT 작업 수행
  - ✓ 컨테이너 간 네트워크 연결
- 각 컨테이너에 격리된 네트워크 공간을 만들고 컨테이너의 eth0에 static IP 할당

--network bridge



HOST의 netfilter + ipTable을 통해
DNAT에서 container IP : port 를 매핑
예) 192.168.1.22:**8888** → 172.17.0.3:**8080**

--network host



※ AWS는 VPC Network 이용하고, calico 도 적용할 수 있음

[참고] 패킷 변환 및 전달 흐름

HostIP: 192.168.1.22:888 → **컨테이너 IP:** 172.17.0.3:8080으로 전달되는 흐름

1. Host 물리 NIC 진입 → Netfilter 최초 낚아채기 (PREROUTING)

- 외부 Client → eth0 → Netfliter PREROUTING → ipTable NAT 규칙 (8888 → 172.17.0.3:8080)

2. Kernel Routing Table 검색 → docker0 결정

- DST IP: 172.17.0.3 → kernel Routing Table → 172.17.0.0/16 dev docker0 --> docker0 결정
- 커널 라우팅 테이블 (Routing Table)의 **172.17.0.0/16 대역은 docker0로 결정**

3. docker0 브리지 연결된 가상 NIC (Veth)로 중계

- Host Kernel 은 Host Network Interface(eth0) Negihbor(or ARP) 테이블을 통해 172.17.0.3 의 MAC 주소 (02:42:ac:11:00:03)를 확인
- Kernel 은 해당 MAC 주소를 destination MAC으로 Ethrenet Frame을 구성하여 docker0로 전달
- docker0는 FDB에서 Destination MAC에 해당하는 Bridge Port(vethxxx)를 검색
- docker0 → veth220960a → Container eth0로 Frame 전달

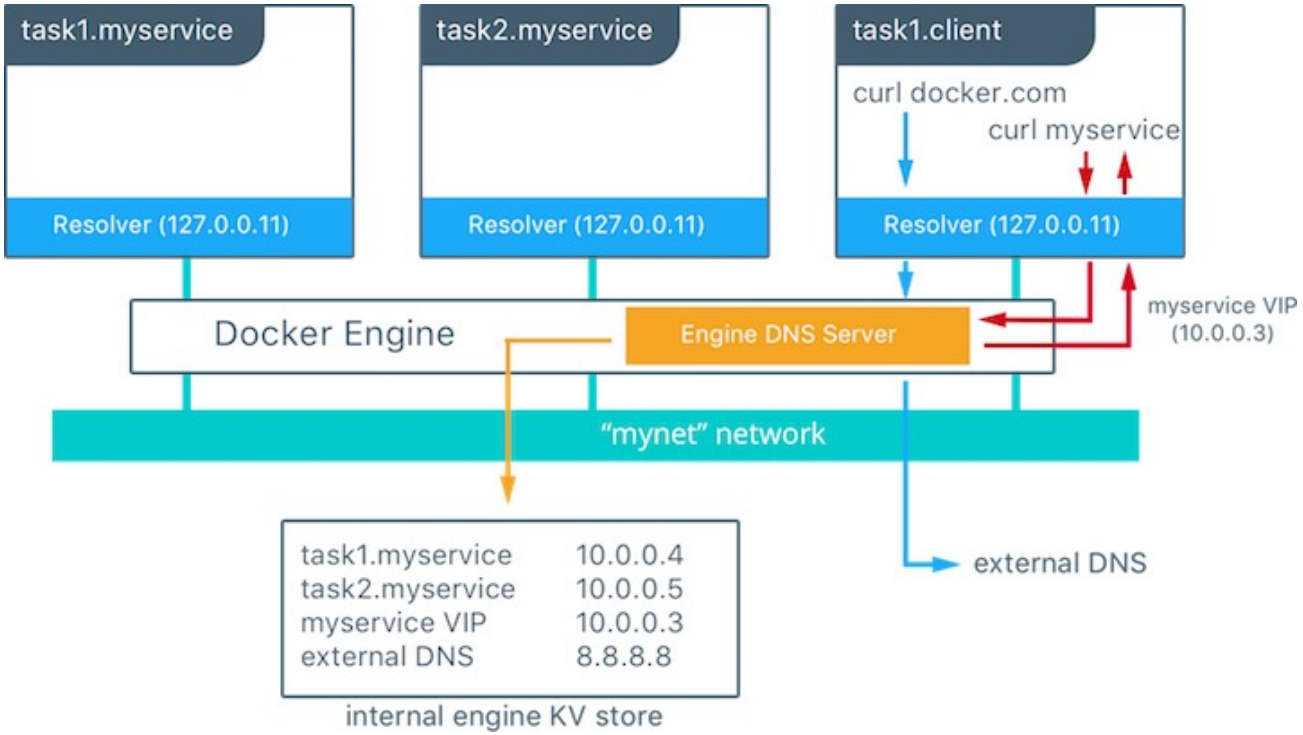
Bridge Port : Bridge에 연결되어 있는 Network Interface를 의미
ARP (Address Resolution Protocol) Table = Negihbor Table: IP 와 MAC Address 매핑 테이블
FDB (Forwarding Database): MAC Address → Bridge Port (veth) 매핑 테이블

Network 유형 별 특성

docker0를 포함한 bridge는 L2 Switch 역할 수행 (MAC 기반 vethXX → eth0 연결)

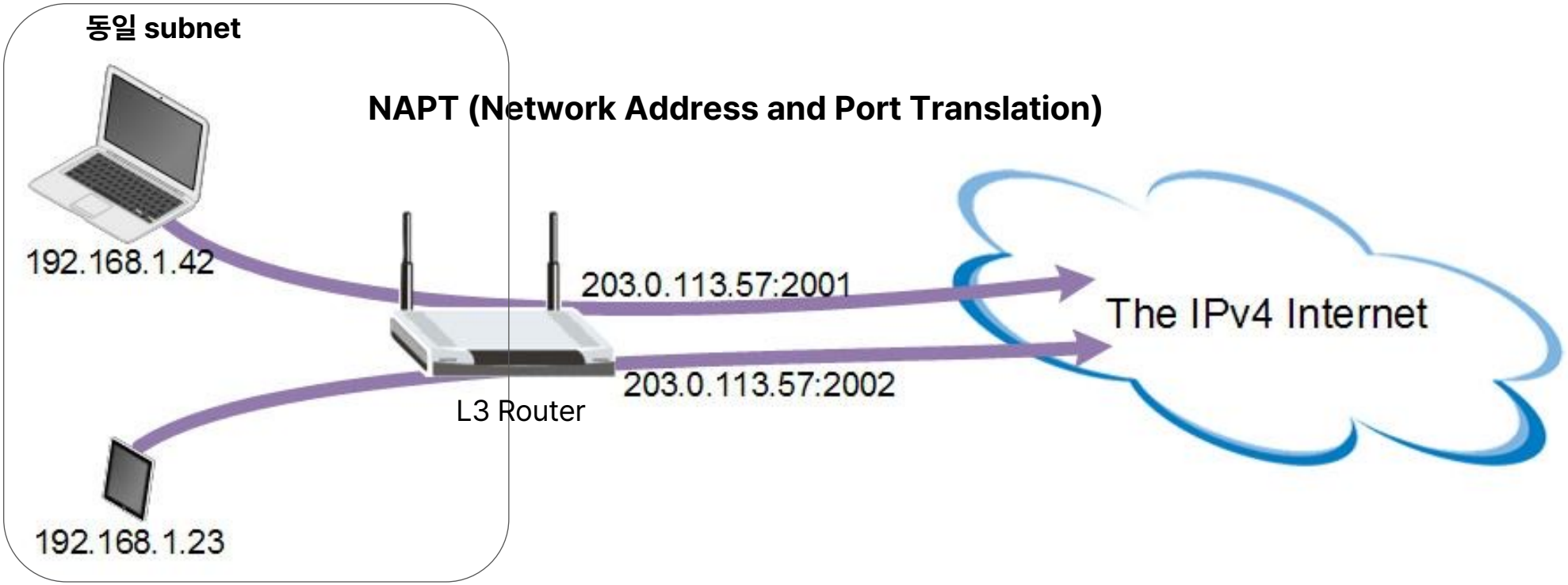
유형	격리	DNS	성능	사용 목적
bridge	O	✗	보통	단순 테스트
custom bridge	O	✓	보통	**실무 기본**
host	✗	✗	최고	성능 최우선

Custom Bridge는 DNS를 지원하는데 이것은 Docker Network 내부에 DNS 서버가 자동으로 동작



[참고] 무선공유기 DHCP (Dynamic Host Configuration Protocol)

무선 공유기(또는 DHCP 서버)가 각 기기에게 자동으로 IP 주소, 서브넷 마스크, 게이트웨이, DNS 서버 등을 할당해 주는 프로토콜



ipconfig / ifconfig로 나의 IP를 확인하고 다른 사람 네트워크로 접속해보기

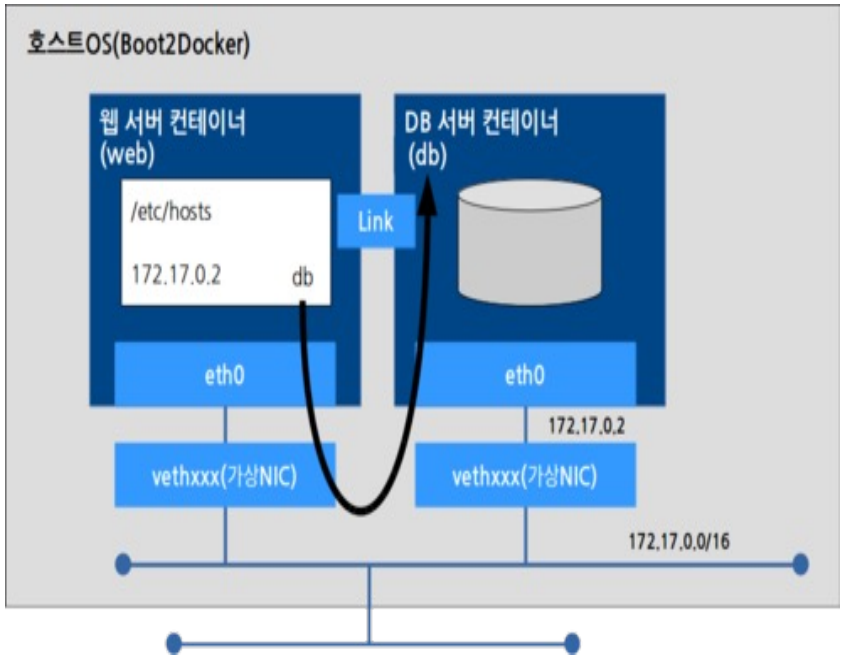
Docker Networking (Docker 컨테이너간 통신)

Docker Network은 **단일 노드에서의 통신**만을 정의하고 있음

Docker 컨테이너간 통신

링크 기능을 사용한 통신

(동일 호스트내 bridge docker0에 접속한 컨테이너간 통신)

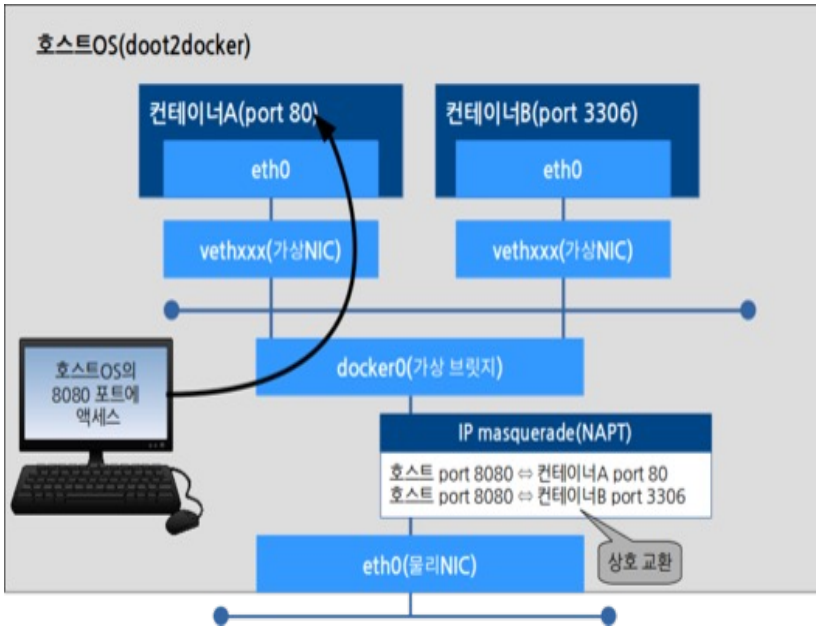


Docker 컨테이너와 외부 네트워크 통신

가상 bridge docker0와 Host OS의 물리 NIC에서 패킷 전송

(DNAT: Destion NAT, IP + PORT 변환)

docker run -d -p 8080:80 nginx



[참고] kubernetes calico Network

Calico는 Kubernetes에서 사용하는 CNI(Container Network Interface) 구현체로,
멀티 노드 환경에서 Pod 간 네트워크 통신을 가능하게 하는 네트워크 플러그인

항목	Docker Network	Calico
대상	컨테이너	Kubernetes Pod
범위	단일 노드 중심	**멀티 노드 기본**
네트워크 생성 주체	Docker Engine	Kubernetes + CNI
서비스 디스커버리	Docker DNS	CoreDNS
보안 정책	제한적	**NetworkPolicy**

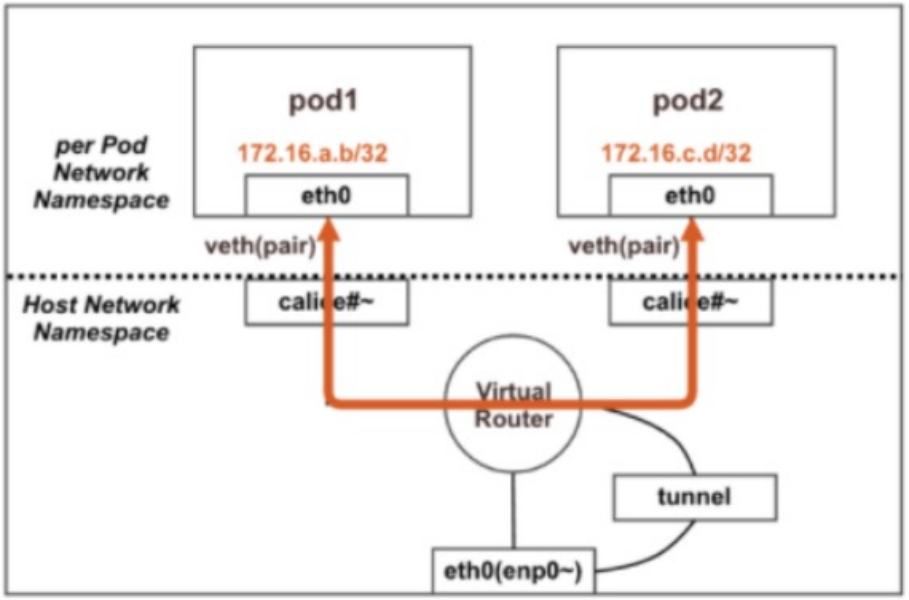
참고: BGP (Border Gateway Protocol) : 라우팅 프로토콜(Routing Protocol)로, L3(IP 기반)의 경로 정보를 교환

[참고] kubernetes calico Network

Kubernetes는 calico, EKS VPC CNI 등의 CNI 기반 네트워크 플러그인을 제공하고 있음

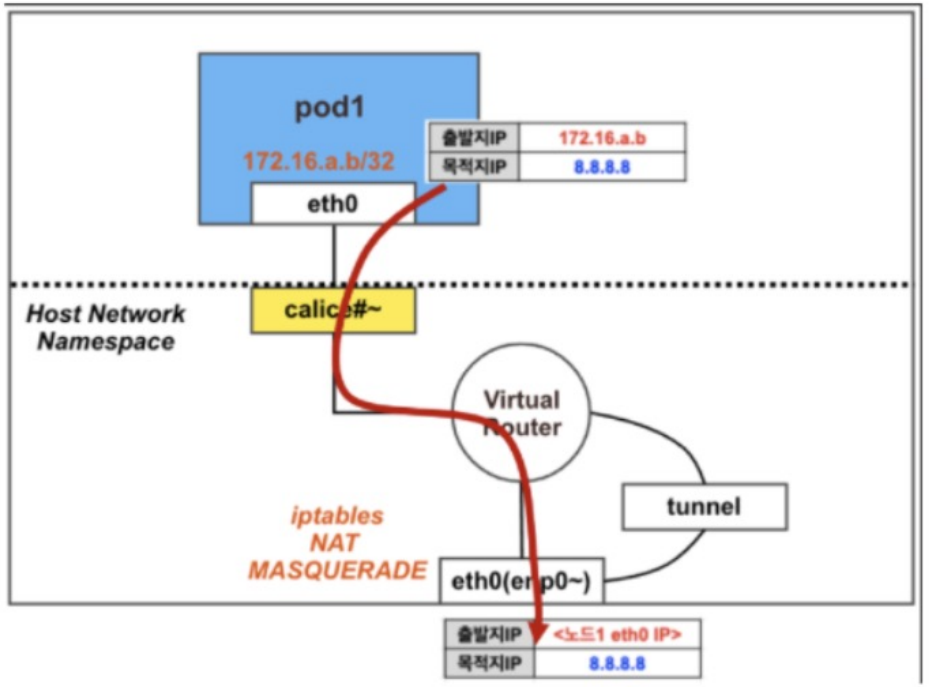
단일 노드 내 컨테이너 간 통신

단일 노드내에서 통신 시 etho0: veth → caliXXX:VR



외부 노드의 컨테이너와 통신

etho0:veth → caliXXX:VR →tunl 0 → etho0



tunnel (tunl0) : 리눅스 커널이 제공하는 **IP-in-IP (IPIP) 터널 인터페이스** 다른 노드에 패킷 전송시 Container IP를 Encapsulation하고 Node IP를 기반 통신

Virtual Router (flex + BGP daemon) : "Pod IP (CIDR 대역)가 어느 노드에 있는지를 알려주는 역할"

예시: 10.1.2.0/24 → 192.168.100.20 eth0

참고: **BGP (Border Gateway Protocol)** : 라우팅 프로토콜(Routing Protocol)로, L3(IP 기반)의 경로 정보를 교환

[참고] CIDR (Classless Inter-Doamin Routing)

IP 주소 범위를 표현하는 방식

```
192.168.0.0/24
```

- 192.168.0.0 : 네트워크 시작 주소
- /24 : 앞의 24비트가 네트워크 부분이라는 뜻

```
      192         168         0           0
       ↓           ↓          ↓           ↓
11000000 . 10101000 . 00000000 . 00000000
```

```
/24
11111111.11111111.11111111.00000000
```

사용 이유

예전에는 클래스로 A, B, C를 나눴는데 너무 비효율적이어서,
CIDR은 필요한 크기만큼 유연하게 IP 대역을 나누기 위해 사용

예시

192.168.0.0/24

- 앞 24비트가 네트워크
- 뒤 8비트가 호스트
- 총 256개 주소 범위
  (Broadcast, NA 제외 총 254개)
- 보통 192.168.0.0 ~ 192.168.0.255

192.168.0.0/24

```
11000000.10101000.00000000.xxxxxxxx
|<---------- /24 ---------->|
                            |
                      00000000 ~ 11111111
                         0       ~   255
```

network address: 11000000.10101000.00000000.**00000000**
broadcast : 11000000.10101000.00000000.**11111111**

[참고] Unix Domain Socket (UDS) 프로그램

네트워크 대신 파일 경로(Socket file)를 주소로 사용하는 로컬 IPC Socket

- **TCP Socket**: 127.0.0.1:8080
- **Unix Domain Socket**: /tmp/my.sock

특징

- IP 스택/라우팅/포트 등 네트워크 계층을 거치지 않음
- 같은 서버 내부 통신에는 TCP 보다 더 빠르고 오버헤드가 적음
- 소켓이 파일이라서 권한 제어 가능
- 외부 네트워크 접근 자체 불가능

사용처

- Docker: /var/run/docker.sock
- Kubernetes: (kubelet과 통신 시)
  - ✓ containerd: /run/containerd/containerd.sock
  - ✓ CRI: /run/crio/crio.sock
- Nginx ↔ upstream 앱 통신(예: PHP-FPM)
- DB 로컬 연결
  - ✓ MySQL: /var/run/mysqld/mysqld.sock
  - ✓ PostgreSQL도 로컬 소켓 사용 가능

[참고] Unix Domain Socket (UDS) 프로그램

dockerd ←→ containerd ←→ containerd-shim ←→ init process 간 통신
일반적으로 Unix Domain Socket을 이용해서 통신 실행

- **TCP Socket**: 127.0.0.1:8080
- **Unix Domain Socket**: /tmp/my.sock

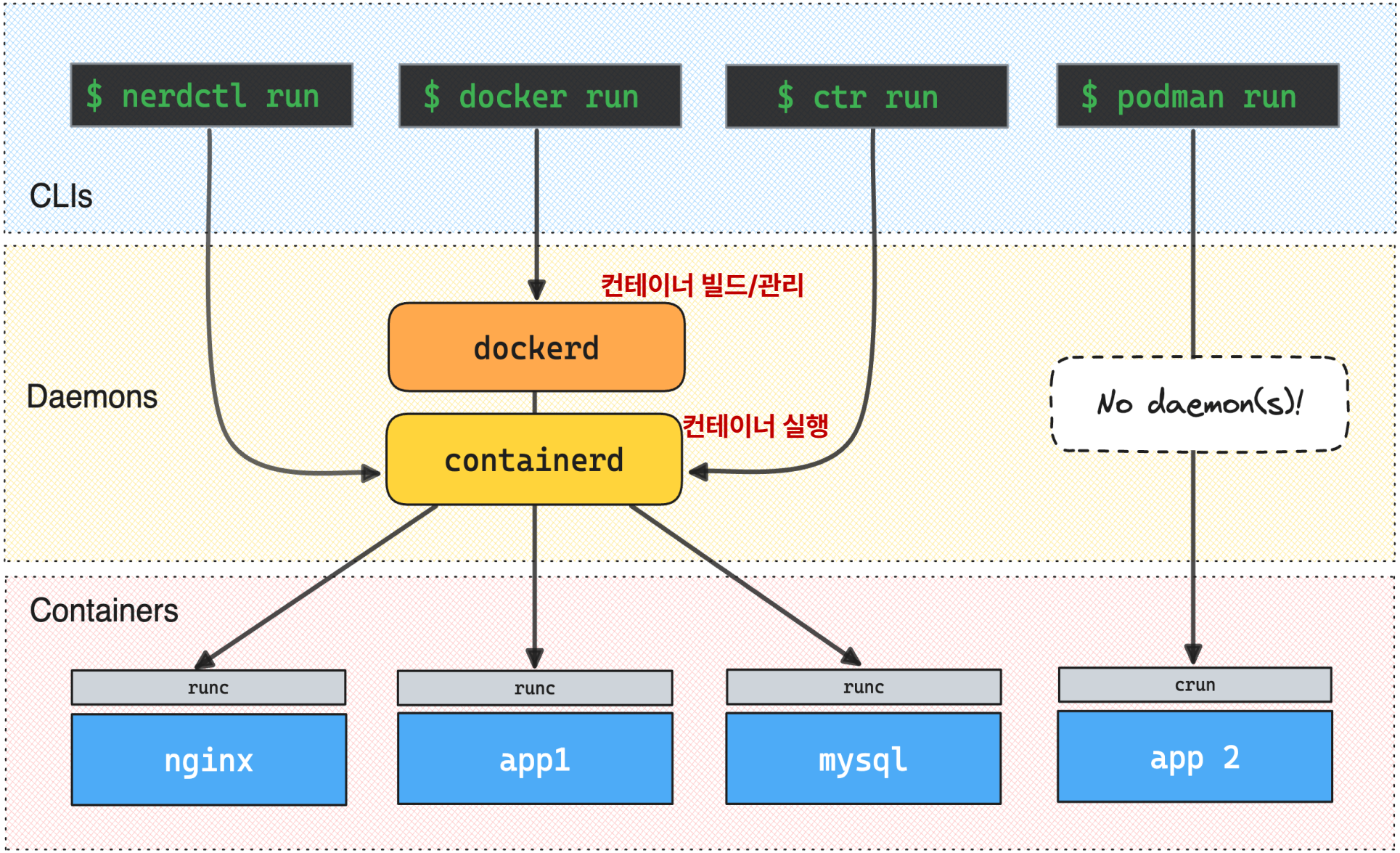
특징

- IP 스택/라우팅/포트 등 네트워크 계층을 거치지 않음
- 같은 서버 내부 통신에는 TCP 보다 더 빠르고 오버헤드가 적음
- 소켓이 파일이라서 권한 제어 가능
- 외부 네트워크 접근 자체 불가능

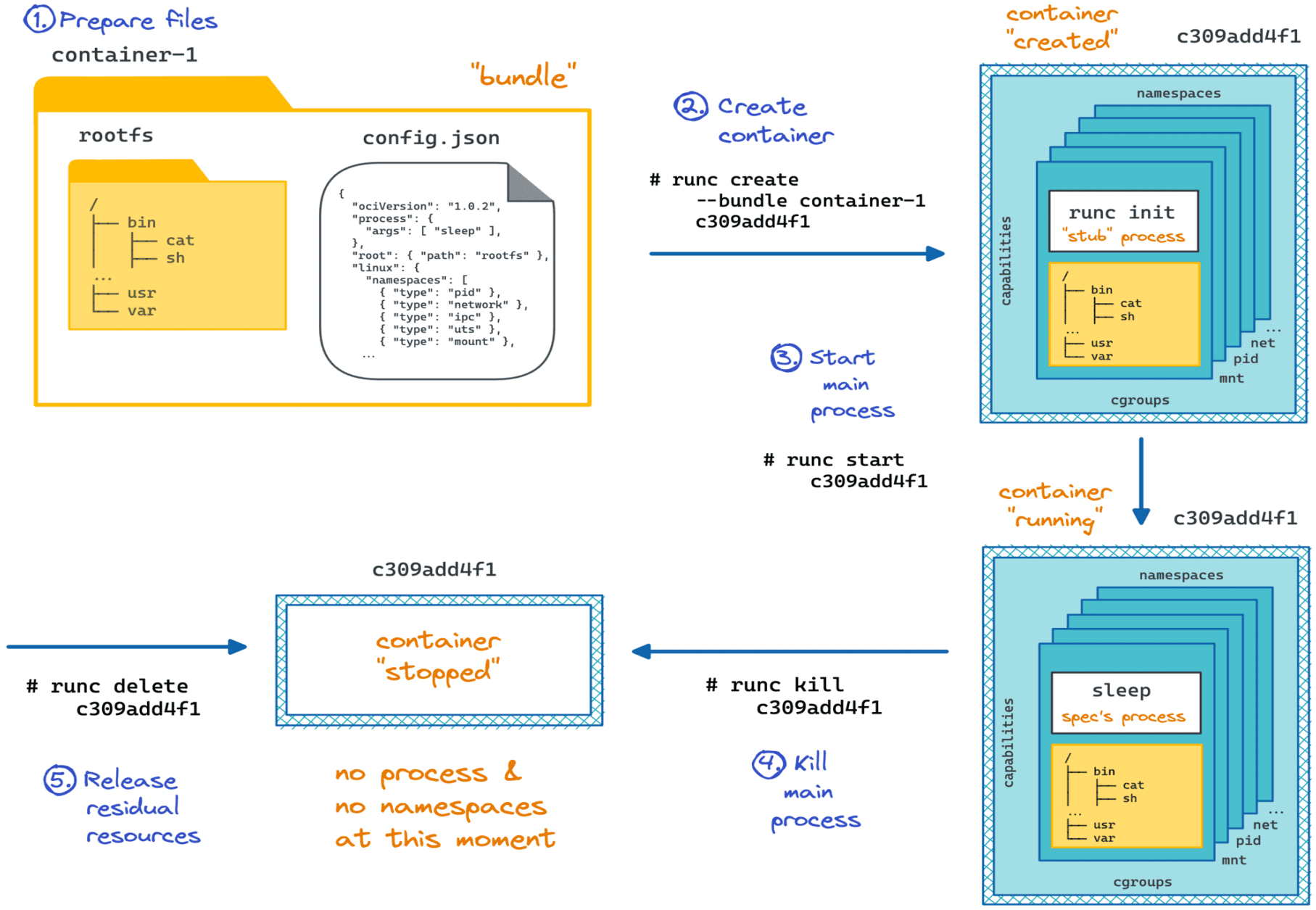
사용처

- **Docker: /var/run/docker.sock**
- **Kubernetes: (kubelet과 통신 시)**
  - ✓ containerd: /run/containerd/containerd.sock
  - ✓ CRI: /run/crio/crio.sock
- **Nginx ↔ upstream 앱 통신(예: PHP-FPM)**
- **DB 로컬 연결**
  - ✓ MySQL: /var/run/mysqld/mysqld.sock
  - ✓ PostgreSQL도 로컬 소켓 사용 가능

runc 들여다 보기


$ nerdctl run
$ docker run
$ ctr run
$ podman run
CLIs
Daemons
컨테이너 빌드/관리
dockerd
containerd
컨테이너 실행
No daemon(s)!
Containers
runc
runc
runc
crun
nginx
app1
mysql
app 2

runc 기반 container 라이프사이클 관리



[실습] runc 직접 동작시켜보기

dockerd-shim이 직접 호출해서 컨테이너를 생성하는 과정을 직접 수동으로 동작시키면서
OCI 런타임 표준과 컨테이너 동작 방식을 이해해보자

ubuntu 컨테이너 내부에서 runC를 설치하고 직접 실행

containerd-shim 대신 본인이 runc를 직접 호출해서 다음 동작을 확인

① namespace와 cgroup을 설정하고,

② pivot_root를 통해 컨테이너의 root filesystem을 변경한 뒤,

③ 해당 namespace 환경에서 프로세스를 실행한다.

실습 코드 위치

```
01.answer-code/runc/
-  Dockerfile
-  build.sh
-  run.sh
```

[실습] runc 직접 동작시켜보기

Dockerfile 확인하기

```
FROM ubuntu:24.04

RUN apt-get update && apt-get install -y curl lsb-release nginx

RUN apt-get install -y python3 python3-pip
RUN apt-get install -y jq vim net-tools lsof tar gzip runc
RUN apt-get clean && \
    rm -rf /var/lib/apt/lists/*
WORKDIR /app
COPY webserver.py .

# Docker CLI 설치
RUN install -m 0755 -d /etc/apt/keyrings && \
    curl -fsSL https://download.docker.com/linux/ubuntu/gpg | gpg --dearmor -o /etc/apt/keyrings/docker.gpg && \
    chmod a+r /etc/apt/keyrings/docker.gpg && \
    bash -lc 'ARCH=$(dpkg --print-architecture); \
    echo "deb [arch=${ARCH} signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu \
    $(. /etc/os-release && echo "$VERSION_CODENAME") stable" > /etc/apt/sources.list.d/docker.list' && \
    apt-get update && \
    apt-get install -y docker-ce-cli && \
    rm -rf /var/lib/apt/lists/*

CMD ["python3", "webserver.py"]
```

[실습] runc 직접 동작시켜보기

build.sh

```
#!/bin/bash
IMAGE_NAME="runc-test"
VERSION="1.0"
#IS_CACHE="--no-cache"

# Docker 이미지 빌드
docker buildx build \
  --tag ${IMAGE_NAME}:${VERSION} \
  --platform linux/arm64,linux/amd64 \
  --file Dockerfile \
  ${IS_CACHE} .
```

run.sh

```
#!/bin/bash

set -eux

IMAGE_NAME="skala-registry.skala-ai.com/library/runc-test"
VERSION="1.0"

if docker ps -a --format '{{.Names}}' | grep -q '^runc-test$'; then
  docker stop runc-test
  docker rm runc-test
fi

docker run -d \
  --name runc-test \
  -p 8888:8080 \
  -v /var/run/docker.sock:/var/run/docker.sock \
  --group-add 0 \
  --privileged \
  ${IMAGE_NAME}:${VERSION} \
  sleep infinity
```

[실습1] runc 직접 동작시켜보기

1. ubuntu 컨테이너 실행

먼저 ubuntu + runc 기능을 포함하는 컨테이너 runc-test:1.0 이미지를 만들어보자

```
# runc-test 빌드하기
./build.sh

# 레포지토리 루트에서
./run.sh

# 컨테이너가 정상적으로 올라왔는지 확인
docker ps --filter name=runc-text
```

2. ubuntu 컨테이너에 진입

```
# 호스트에서 컨테이너로 진입
docker exec -it runc-test /bin/bash
```

3. runc-test container 내부

```
# 설치 확인
> runc --version
```

[실습 1] runc 직접 동작시켜보기

4. OCI bundle(rootfs + config.json) 준비

OCI bundle은 container image를 풀어놓은 rootfs와 컨테이너 실행을 위한 설정 환경 정보입니다. **(Containerd의 역할)**

```
# 작업 디렉토리 생성 및 이동
> cd

> mkdir -p ./mybundle/rootfs   # alpine:latest image 를 풀어헤치기 위한 rootfs

> cd ./mybundle

# Docker 소켓을 공유했으므로 컨테이너 내부에서도 Docker 명령 사용 가능
# Alpine 이미지의 rootfs 추출
> docker export $(docker create alpine:latest) | tar -C rootfs -xf -

# 풀어진 이미지 내용 확인하기
> ls -al rootfs

# 기본 OCI spec 생성 (config.json)
> runc spec

> ls
config.json  rootfs
```

[실습 1] runc 직접 동작시켜보기

5. OCI Bundle 기반 container 생성 및 실행

```
# container 생성/실행
> runc run mycontainer

/# ps -ef
PID   USER     TIME  COMMAND
    1 root      0:00 sh
    7 root      0:00 ps –ef

/# ifconfig

/# exit
```

별도의 터미널을 열고 생성된 컨테이너를 확인해보자

```
> docker exec -it runc-test /bin/bash

> runc list
ID            PID         STATUS      BUNDLE           CREATED
mycontainer   230         running     /app/mybundle    Z   root
```

[실습 2] 볼륨 마운트하기

볼륨 마운트하기

config.json에 볼륨을 마운트해보자

```
# Ubuntu 컨테이너 내부에서 공유할 디렉토리 생성
> mkdir -p /mydata
> echo "Hello from host" > /mydata/test.txt

# config.json에 마운트 추가
> cd /app/mybundle
> cat config.json | jq '.mounts += [{
  "destination": "/data",
  "type": "bind",
  "source": "/mydata",
  "options": ["rbind", "rw"]
}]' > config_volume.json

# 추가된 마운트 확인
> cat config_volume.json | jq '.mounts[] | select(.destination == "/data")'

> cp config_volume.json config.json
```

마운트 대상 디렉토리와 파일 추가하기

```
> runc run mycontainer
/# cd /data
/data # ls
/data # cat test.txt

/data # exit
```

[실습 3] Local Host와 컨테이너 간 PID 공유하기

config.json의 namespaces 에 PID type을 제거 후 Host 와 PID 공유를 확인해보자

config.json에서 PID Namespace를 제거해서 host PID 와 공유하는 것을 확인해보자

```
# config.json의 namespace의 pid와 network을 각각 제거해보기
> vi config.json
    "namespaces": [
                {
                    "type":  " pid"
                },
                {
                    "type": "uts"
                },
                {
                    "type": "network"
                },
    ],
<ESC>
:wq!
```

제거 하기

컨테이너 내에서 PID 검색 시 Host PID 목록이 보임

```
# /mydata 생성하기
> runc run mycontainer
# ps -ef
PID   USER     TIME  COMMAND
    1 root      0:00 sleep infinity
  240 root      0:00 /bin/bash
  300 root      0:00 runc run mycontainer
  314 root      0:00 sh
```

[실습 4] Local Host와 컨테이너 간 Network 공유하기

config.json의 namespaces 에 network type을 제거 후 Host 와 네트워크 공유를 확인해보자

```
# config.json의 namespace의 pid와 network을 각각 제거해보기
> vi config.json

     "namespaces": [
                {
                    "type": "uts"
                },
                {
                    "type": "network"
                },
    ],

<ESC>
:wq!
```

제거 하기

컨테이너 내에서 Network 검색 시 Host Network 목록이 보임

```
# /mydata 생성하기
> runc run mycontainer
/ # ifconfig
eth0      Link encap:Ethernet  HWaddr 8A:63:9C:1E:23:23
          inet addr:172.17.0.2  Bcast:172.17.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:65535  Metric:1
          RX packets:12 errors:0 dropped:0 overruns:0 frame:0
          TX packets:3 errors:0 dropped:0 overruns:0 carrier:0
          collisions:0 txqueuelen:0
          RX bytes:1172 (1.1 KiB)  TX bytes:126 (126.0 B)
```

다른 터미널에

```
> docker exec -it runc-test /bin/bash
```

실행후

```
>ifconfig
```

확인

[실습 5] runc 를 통해 직접 init process 실행하기

runc를 통해 init(PID 1)을 직접 실행하는 스크립트를 만들어보자 Dockerfile의 CMD ["/bin/sh", "/init.sh"] 과 동일하게 동작

1. init.sh PID 1 프로세스 만들기

```
cd /app/mybundle

cat > rootfs/init.sh <<'EOF'
#!/bin/sh
echo
echo [init] hello from runc
date

sleep 1
echo [init] done
sleep 3600
EOF

chmod +x rootfs/init.sh
```

2. config.json에서 init 프로세스 등록하기

```
"process": {
   "terminal": true,
   "user": {
     "uid": 0,
     "gid": 0
   },
   "args": [
     "sh", "/init.sh"
   ]
```

false 로 변경

추가

3. runc를 통해 container 실행하기

```
# runc 를 동작시킬때 config.json이 있는 위치여야 함
> ls
config.json rootfs

> runc run -d mycontainer

> runc list
ID           PID         STATUS      BUNDLE      CREATED                          OWNER
mycontainer  1112        running     /mybundle   2026-02-25T22:37:27.852760555Z   root

> runc state mycontainer
ID           PID         STATUS      BUNDLE          CREATED                          OWNER
mycontainer  231         running     /app/mybundle   2026-08-20T11:41:42.783856045Z   root
```

[실습 5] runc 를 통해 직접 init process 실행하기

runc를 통해 init(PID 1)을 직접 실행하는 스크립트를 만들어보자 **Dockerfile의 CMD ["/bin/sh", "/init.sh"] 과 동일하게 동작**

4. mycontainer 내부 init.sh 실행 확인하기

```
# runc 를 동작시킬때 config.json이 있는 위치여야 함
> runc exec -t mycontainer /bin/sh
/ # ps -ef
PID   USER     TIME  COMMAND
    1 root      0:00 sh /init.sh
    9 root      0:00 sleep 3600
   23 root      0:00 /bin/sh
   29 root      0:00 ps -ef

/ # exit

> ps -ef
```

4. container 종료시키기

```
> runc list
ID            PID         STATUS      BUNDLE           CREATED                          OWNER
mycontainer   231         running     /app/mybundle    2026-08-20T11:41:42.783856045Z   root
> runc kill mycontainer KILL
ID            PID         STATUS      BUNDLE           CREATED                          OWNER
mycontainer   0           stopped     /app/mybundle    2026-08-20T11:41:42.783856045Z   root

> runc delete mycontainer
> runc list
```

컨테이너가 실행단위인가?

컨테이너란?
containerd나 dockerd 등의 컨테이너 런타임이 정의한 **논리적 그룹**이며,
Linux 커널의 namespace와 cgroup 기능으로 프로세스 그룹을 격리·제한하여 구현된 것

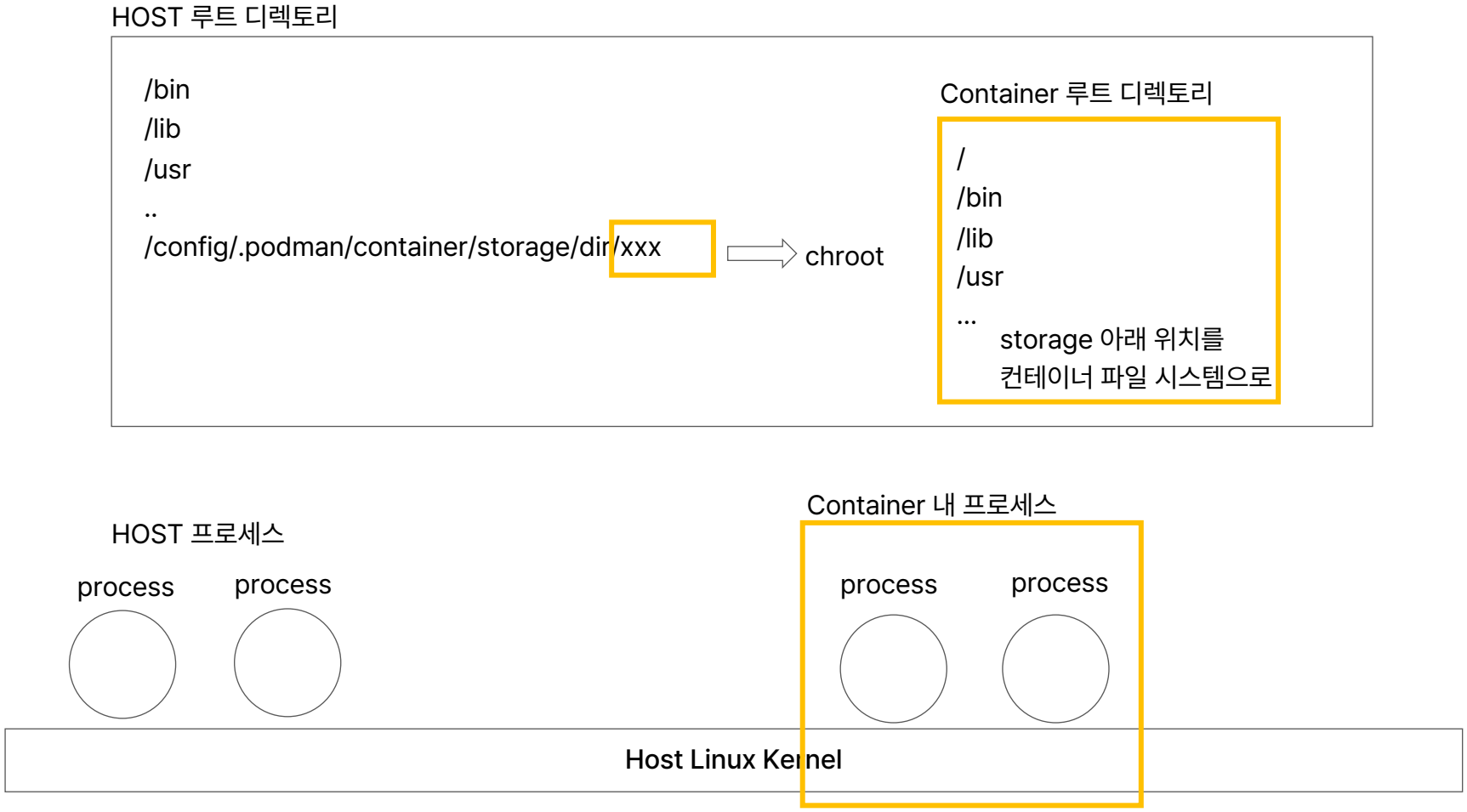
그리고, dockerd, containerd 등이 컨테이너라는 단위로 논리적으로 추상화하여 관리

컨테이너는 결국 **"프로세스의 모음"**이며, 그 실행은 **일반 리눅스 프로세스와 같지만**,
단지 런타임에 의해 격리(namespace) 및 자원 제한(cgroup) 된 상태로 운영되는 것

dockerd, containerd 등이 컨테이너라는 단위로 논리적으로 추상화하여 관리

컨테이너의 실행 주체는 프로세스

컨테이너는 Host 환경의 파일 디렉토리와 프로세스로 동작한다.





KDT Cloud – 3. Container의 이해

Docker-compose 명령어

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

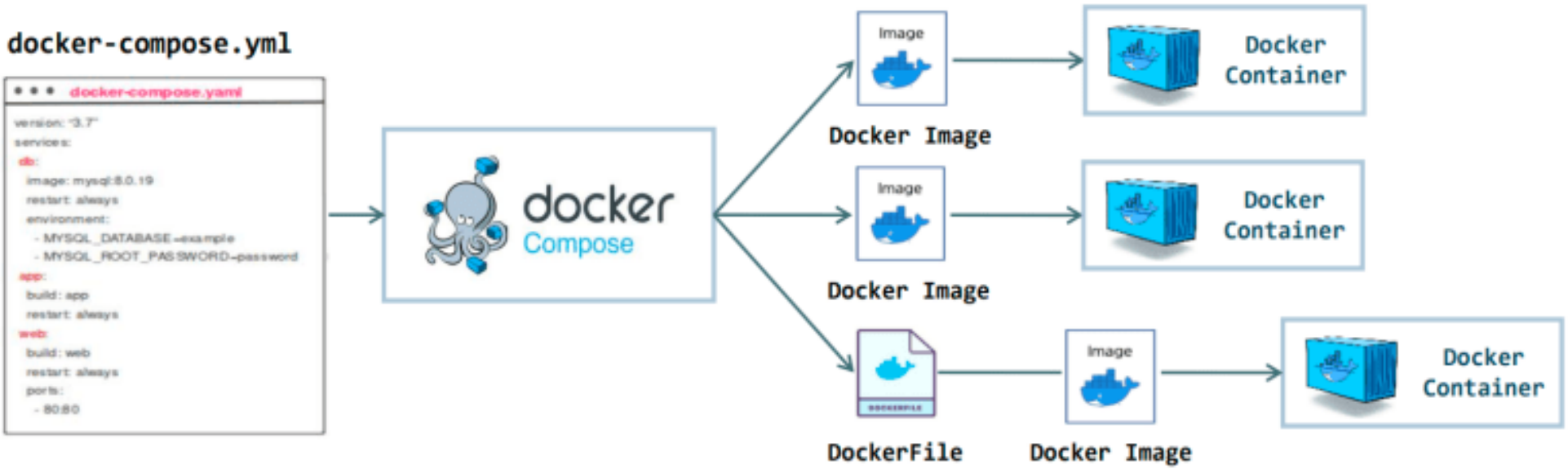
SK 주식회사 C&C

Docker-Compose란?

단일 노드 내에서 여러 개의 컨테이너를 조율하는 컨테이너 오케스트레이션 플랫폼

주요 특성

- 여러 컨테이너로 이루어진 애플리케이션 실행
- 로컬 개발 환경 구성
- 교육 및 실습 환경 구성 시 활용
- 통합 테스트 환경 구성 시 활용



Docker-Compose란?

여러 개의 컨테이너로 구성된 애플리케이션을 하나의 Yaml (docker-compose.yaml)로 정의하고 한 번의 명령으로 실행/중지/관리할 수 있게 해주는 도구

- 컨테이너가 많아질수록 명령어가 복잡
- 네크워크/볼륨/환경변수 반복 설정
- 실행 순서 관리 어려움

역할	설명
서비스 정의	컨테이너를 “**서비스**”로 선언
네트워크 구성	자동 custom bridge 생성
볼륨 관리	데이터 영속성
실행 순서	depends_on
환경 설정	env, secrets
전체 수명주기	up / down / restart

```
services:
  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: root

  backend:
    image: my-backend
    depends_on:
      - db

  frontend:
    image: my-frontend
    ports:
      - "8080:80"
```

docker-compose 기본 구조

docker compose yaml 구조

```
services:
  db:
    image: postgres:15
    volumes:
    - ./data:/var/lib/postresql/data
    environment:
      POSTGRES_PASSWORD: postgres

  backend:
    image: my-backend
    depends_on:
    - db

  frontend:
    image: my-frontend
    ports:
    - "8080:80"
```

서비스 단위 (컨테이너 실행 과 통신 단위)

- **db**: postgres 서비스 명
- **image**: 컨테이너 이미지 명
- **volumes**: 로컬 ./docker/data와 마운트
- **environment**: 환경 변수 설정
- **ports**: 포트 지정

참고: docker run –v 와 docker-compose volumes의 경로 설정 방식??
- 절대 경로와 상대 경로

[실습] docker-compose 시작하기

docker compose를 실행해보기

01.answer-code/10.docker-compose/01.base

1. docker-compose.yaml

```yaml
services:
  db:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: postgres
    ports:
      - 5432:5432
    restart: always

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile.backend
    ports:
      - 9090:8080
```

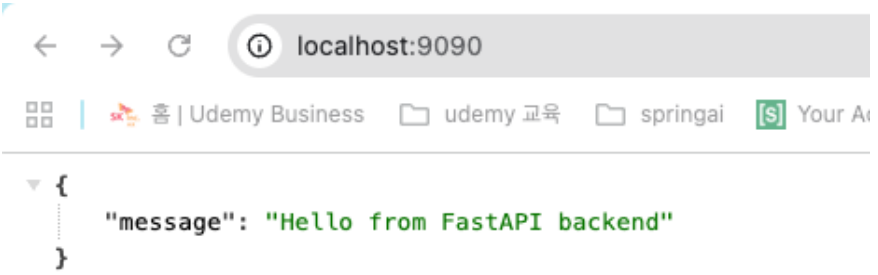
2. 실행하기

```
> cd 10.docker-compose/01.start

# backend image를 빌드하고
# db 와 backend를 실행해보자
> docker compose up --build -d
```

3. 접속하기

http://localhost:9090



4. 종료 및 네트워크 삭제

```
> docker compose down
```

명령어: 컨테이너 빌드/실행/중지

docker-compose.yaml 로 여러 컨테이너를 한 번에 정의/실행/종료/복구하는 도구

이미지 빌드

```
docker compose build
docker compose build --no-cache      # 캐시 없이 완전 빌드
docker compose build backend         # 특정 서비스만 빌드
```

빌드 + 실행

```
docker compose up                    # forground 실행
docker compose up -d                 # background 실행
docker compose up -d db backend      # 특정 서비스 만 실행

docker compose up -d --build         # 빌드 및 실행
```

중지 및 실행

```
docker compose stop                  # 중지
docker compose start                 # 중단된 서비스 시작
docker compose restart               # stop & start
docker compose restart  backend      # 특정 서비스만 stop & start
```

명령어: 상태 확인

로그 추적

```
docker compose logs -f                  # 모든 서비스 로그 보기
docker compose logs -f  backend         # 대상 지정
```

컨테이너 상태 확인

```
docker compose ps                                   # 실행 중인 컨테이너 상태 요약
docker compose ps -a                                # 실행 중 + stop 컨테이너 상태 요약
```

특정 컨테이너 메타데이터 상세

```
docker  inspect  {docker-name}      | less          # less 는 결과를 보기좋게 보이게 함
```

- 컨테이너 상태
- 실행 여부, 종료 코드, OOM 여부
- 네트워크 정보
- IP 주소, 연결된 네트워크
- 마운트
- 볼륨/바인드 마운트 경로
- 환경 변수
- 실행 명령 (CMD, ENTRYPOINT)
- restart 정책
- healthcheck 결과

명령어: 리소스 제거

중지 후 컨테이너 제거 (네트워크는 제거되지 않음)

```
docker compose rm
```

컨테이너 + 네트워크 제거

```
docker compose down          # container와 network 모두 제거
```

컨테이너 + 네트워크 + 볼륨 제거

```
docker compose down   -v         # container, network, volume 모두 제거
```

명령어: 장애 복구

설정/환경 반영을 위해 재 생성 (recreate)

```
docker compose up -d --force-recreate backend
```

초기화 및 재 기동 (네트워크까지 초기화)

```
docker compose down
docker compose up -d --build
```

명령어: 디버깅

컨테이너 내부 쉘 접속

```
docker compose exec backend sh

# (ubuntu/debian 계열이면 bash)
docker compose exec backend bash
```

컨테이너에서 네트워크 확인

```
docker compose exec backend cat /etc/resolv.conf
docker compose exec backend ping db
```

명령어: 네트워크 점검

네크워크 목록/상세

```
docker network ls                          # 네트워크 목록 검색
docker network inspect <network_name>      # 네트워크에 어떤 컨테이너가 연결되어 있는지 확인
```

컨테이너가 어떤 네트워크에 연결되었는지 확인

```
docker inspect docker-compose-backend-1 | grep -i network –n
```

grep

- -i: 대소문자 무시
- -n: 줄 번호 표시

네크워크 삭제

```
docker network rm <network name>           # 특정 네트워크 삭제
docker network prune                       # 어떤 컨테이너도 연결되어 있지 않은 네트워크만 삭제
docker network prune -f                    # 네트워크 강제 삭제
```

명령어 : 특정 compose 파일 지정하기

특정 docker-compose yaml 또는 프로젝트 명으로 대상 지정 가능

-f option으로 compose 파일 지정

```
docker compose –f docker-compose-app1.yaml  ps
docker compose –f docker-compose-app2.yaml  log -f
```

-p option으로 compose 파일 지정

```
/home/user/
├─ app1/
│   └─ docker-compose.yml
├─ app2/
│   └─ docker-compose.yml
```

```
docker compose –p app1  ps
docker compose –p app2 log -f
```

project 이름 지정 방식

docker compose up은 기본적으로 현재 디렉토리 이름(app1, app2)을 compose project name으로 사용

docker compose –p myapp up -d 처럼 –p myapp 을 명시하면 myapp 을 compose project name으로 사용

[실습] docker-compose 확장하기

docker compose.yaml을 확장해보자

1. docker-compose.yaml

```
…
 frontend:
    build:
      context: ./frontend
      dockerfile: Dockerfile.frontend
    ports:
      - 8080:80
    volumes:
      - ./nginx.conf:/etc/nginx/conf.d/default.conf
```

2.nginx.conf

```
server {
    listen 80;

    # 정적 파일 제공
    location / {
        root  /usr/share/nginx/html;
        index index.html;
    }

    # 백엔드 API 프록시: /api/* → backend:8080/*
    location /api/ {
        proxy_pass         http://backend:8080/;
        proxy_set_header   Host $host;
        proxy_set_header   X-Real-IP $remote_addr;
    }
}
```

3. 실행하기

```
> docker compose up -build -d
```

4. 접속하기

http://localhost:8080

01.answer-code/10.docker-compose/02.add-fe 참고 가능

[검토] nginx.conf

frontend nginx 의 routing 을 위한 설정 정보는 nginx.conf를 default.conf로 마운트

```
server {
    listen 80;

    # 정적 파일 제공
    location / {
        root   /usr/share/nginx/html;
        index index.html;
    }

    # 백엔드 API 프록시: /api/* → backend:8080/*
    location /api/ {
        proxy_pass          http://backend:8080/;
        proxy_set_header    Host $host;
        proxy_set_header    X-Real-IP $remote_addr;
    }
}
```

- **proxy_pass:** 전달
- **$host**
  - 요청을 받는 도메인 주소
  - example.com
- **$remote_addr**
  - 클라이언트 IP 주소
  - 12.45.67.89



KDT Cloud – 3. Container의 이해

Docker-compose 이해

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

command 필드

컨테이너가 시작될 때 실행할 기본 CMD를 덮어쓰는 설정

- Dockerfile의 CMD를 대체 (override) 함
- Dockerfile의 ENTRYPOINT는 유지

Dockerfile

- container CMD

```
FROM nginx
CMD ["nginx", "-g", "daemon on;"]
```

- container ENTRYPOINT

```
FROM nginx
ENTRYPOINT ["docker-entrypoint.sh"]
CMD ["nginx", "-g", "daemon on;"]
```

docker-compose.yaml

- Dockerfile의 CMD를 덮어씀

```
services:
  web:
    image: nginx
    command: ["nginx", "-g", "daemon off;"]
```

- Dockerfile의 ENTRYPOINT와 CMD 모두 엎어씀

```
services:
  web:
    image: nginx
    # 1. 실행할 실행파일(Entrypoint)을 덮어씁니다.
    entrypoint: ["docker-entrypoint.sh"]
    # 2. Dockerfile에서 CMD가 무시되므로 다시 선언
    command: ["nginx", "-g", "daemon off;"]
```

이미지 빌드 및 실행

이미지를 빌드를 위한 환경 설정

```
services:
  app:
    build:
      context: ./app
      dockerfile: Dockerfile.prod
      args:
        APP_ENV: prod
    ports:
      - "8080:8080"
```

```
docker build --tag myapp:1.0 .
```

디렉토리 구조

```
project/
├─ docker-compose.yml
└─ app/
   ├─ Dockerfile.prod
   └─ app.jar
```

Dockerfile.prod

```
FROM eclipse-temurin:17-jre
ARG APP_ENV
ENV APP_ENV=${APP_ENV}
```

예시) docker build ... --build-arg APP_ENV=prod

./app 디렉토리를 context로 이미지 빌드 후 바로 컨테이너 실행

```
docker-compose up --build
```

=

```
docker-compose build
docker-compose up
```

장애 발생 시 재 시작 모드 설정

docker-compose.yaml

```
services:
  db:
    image: postgres:9.6.1
    restart: always
```

값	의미	
no	재시작 안 함 설정하지 않으면 **no가 defult**	일회성 작업 스크립트, 마이그레이션 도구
on-failure	비정상 종료 시	정상 종료될 수 있는 배치 작업, 테스트 컨테이너
always	항상	상시 구동 서비스
unless-stopped	사용자 수동 중지(docker stop) 재시작하지 않음	Stop 종료가 아닌경우는 Always와 동일

always 를 통해 재 시작 규칙

- 컨테이너의 메인 프로세스(PID 1)가 비정상 종료
- OOM, crash, SIGKILL
- Docker 데몬(dockerd) 재시작
- 서버 재부팅

unless-stop 를 통해 재 시작 규칙

- 컨테이너의 메인 프로세스(PID 1)가 비정상 종료
- OOM, crash, SIGKILL
- Docker 데몬(dockerd) 재시작
- stop이 아닌 상태에서 서버 재부팅

Health Check

컨테이너 동작 시 주기적으로 Health 상태를 확인해서 **상태를 healty, unhealthy 로 설정**
종료 및 복구 기능은 미지원 (kubernetes의 probe는 지원)

docker-compose.yaml

```
services:
  db:
    image: postgres:9.6.1
    healthcheck:
      test: "curl -f http://localhost:8080/health"
      interval: 10s
      retries: 5
      start_period: 10s
```

- **test**: 헬스 체크 명령
- **interval**: 체크 주기
- **timeout**: 명령 제한 시간
- **retries**: 연속 실패 허용 횟수
- **start_period**: 초기 기동 유예 시간

환경변수 주입 방법

environment 옵션 사용

- Key-Value Map 형태

```
services:
  web:
    image: nginx
    environment:
      # 방법 A: Key-Value 딕셔너리 형태 (추천: 가독성이 좋음)
      APP_ENV: production
      PORT: 8080
```

- List 형태

```
services:
  web:
    image: nginx
    environment:
      # 방법 B: List 형태 ("=" 기호 사용)
      - APP_ENV=production
      - PORT=8080
```

환경변수 주입 방법

env_file 옵션 사용

- 환경 변수 파일 (env.dev)

```
DB_HOST=localhost
DB_USER=admin
DB_PASS=secret123
```

- docker-compose.yaml 파일 경로 지정
  - 파일로 부터 여러개의 환경 변수 로딩 가능

```
services:
  backend:
    image: node:18
    env_file:
      - env.dev
```

기본 .env 환경 변수 파일 사용

- 환경 변수 파일 (.env)

```
PORT=3000
DB_PASSWORD=my_secure_password
```

- docker-compose.yaml에
  환경 변수로 지정

```
services:
  web:
    image: nginx
    ports:
      - "${PORT}:80"
    environment:
      DATABASE_PASS: ${DB_PASSWORD}
```

환경변수 주입 방법

-e 또는 --env 옵션으로 주입

```
docker compose -e  APP_ENV=prod up -d
```

${APP_ENV} 변수를 가진 service에서만 환경 변수 주입

```
services:
  web:
    image: nginx
    environment:
    - APP_ENV=${APP_ENV}
```

다른 파일로 주입 (환경 변수를 자동으로 주입)

명령어로 주입하기

```
docker compose --env-file .env.production up -d
docker compose --env-file config/my.env up -d
```

네트워크 호출

서비스 명으로 컨테이너간 호출하며, IP, localhost 등으로 호출 할 수 없음

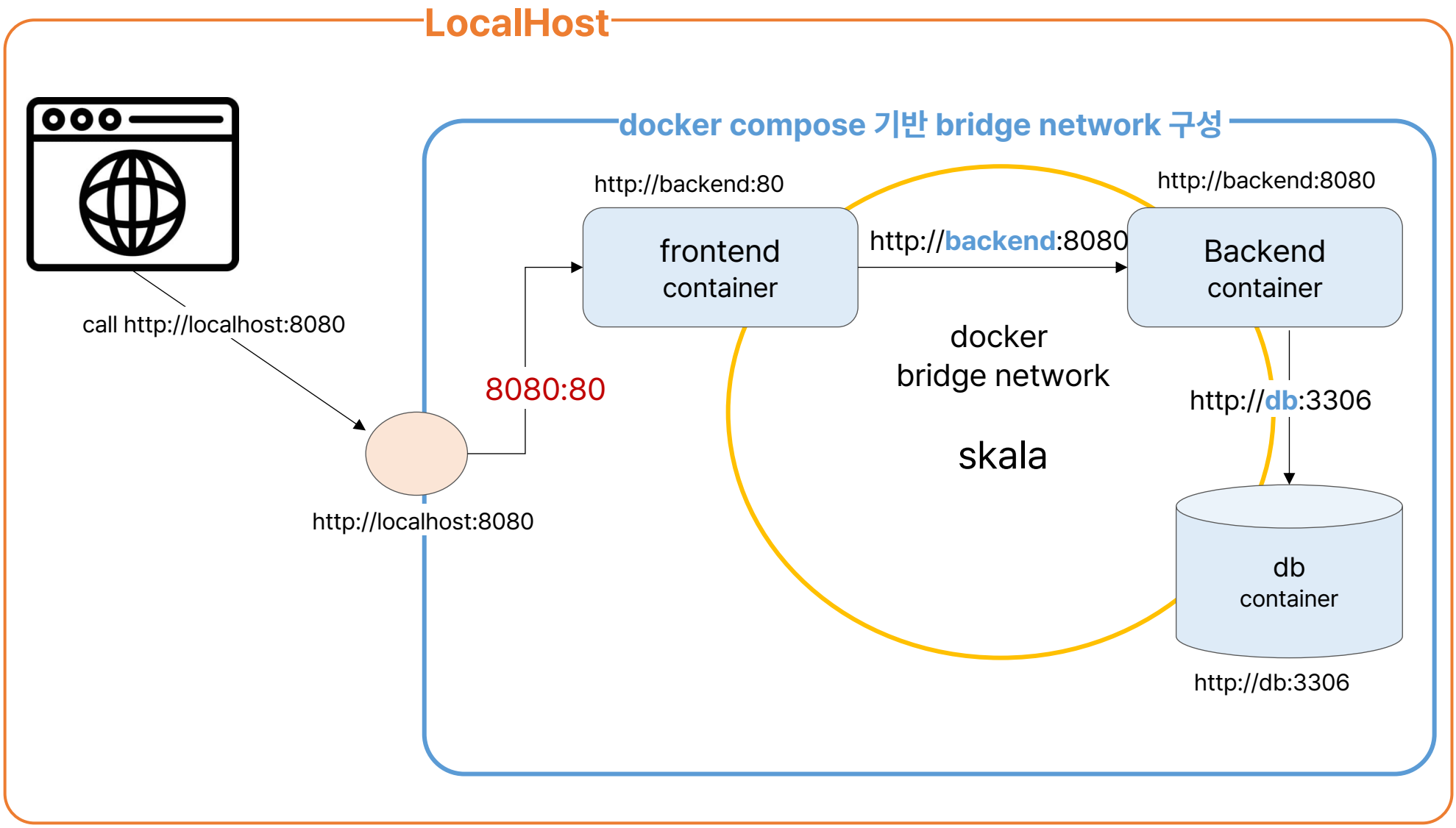
```yaml
services:
 frontend:
    build:
      context: ./frontend
    ports:
      - "8080:80"
 backend:
    build:
      context: ./backend
    expose:
      - 8080
 db:
    image: postgres:15
    environment:
      POSTGRES_PASSWORD: postgres
    restart: always
```

```
[ Browser ]
     |
     | http://localhost:8080
     v
[ Frontend 컨테이너 ]
     |
     | http://backend:8080
     v
[ Backend 컨테이너 ]
     |
     |
jdbc:postgresql://db:5432/postgres
     v
[ DB 컨테이너 ]
```

참고:
서비스 간 통신은 **service name**으로 호출하는 것이 권장
같은 Docker Compose 네트워크 내에서는 **컨테이너 IP**로도 호출 가능
하지만 컨테이너를 삭제하거나 재생성하면 IP가 변경될 수 있으므로,

네트워크 호출

서비스 명으로 컨테이너간 호출하며, IP, localhost 등으로 호출 할 수 없음



서비스 외부 노출 방법

(1) ports: 로 호스트 포트에 공개 (가장 일반적)

```
services:
  frontend:
    image: my-frontend
    ports:
      - "8080:80"                    # Host:8080 → Container:80
```

- 외부(브라우저/다른 PC)에서 http://<host-ip>:8080 접근 가능
- 8080:80: **0.0.0.0**:8080 → container:80 과 동일 (0.0.0.0 은 외부 모든 IP 가능)

(2) 특정 IP에만 바인딩해서 부분 공개

```
services:
  frontend:
    image: my-frontend
    ports:
      - "127.0.0.1:8080:80"          # 로컬(호스트)에서만 접근 가능
```

- 같은 호스트에서만 접근 가능 (외부 PC를 통해 들어오는 호출은 접근 불가)

  → "외부 노출 차단(로컬 전용)"으로 많이 사용

서비스 외부 차단 방법 (1/2)

(1) Port 미지정시 외부 노출 포트 없음

```
services:
  db:
    image: postgres
    # ports: 없음
```

- 컨테이너는 내부 네트워크에서만 접근 가능
- 같은 compose 네트워크의 다른 서비스가 db:5432 로 접근

(2) 내부 서비스에만 알려주기 (3번과 동일)

```
services:
  backend:
    image: my-backend
    expose:
      - "8080"
```

- 외부 노출 불가
- 같은 네트워크의 다른 컨테이너가 접근할 포트 단순 명시
- 교육용/가독성용으로 이용

서비스 외부 차단 방법 (2/2)

(3) 네트워크를 분리를 통해 내부 서비스간 접근 통제

```
services:
  frontend:
    image: my-frontend
    ports:
      - "8080:80”
    networks:
      - public

  backend:
    image: my-backend
    networks:
      - public
      - private

  db:
    image: postgres:15
    networks:
      - private

networks:
  public:          → public이라는 네트워크 생성
  private:         → private이라는 네트워크 생성
     internal: true
```

1) networks 을 통해 정의된 public 과 private 은 docker 내부 통신에서 네트워크를 분리하는 역할 만 수행

- 외부 노출은 frontend 만 가능
- frontend는 backend와 통신 가능, **db와는 통신 불가**
- backend는 db와 통신 가능

2) 컨테이너 내부 → 외부로 나가는 것을 차단하는 방법

- internal: true (기본은 false)

3) 기 생성되어 있는 외부 Bridge 네트워크를 사용하는 방법

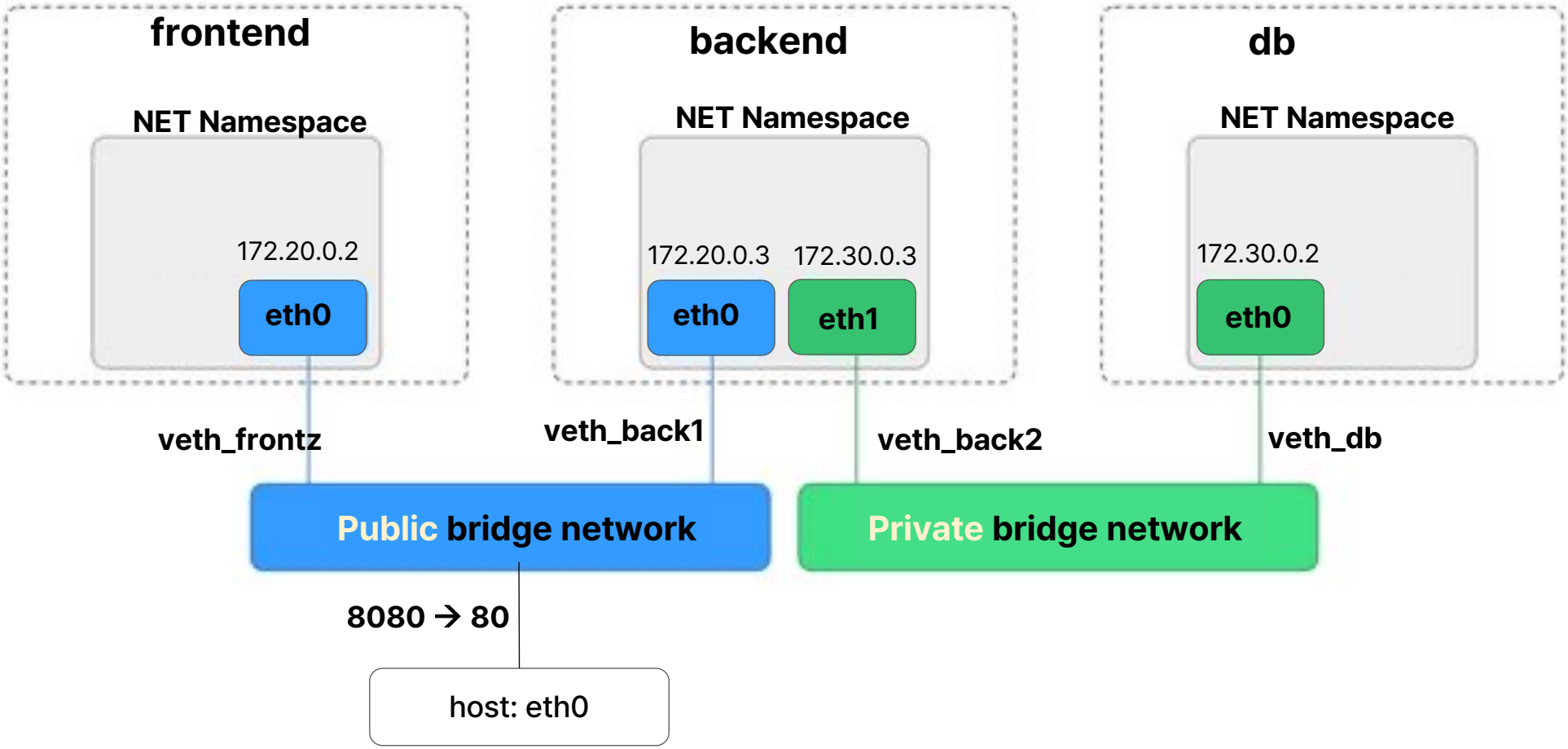
- external: true

예) docker create network my-network 후

```
networks:
  my-network:
    external: true     # 이것은 외부 네트워크를 사용
```

[참고] 네트워크 분리 구조

분리된 네트워크를 구성해서 DB Container 접속을 Backend에서만 접속 하도록 지원



서비스간 의존 관계 설정

depends_on은 컨테이너 시작 순서 제어

```yaml
services:
  db:
    image: postgres:9.6.1
    volumes:
      - ./docker/data:/var/lib/postgresql/data
    environment:
      POSTGRES_PASSWORD: root

    # 1단계: DB가 정상적으로 요청을 받을 준비가 되었는지 Check (정상 처리시 healthy 상태)
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U postgres"]
      interval: 5s       # 5초 간격으로 확인
      timeout: 5s        # 응답 타임아웃 5초
      retries: 5         # 5번 연속 실패 시 unhealthy 처리
      start_period: 10s  # 초기 구동 유예시간 10초

  backend:
    image: my-backend
    # 2단계: DB의 상태가 'service_healthy'가 될 때까지 백엔드 컨테이너 구동을 미룹니다.
    depends_on:
      db:
        condition: service_healthy
```

[실습] docker-compose 확장하기

목표 01.answer-code/10.docker-compose/01.base를 기반으로 db, backend, frontend를 확장했다. 이것을 지금까지 배운 내용으로 확장해보자

가이드

1. network를 public과 private으로 분리

① networks.private.internal: true를 추가해서 private 전용 bridge 구성
② db service는 private으로만 접속, backend는 public과 private 모두 사용 가능하도록 구성

2. db service 구성

① volumes을 추가해서 postgres db가 종료 후 다시 실행되더라도 기존 데이터가 유지되도록 구성
- ./db_data:/var/lib/postgresql/data

② db 서비스는 외부와 격리된 private 네트워크에만 연결
③ healthcheck를 backend에 추가해서 실행
- test: ["CMD", "pg_isready", "-U", "postgres"]
- 10초 주기, 대기 5초, 5번 반복하며, 컨테이너 시작 10초 뒤부터 진행

10.docker-compose/03.full/ 아래 참고 가능

[실습] docker-compose 실행하기

가이드

3. backend service

① 서비스 이름은 backend로 설정
② ./backend 디렉토리 내에 있는 Dockerfile.backend 파일을 지정해서 빌드 지정
③ build 블록을 만들고 다음을 채운다 — context, dockerfile
④ 포트 바인딩을 통해 외부 포트 : 9090, docker bridge port : 8080 지정
⑤ 네트워크는 외부 통신을 위해 public을 지정 (db와 통신을 위해 private도 추가)
⑥ 재시작 정책은 "죽으면 다시 살아나되, 내가 일부러 멈추면 그대로 멈춤" 이 목표
⑦ healthcheck를 backend에 추가해서 실행
- test: ["CMD", "wget", "-qO-", "http://localhost:8080/health"]
- 10초 주기, 대기 5초, 5번 반복하며, 컨테이너 시작 10초 뒤부터 진행
⑧ DB 서비스가 healthy 상태가 된 이후에 실행

[실습] docker-compose 실행하기

가이드

3. Frontend service

① 서비스 이름은 frontend로 설정
② ./backend 디렉토리 내에 있는 Dockerfile.frontend 파일을 지정해서 빌드 지정
③ build 블록을 만들고 다음을 채운다 — context, dockerfile
④ 네트워크는 외부 통신을 위해 public 지정
(backend도 public 지원하므로 Backend와도 통신 가능)
⑤ 포트 바인딩을 통해 외부 포트 : 8080, docker bridge port : 80 지정
⑥ volumes 추가: ./nginx.conf:/etc/nginx/conf.d/default.conf:ro
⑦ frontend 서비스는 backend 서비스가 healthy 상태가 된 이후에 실행
⑧ 재시작 정책은 "죽으면 다시 살아나되, 내가 일부러 멈추면 그대로 멈춤" 이 목표

[실습] docker-compose 실행하기

완성 구조

```
시작 순서: db (healthy) → backend (healthy) → frontend

네트워크:
  db       ↔ private
  backend  ↔ private + public
  frontend ↔ public
```

검증 명령어

```
# 전체 빌드 및  실행
docker compose up --build -d

# 서비스 상태 확인
docker compose ps

# 로그 확인
docker compose logs -f backend

# 헬스체크 확인
docker inspect <container_name> --format '{{json .State.Health}}'
```

[실습] docker-compose 실행하기

네트워크 확인

```
> docker network ls
NETWORK ID     NAME                 DRIVER    SCOPE
549f521bd787   03anwsers_private    bridge    local
55b8770bc0b4   03anwsers_public     bridge    local
f9040b5d647b   bridge               bridge    local
bf307fd8d9c7   host                 host      local
3bd1eb6f2217   none                 null      local

> docker network inspect 03anwsers_public
  # 03anwsers-backend-1 의 Container IP 확인하기

> docker network inspect 03anwsers_private
  # 03anwsers-backend-1 의 Container IP 확인하기
```

Browser 로 접속해보기

- localhost:8080
- localhost:8080/heath
- localhost:8080/users



KDT Cloud – 3. Container의 이해

컨테이너 관련 도구들

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

이들의 이름?



VS



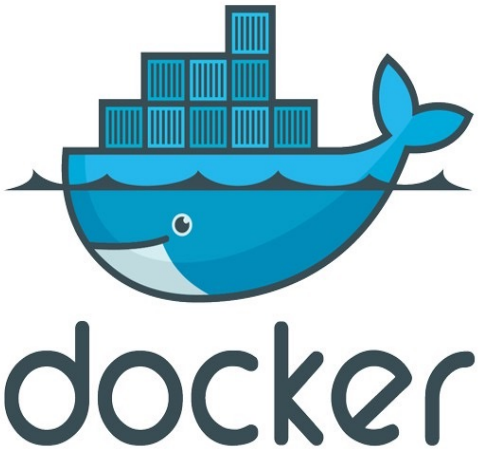


VS



통합 컨테이너 플랫폼

이미지 빌드, 컨테이너 실행, 네트워크, 볼륨 관리 등을 한번에 제공



VS



podman



RANCHER DESKTOP



Colima

컨테이너 이미지 빌드 도구

Dockerfile 등을 읽어 OCI 이미지를 만드는 역할 수행

BuildKit





buildah

도구	실행 방식	주요 용도
Docker Build	Docker 데몬 사용	일반적인 로컬 이미지 빌드
BuildKit	BuildKit 데몬 또는 독립 실행	고속·병렬·캐시 기반 빌드
Docker Buildx	BuildKit을 사용하는 Docker 플러그인	멀티 아키텍처 이미지 빌드
Kaniko	컨테이너 안에서 빌드	Kubernetes CI 환경
Buildah	데몬 없이 이미지 빌드	Podman 생태계, Rootless 빌드
Buildpacks	Dockerfile 없이 이미지 빌드	애플리케이션 중심 자동 빌드
Jib	Java 애플리케이션 전용	Maven·Gradle에서 바로 이미지 생성
Bazel	빌드 시스템 기반	대규모 프로젝트와 재현 가능한 빌드
img	Rootless BuildKit 기반	데몬 없는 Dockerfile 빌드

컨테이너 런타임

고수준 컨테이너 런타임

이미지를 가져오고, 컨테이너 생명주기를 관리하며, 저수준 런타임 호출

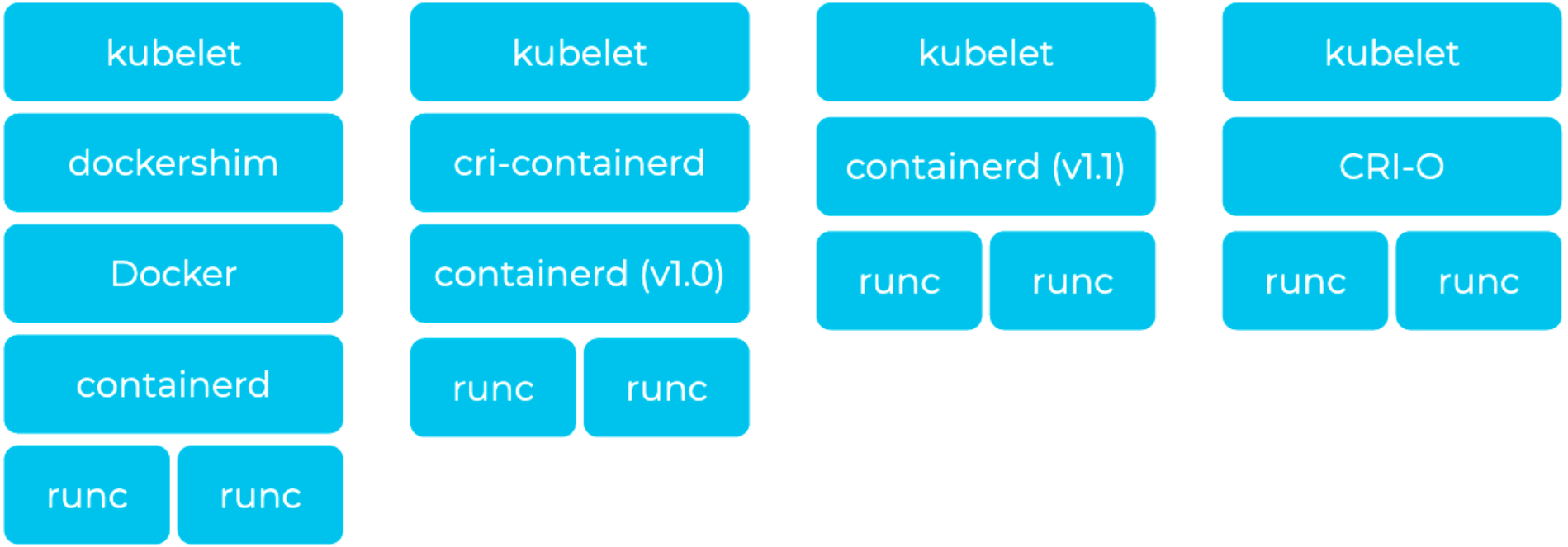
런타임	주요 특징
containerd	Docker와 Kubernetes에서 널리 사용
CRI-O	Kubernetes 전용으로 설계된 경량 런타임
Docker Engine	dockerd가 containerd를 통해 컨테이너 관리

저수준 컨테이너 런타임

실제로 Linux namespace, cgrup을 설정하고 컨테이너 프로세스를 실행

런타임	주요 특징
runc	가장 대표적인 OCI 저수준 런타임
crun	C 언어 기반, 빠르고 경량
youki	Rust 기반 OCI 런타임
runsc	gVisor의 런타임, 보안 격리 강화
kata-runtime	경량 VM 기반 격리

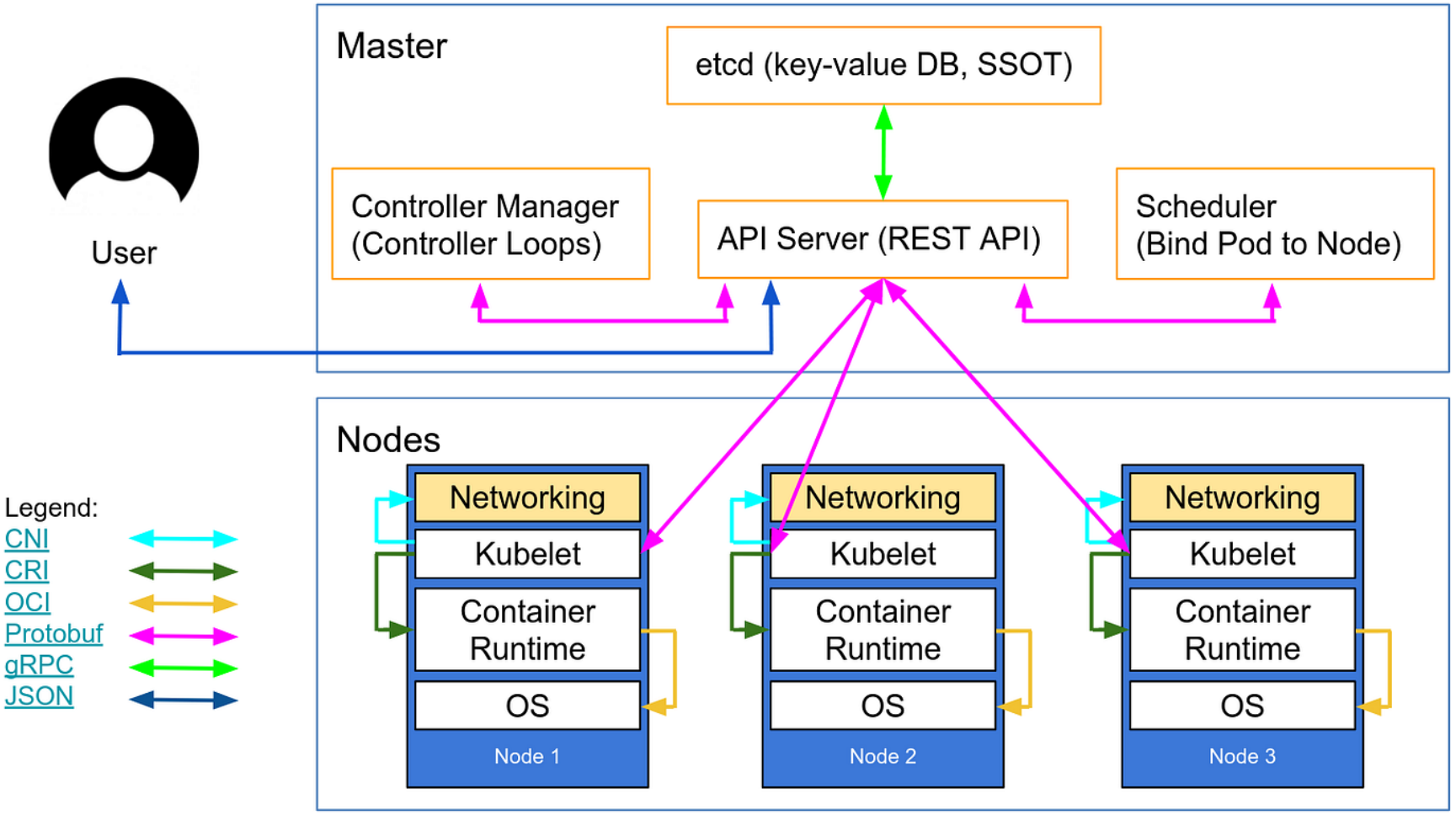
kubernetes 내 container 실행 프로세스



컨테이너 오케스트레이션 플랫폼

컨테이너를 직접 빌드하거나 실행하는 도구라기보다는

여러 서버에 분산된 컨테이너를 배포하고 운영하는 플랫폼





KDT Cloud – 3. Container의 이해

웹 서비스의 docker-compse화

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C

[종합 실습] 웹 서비스 docker-compose로 전환해보기

자신만의 API Server (spring or fastapi etc) 과 Frontend 그리고 Database를 만들고
이것을 docker-compose로 구조화해서 실행해보자

실습 목표

- 자신의 Frontend, Backend, DB 등에 대한 Dockerfile 구성하기
- docker-compose.yaml 정의하기
- 로컬에서 배포 및 실행해서 동작 확인하기

스스로 확인해보기

- 웹서비스가 어떻게 구성되어서 호출되거 연결되는지 아키텍처를 정리해보자
- docker-compose.yaml 내용 확인
- 실제 실행 결과 확인해보기
- 전체 container network 구성을 확이해보기
- 컨테이너 간 호출 흐름을 확인해보기

[종합 실습] 웹 서비스 docker-compose로 전환해보기

직접 코드를 만들기 어려운 경우에는

[실습]나의 서비스 컨테이너 만들기에서 작업했던 Frontend, Backend, DB를 활용해서 실행



수고하셨습니다.

본 문서는 SK㈜C&C의 컨텐츠 자산으로, 무단 사용 및 불법 배포 시 법적 조치를 받을 수 있습니다.

SK 주식회사 C&C